附录 3：PO 算法的详细信息

算法 1. Direct Preference Optimization (DPO)

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，参考策略π_{ref}，温度参数β，剪裁参数ϵ

输出：更新策略π_θ

1) 数据格式：偏好数据集D由三元组(x,y_w,y_l)构成，其中x代表输入提示，y_w表示首选响应，而y_l则指代拒绝响应。

2) 基于 Bradley-Terry 模型的偏好概率：假设偏好概率遵循该模型，则y_w优于y_l的概率可表示为公式（4）所示：

$$p(y_w \succ y_l|x)=\frac{\exp(R(y_w,x))}{\exp(R(y_w,x))+\exp(R(y_l,x))}. \tag{4}$$

3) 奖励参数化：正如公式（5）所展示的，DPO 通过最优策略的闭式表达式对奖励函数$R(y,x)$进行重新参数化：

$$R(x,y)=\beta \log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}+\beta \log Z(x), \tag{5}$$

其中，β作为温度参数，调控探索与利用之间的平衡关系；而$Z(x)$则是为确保奖励函数在数学上的一致性而引入的归一化常数。

4) 目标函数：采用布拉德利-特里模型对奖励参数进行参数化处理，进而确立 DPO 的优化目标，具体如公式（6）所示：

$$L_{DPO}(\theta)=-E_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right], \tag{6}$$

其中，$L_{DPO}(\theta)=-\log p(y_w \succ y_l|x)$被定义为负对数似然损失函数，其作用等同于最大化似然函数；$\sigma(z)=\frac{1}{1+e^{-z}}$是一种 sigmoid 型函数，其输出值域限定在（0,1）区间内，能够有效将策略概率比的差异转化为偏好概率。

这种损失促使策略提高偏好结果y_w的概率，同时降低非偏好结果y_l的概率，具体如公式（7）所示：

$$p(y_w \succ y_l|x)=\frac{1}{1+e^{R(y_l,x)-R(y_w,x)}}=\sigma(R(y_w,x)-R(y_l,x)). \tag{7}$$

算法 2. Multimodal Direct Preference Optimization (m-DPO)

输入：图像m，问题q，优选回答y_w，拒绝回答y_l，策略模型π_θ，参考模型π_{ref}，超参数β,δ。

输出：更新策略 π_θ

标准 DPO 目标如（8）所示，条件偏好优化目标如（9）所示，奖励锚定目标如（10）所示，从而得到总目标函数如（11）所示：

$$\mathrm{L}_{DPO_m}=-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|m,q)}{\pi_{ref}(y_w|m,q)}-\beta\log\frac{\pi_\theta(y_l|m,q)}{\pi_{ref}(y_l|m,q)}\right), \tag{8}$$

其中，m为图像，q为问题，y_w和y_l分别为优选和拒绝回答，π_θ为待优化模型，π_{ref}为参考模型，β为控制偏离参考模型的超参数，σ为 sigmoid 函数。

$$L_{CDPO}=-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|m_w,q)}{\pi_{ref}(y_w|m_w,q)}-\beta\log\frac{\pi_\theta(y_w|m_l,q)}{\pi_{ref}(y_w|m_l,q)}\right), \tag{9}$$

其中，m_w为原始图像，m_l为通过裁剪等方式构造的视觉信息较少的负样本图像，强制模型学习图像与回答之间的依赖关系。

$$L_{AncPO}=-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|m_w,q)}{\pi_{ref}(y_w|m_w,q)}-\delta\right), \tag{10}$$

其中，δ为锚定值，用于确保优选回答的奖励值为正，避免其概率下降。

$$L_{MDPO}=L_{DPO_m}+L_{CDPO}+L_{AncPO}. \tag{11}$$

m-DPO 在多模态偏好对齐方面展现出显著优势，尤其是在减少幻觉和提升图像条件感知能力方面表现尤为突出。其独特的条件偏好机制有效缓解了 DPO 在跨模态任务中常见的“视觉忽略”问题，而奖励锚定策略则进一步稳定了训练过程。然而，引入 m-DPO 也带来了额外的负样本构造和超参数设置需求，增加了实现的复杂度。

算法 3. Multimodal-Model-Guided Preference Optimization（M3PO）

输入：图像I，文本指令Q，基础大型视觉语言模型，预训练的多模态评估模型π_θ，M3P-Score 超参数α和δ。

输出：更新策略π_θ

1) 候选回答生成：针对给定的图像-指令对(I, Q)，借助基础 LVLM 生成N个多样化的候选回答$\{R_1,R_2, ..., R_n\}$。
2) 多模态对齐分数计算：采用一个独立的外部视觉语言评估模型，计算每个候选回答R_i的 MAS 分数，记作$MAS(R_i|I,Q)$。该分数全面评估回答的视觉相关性、语义准确性和指令遵循程度。
3) 模型自一致性/置信度计算：计算基础 LVLM 生成每个候选回答R_i的对数概率，记作$\log P(R_i|I,Q)$，以衡量模型对该回答的内部置信度。
4) M3P-Score 与偏好对选择：
   - 首选回答：选择 MAS 分数最高的候选回答作为首选回答R_w，具体如公式（12）所示：

$$R_w=\arg\max_{R_i\in\{R_1,\ldots,R_N\}} MAS(R_i|I,Q), \tag{12}$$

   其中，R_w表示被选中的首选回答。该回答是从所有候选回答R_1到R_n中，使得多模态对齐分数$MAS(R_i|I,Q)$取得最大值的那一个。

   - 次选回答：从剩余候选回答中，选择一个能最大化 M3P-Score 的R_j作为次选回答R_l。M3P-Score 定义如（13）所示：

$$S(R_j)=(MAS(R_w|I,Q)-MAS(R_j|I,Q))-\alpha\cdot\max\left(0,\log P(R_w|I,Q)-\log P(R_j|I,Q)-\delta\right), \tag{13}$$

   其中，$MAS(R_w|I,Q)-MAS(R_j|I,Q)$度量候选回答$R_j$与首选回答$R_w$在多模态对齐质量上的差距。差距越大，表明$R_j$的质量越低，此项正值越大，越倾向于将其选为次选；$\log P(R_w|I,Q)-\log P(R_j|I,Q)$度量$R_j$与$R_w$在模型生成置信度（以对数概率表示）上的差距；$\delta$为边际参数，定义置信度差异的容忍范围；$\alpha$为权重超参数，控制置信度差异惩罚项在整个分数中的影响程度；$\max(0,...-\delta)$是一个惩罚项。如果R_j的置信度显著低于R_w（差值

超过 δ），则施加惩罚；如果R_j的置信度接近甚至高于R_w（差值在 δ 范围内），则惩罚项为零或变为负值（相当于奖励），从而鼓励选择那些模型自信生成但实际质量较差的“困难负样本”。

- 次选回答的选取公式：次选回答的选取公式具体如公式（14）所示：

$$R_l = arg \max_{R_j \in \{R_1,\ldots,R_N\} \setminus \{R_w\}} S(R_j), \tag{14}$$

其中，R_l表示被选中的次选回答。它是从所有候选回答中除去首选回答R_w后的集合里，使得*M3P-Score* $S(R_j)$取最大值的那一个。

M3PO 的核心优势在于其卓越的数据效率和性能提升能力。该技术通过深入挖掘模型自身的“认知盲区”（即那些自信却错误的回答），精心构建出更具挑战性的训练数据集，进而引导模型培养出更为精细的区分能力。然而，该方法成效在很大程度上依赖于外部评估模型的准确性与可靠性，这可能会不可避免地引入一定的噪声或偏差。

算法 4. Direct Semantic Preference Optimization（DSPO）

输入：低分辨率图像x_{LQ}，预训练的 SR 模型，SAM 模型，VLM 模型，人类偏好数据集。

输出：经过 DSPO 优化后的 SR 模型

1) 语义实例对齐策略：
   - 使用 SAM 模型对 SR 输出图像进行实例分割，得到多个语义区域s_m；
   - 对每个语义实例进行 Best/Worst-of-N 选择，得到偏好对(x_w,x_l)；
   - 引入实例面积权重$w_m=\frac{|s_m|}{\sum_{m=1}^{M}|s_m|}$，优化目标如公式（15）所示：

$$\max_{\theta} E_{(x_{LQ},x_w,x_l)} \sum_{m=1}^{M} w_m \cdot \log \sigma \left(\beta \log \frac{p_\theta(x_w|x_{LQ},s_m)}{p_{ref}(x_w|x_{LQ},s_m)} - \beta \log \frac{p_\theta(x_l|x_{LQ},s_m)}{p_{ref}(x_l|x_{LQ},s_m)} \right). \tag{15}$$

2) 损失函数设计：在扩散模型框架下，将 DPO 目标转化为噪声预测误差形式，具体如公式（16）和（17）所示：

$$L_{SR} = -E_{(x_{LQ},x_w,x_l),t} \sum_{m=1}^{M} w_m \log \sigma \left[-\beta T \left(L_\theta^{diff}(x_w,t,s_m) - L_{ref}^{diff}(x_w,t,s_m) - \left(L_\theta^{diff}(x_l,t,s_m) - L_{ref}^{diff}(x_l,t,s_m) \right) \right) \right], \tag{16}$$

$$L_{diff}(x,t,s_m) = \gamma(\lambda_t) \| s_m \cdot (\epsilon - \epsilon_\theta(x,t|x_{LQ})) \|^2, \tag{17}$$

其中，ϵ_θ为模型预测噪声，ϵ为真实噪声，$\gamma(\lambda_t)$为权重函数。

DSPO 作为首个在超分辨率任务中系统性地引入人眼偏好对齐的方法，展现出显著的创新性，尤其在实例级语义对齐和幻觉抑制方面表现尤为突出。然而，该方法依赖于 SAM 和 VLM 等外部模型，导致计算开销较大，并且偏好数据的构建成本亦较高。

算法 5. Symmetric Multimodal Preference Optimization（SymMPO）

输入：原始图像m和文本提示x；对比图像m'（与m视觉相似）；偏好响应对(y_w,y_l)和(y_w',y_l')；参考模型π_{ref}和策略模型π_θ；超参数$\beta,\delta,\lambda,\gamma,\eta$。

输出：更新后的策略π_θ

1) 对称成对偏好学习：通过使用对比图像m'及其对应的偏好响应y_w'，构建对称的偏好对，优化目标如公式（18）所示：

$$L_{Pair}=-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|m,x)}{\pi_{ref}(y_w|m,x)}-\beta\log\frac{\pi_\theta(y_w'|m,x)}{\pi_{ref}(y_w'|m,x)}\right)-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w'|m',x)}{\pi_{ref}(y_w'|m',x)}-\beta\log\frac{\pi_\theta(y_w|m',x)}{\pi_{ref}(y_w|m',x)}\right), \tag{18}$$

其中，π_θ为策略模型，π_{ref}为参考模型，β为控制偏离参考模型的超参数，σ为 sigmoid 函数。

2) 偏好间隔一致性正则化：对对称响应对之间的偏好间隔一致性进行约束，具体方法如公式（19）所示：

$$L_{Margin}=\left(\Delta(m,x,y_w,y_w')-\Delta(m',x,y_w',y_w)\right)^2, \tag{19}$$

其中，$\Delta(m,x,y_w,y_w')=\beta\log\frac{\pi_\theta(y_w|m,x)}{\pi_{ref}(y_w|m,x)}-\beta\log\frac{\pi_\theta(y_w'|m,x)}{\pi_{ref}(y_w'|m,x)}$表示偏好间隔。

3) 锚定偏好正则化：采用锚定偏好正则化，如公式（20）所示，以防止偏好响应的概率下降：

$$L_{AncPO}=-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|m,x)}{\pi_{ref}(y_w|m,x)}-\delta\right)-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w'|m',x)}{\pi_{ref}(y_w'|m',x)}-\delta\right), \tag{20}$$

其中，δ为锚定阈值。

4) 总目标函数：总目标函数的定义如公式（21）所示：

$$L_{SymMPO}=L_{DPO_m}+\lambda L_{Pair}+\gamma L_{Margin}+\eta L_{AncPO}, \tag{21}$$

其中，λ,γ,η为权重超参数。

SymMPO 在理论一致性和监督信号直接性方面显著超越现有方法，能够有效缓解 MLLM 的幻觉问题，并在多个基准测试中展现出卓越性能。然而，该方法在细粒度视觉理解任务上仍存在局限性，且由于需要为每个对比图像生成偏好响应，导致额外的计算开销。

算法 6. Video Hierarchical Spatial-Temporal Direct Preference Optimization（VistaDPO）

输入：视频v，文本提示x，优选响应y_w，非优选响应y_l。

输出：更新后的策略π_θ

1) 实例级语义偏好优化：

- 响应级对齐L_{DPO_v}：通过区分优选响应y_w与非优选响应y_l，优化模型对用户意图的理解；
- 视频级对齐L_{DPO_r}：引入视频层面的正负样本对(v_w^c,v_l^c)，减少模型对语言的过度依赖，具体如公式（22）所示：

$$L_{DPO_r}=-E[\log\sigma(u_r)],\quad u_r=\beta\log\frac{\pi_\theta(y_w|v,x)}{\pi_{ref}(y_w|v,x)}-\beta\log\frac{\pi_\theta(y_l|v,x)}{\pi_{ref}(y_l|v,x)}, \tag{22}$$

其中，β为 KL 惩罚系数，π_θ为目标模型，π_{ref}为参考模型。

2) 时序语义偏好优化：

- 片段级对齐L_{DPO_c}：将视频中与提示相关的时间段作为优选片段v_w^c，不相关的作为非优选片段v_l^c，具体如公式（23）所示：

$$\mathrm{L}_{DPO_c} \sim log\,\sigma(u_c),\quad u_c=\beta\, log\frac{\pi_\theta(y_w|v_w^c,x)}{\pi_{ref}(y_w|v_w^c,x)}-\beta\, log\frac{\pi_\theta(y_l|v_l^c,x)}{\pi_{ref}(y_l|v_l^c,x)}, \tag{23}$$

3) 感知级空间-对象偏好优化：

- 对象级对齐L_{DPO_o}：选取关键帧v_w^f作为正样本，对其关键区域进行掩码或移动构造负样本v_l^f；
- Token 级对齐L_{DPO_t}：通过序列 KL 散度实现细粒度语义对齐。具体的损失函数如公式（24）所示：

$$\mathrm{L}_{DPO_t}=sg\left(\beta D_{SeqKL}(y_w)\right)-\beta D_{SeqKL}(y_l), \tag{24}$$

其中，$D_{SeqKL}=\sum_{t=1}^{T} D_{KL}(\pi_{ref}(y^t|y^{<t})\|\pi_\theta(y^t|y^{<t}))$。

4) 损失函数：总损失函数如公式（25）所示：

$$\mathrm{L}_{VistaPO}=\mathrm{L}_{DPO_v}+\mathrm{L}_{DPO_r}+\lambda\mathrm{L}_{DPO_c}+\mu\mathrm{L}_{DPO_o}+\rho\mathrm{L}_{DPO_t}, \tag{25}$$

其中，λ,μ,ρ为各层损失的权重。

VistaDPO 通过层次化的时空对齐机制，显著增强了视频语言模型的语义理解能力和抗幻觉能力，尤其在时序推理和空间对象感知方面表现尤为出色。然而，该模型在处理长视频中的复杂时序依赖方面仍存在不足，且高度依赖高质量的标注数据以及复杂的负样本构造策略，导致计算成本较高。

算法 7. Hybrid-Modal Preference Optimization（HIPPO）

输入：表格T；问题Q；任务指令$Instruct_z$；文本表示$L(T)$；图像表示$V(T)$。

输出：模型生成的回答y

1) 混合模态响应采样：模型依据三种表格表示分别生成响应，具体如公式（26）、（27）和（28）所示：

$$\tilde{y}_l \sim \mathrm{M}(Instruct_z,Q,L(T)), \tag{26}$$

$$\tilde{y}_v \sim \mathrm{M}(Instruct_z,Q,V(T)), \tag{27}$$

$$\tilde{y} \sim \mathrm{M}(Instruct_z,Q,L(T),V(T)), \tag{28}$$

其中，$\tilde{y}_l$、$\tilde{y}_v$、$\tilde{y}$分别表示基于文本、图像和多模态表示的响应，$L(T)$是文本表示，$V(T)$是图像表示，$Instruct_z$是任务指令。

2) 模态一致性负例选择：从所有错误响应集合$\tilde{Y}_{Neg}$中挑选出现频率最高的作为负例，具体过程如公式（29）所示：

$$\tilde{y}^- = arg\,\max_{\tilde{y}\in \tilde{Y}_{Neg}} Freq(\tilde{y}). \tag{29}$$

3) 损失函数：HIPPO 的损失函数如公式（30）所示：

$$\mathrm{L}=-E_D\left[log\,\sigma\left(\beta\, log\frac{\mathrm{M}(\tilde{y}^+|Q,T)}{\mathrm{M}^{ref}(\tilde{y}^+|Q,T)}-\beta\, log\frac{\mathrm{M}(\tilde{y}^-|Q,T)}{\mathrm{M}^{ref}(\tilde{y}^-|Q,T)}\right)\right], \tag{30}$$

其中，M表示待优化模型，M^{ref}表示参考模型（固定），β是控制偏好强度的超参数，σ是 Sigmoid 函数，D是由四元组$(Q,T,\tilde{y}^{+},\tilde{y}^{-})$构成的训练集。

HIPPO 通过多模态融合与偏好优化，显著提高了表格理解的准确性和鲁棒性，尤其在处理复杂结构和视觉信息方面表现尤为出色。然而，其依赖多模态输入的特性也导致计算和存储开销增加，并且在仅包含单一模态的实际应用场景中可能面临局限性。

算法 8. Perceptual Preference Optimization（PerPO）

输入：图像x，问题或指令，参考模型π_{ref}，待优化模型π_{θ}，判别奖励函数f，超参数β,γ。

输出：更新后的策略π_{θ}

1) 判别性奖励定义：对于每个样本对(x,y_i)，其判别性奖励得分定义如公式（31）所示：

$$\widehat{R}_i=f(x,y_i), \tag{31}$$

其中，f是用于衡量模型输出y_i与真实标签之间差异的函数。

2) 奖励权重计算：针对一对样本(y_i,y_j)，其权重定义如公式（32）所示：

$$w_{ij}=\frac{(\widehat{R}_i-\widehat{R}_j)^{\gamma}}{\sum_{\widehat{R}_i>\widehat{R}_j}(\widehat{R}_i-\widehat{R}_j)^{\gamma}}, \tag{32}$$

其中，γ是缩放因子，用于调节奖励差异的影响。

3) PerPO 损失函数：PerPO 损失函数的具体形式如公式（33）所示：

$$\mathrm{L}_{PerPO}(\theta)=-E_{(x,Y)\sim D}\left[\sum_{R_i>R_j} w_{ij}\log\sigma(\beta(R_i-R_j))\right], \tag{33}$$

其中，$R_i=\log\frac{\pi_{\theta}(y_i|x)}{\pi_{ref}(y_i|x)}$代表隐式奖励得分；$\beta$是用于调节偏好强度的超参数；$\sigma$指的是 sigmoid 函数；$Y$则是模型生成的一组响应。

PerPO 通过引入判别性奖励机制和列表化优化策略，显著提升了 MLLM 在视觉判别任务中的性能，同时有效降低了对图像无关奖励的依赖。然而，其效能仍受限于模型的基础能力，并且在数据质量和多样性不足的情况下，其效果可能受到制约。

算法 9. Fine-grained Direct Preference Optimization（FDPO）

输入：多模态输入$T=(I,Q,P)$，包括图像I、文本查询Q、视觉提示P；优选响应R_p与次选响应R_l。

输出：更新策略π_{θ}

1) 响应分段：将完整响应R划分为描述段R_{desc}和推理段R_{reason}。

2) 段落偏好差分：段落偏好差分计算如公式（34）所示：

$$\Delta R_s=score(R_s^p)-score(R_s^l),\quad s\in\{desc,reason\}, \tag{34}$$

其中，R_s^p和R_s^l分别为优选和次选响应段，$score(\cdot)$是由视觉、空间和逻辑奖励组成的综合评分。

3) 动态权重调整：具体如公式（35）所示：

$$w_s=\frac{exp(\lambda\cdot\Delta R_s)}{exp(\lambda\cdot\Delta R_{desc})+exp(\lambda\cdot\Delta R_{reason})}, \tag{35}$$

其中，λ控制着权重对偏好差分的敏感程度。

4) 段落特定优化参数：段落特定优化参数如公式（36）所示：

$$\beta_s=\beta[1+\alpha(2w_s-1)], \tag{36}$$

其中，β为基础参数，α控制调整幅度，w_s为段落权重。

5) 优化目标：优化目标具体如公式（37）所示：

$$L_{FDPO}(\theta)=-E_{(T,R^p,R^l)}\left[log\,\sigma\left(\beta_{desc}F_{desc}+\beta_{reason}F_{reason}\right)\right], \tag{37}$$

其中，F_s表示段落s在策略π_θ和参考策略π_{ref}下的对数几率差。

FDPO 通过细粒度分段优化显著提升了模型在复杂空间推理任务中的表现，尤其在多步逻辑推断和空间对齐方面优于传统 DPO。然而，该方法依赖于高质量的分段标注与多模型协同生成数据，计算成本较高，且对超参数敏感。

算法 10. MMSafe-PO

输入：多模态指令x；参考模型π_{ref}；优选响应y_w；拒绝响应y_l；盲化指令x_b（仅文本）及其响应y_b。

输出：更新策略π_θ

对于每一个多模态指令x，构建一个“盲化”版本x_b（即去除图像部分）。借助参考模型π_{ref}生成盲化响应y_b，并结合原始偏好对(y_w,y_l)以及新增偏好对(y_w,y_b)进行优化。从而得到 MMSafe-PO 的损失函数如公式（38）所示：

$$L_{BPO}=-E_{(x,y_w,y_l,y_b)}\left[log\,\sigma(r(x,y_w)-r(x,y_l))+log\,\sigma(r(x,y_w)-r(x,y_b))\right], \tag{38}$$

其中，x代表多模态指令（包含文本和图像），y_w为人工选定的优选响应，y_l为人工拒绝的响应，y_b为经过盲化处理后的模型生成响应，$r(x,y)$为奖励函数，该函数由策略模型与参考模型的对数比所表示，σ为 sigmoid 函数。

MMSafe-PO 通过引入真实人类反馈和模态转换机制，有效提升了 MLLM 的安全对齐能力，尤其在缓解语言主导的“模态欺骗”方面表现突出。然而，该方法依赖从文本数据转换而来，未能完全模拟真实多模态交互场景，且“盲化”机制在纯语言模型中的适用性仍有待验证。

算法 11. Listwise Preference Optimization（LPOI）

输入：图像I，问题/指令q，正文本响应w^+，负文本响应w^-（源自原始偏好对），基础 VLM 模型π_{ref}。

输出：更新策略π_θ

1) 关键对象识别与掩码：利用目标检测模型，精准定位图像中与文本关联度最高的对象，并据此生成相应的掩码区域。

2) 列表样本生成：通过在正样本（完整图像）与负样本（关键对象完全掩码的图像）之间进行插值处理，生成L个图像。其中，第k个图像的掩码比例计算公式为$\frac{(k-1)}{(L-1)}\times 100\%$。

3) 列表排序损失：采用如（39）和（40）所示的损失函数对模型进行训练，以确保模型对图像序列的偏好排序与对象的可见性程度保持一致：

$$\mathrm{L}_{Listwise}(\theta)=-\log\left(\prod_{k=1}^{L}\frac{exp\,(S_k)}{\sum_{j=k}^{L}exp\,(S_j)}\right), \tag{39}$$

$$S_k=\beta\log\frac{\pi_\theta(w|x_k,q)}{\pi_{ref}(w|x_k,q)}, \tag{40}$$

其中，π_θ代表待优化的 VLM 模型，π_{ref}则为参考模型，x_k为第k个插值图像，w表示正文本响应，β为温度超参数，L表示列表长度。

4) 总损失：结合 DPO 损失、Anchor 损失和列表损失，具体如公式（41）所示：

$$\mathrm{L}_{Total}=\mathrm{L}_{DPO}+\mathrm{L}_{Anchor}+\mathrm{L}_{Listwise}. \tag{41}$$

LPOI 的优势在于无需额外标注即可构建高质量列表样本，从而有效提升模型对视觉细节的敏感度，显著减少幻觉现象。然而，该方法依赖于目标检测模型的准确性，且列表生成与训练过程成本较高，这可能会限制其在资源受限场景中的实际应用。

算法 12. Mix Preference Optimization（MPO）

输入：超参数$\lambda_1,\lambda_2,\lambda_3,\beta$

输出：更新策略π_θ

1) 混合奖励设计：MPO 的奖励函数由多个奖励信号的加权组合构成，具体如公式（42）所示：

$$R_{MPO}(x,y)=\lambda_1 r_{final}(x,y)+\lambda_2 r_{process}(x,y)+\lambda_3 r_{task}(x,y), \tag{42}$$

其中，r_{final}是 Skywork-VL 奖励系统为最终输出所提供的分数，$r_{process}$是过程监督奖励，r_{task}是任务特定奖励；λ_1、λ_2、λ_3是用于调节各项奖励权重的超参数。

2) 优化目标：MPO 的训练以 DPO 为基础，其优化目标如公式（43）或公式（44）所示：

$$L_{RL\text{-}MPO}=E_{(x,y)\sim\pi_\theta}\left[R_{MPO}(x,y)-\beta\cdot KL(\pi_\theta(y|x)\|\pi_{ref}(y|x))\right], \tag{43}$$

$$L_{DPO\text{-}MPO}=-E_{(x,y^+,y^-)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)}-\beta\log\frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)}\right)\right]. \tag{44}$$

算法 13. Modality-Balancing Preference Optimization（MBPO）

输入：图像I，指令q，优选回答o_w；闭端问题数据集（包含真实答案）；初始模型π_θ，参考模型π_{ref}。

输出：更新策略π_θ

1) 离线数据构建：采用对抗攻击方法生成扰动图像I_{adv}，利用I_{adv}生成负样本$o_l\sim\pi_\theta(\cdot|q,I_{adv})$，旨在最小化正确回答的概率。具体过程如公式（45）所示：

$$I^{(t+1)}=\Pi_{B_\epsilon(I)}\left(I^{(t)}+\alpha\cdot sign\left(\nabla_I\left(-\log p_\theta(o_w|q,I)\right)\right)\right), \tag{45}$$

其中，o_w为优选回答，α为步长，ϵ为扰动范围，Π为投影操作。

2) 在线数据构建：从闭端问题中进行采样，生成多个回答，并根据答案的正确性分配奖励。

3) 混合优化目标：通过 GRPO 损失函数联合优化离线与在线数据，具体方法如公式（46）所示：

$$J_{MBPO}(\theta)=\frac{1}{B}\left\{\sum_{off} GRPO\text{ 项}+\sum_{on} GRPO\text{ 项}-\beta D_{KL}(\pi_\theta\|\pi_{ref})\right\}, \tag{46}$$

其中，B表示批次大小，β为 KL 散度系数，π_{ref}代表参考策略，GRPO 项涵盖了优势函数A_j及策略比率裁剪。

MBPO 通过引入对抗负样本和在线奖励机制，有效缓解了模态不平衡问题，显著提升了模型的视觉依赖性和任务表现，并在多个基准测试中大幅降低了幻觉现象。然而，该方法目前仍局限于图像-文本模态，尚未扩展至视频、音频等多模态场景，且对抗样本生成和在线解码过程增加了计算开销。

算法 14. SDPO

输入：条件c，偏好对(x_0^w,x_0^l)，参考模型π_{ref}，采样分布q，超参数β,ϵ。

输出：更新策略π_θ

1) 重要性权重定义：重要性权重w_θ用于评估当前模型分布与采样分布之间的差异，如（47）所示：

$$w_\theta(x_0|c)=\frac{\pi_\theta(x_0|c)}{q(x_0|c)}, \tag{47}$$

其中，π_θ是当前模型分布，q是采样分布，x_0是生成样本，c是条件。

2) 裁剪与共享权重：为避免权重过大或过小导致训练不稳定，对权重进行裁剪，具体如（48）所示：

$$\widetilde{w}_\theta=clip\left(max\left(\frac{1}{w_\theta^w},\frac{1}{w_\theta^l}\right),1-\epsilon,1+\epsilon\right), \tag{48}$$

其中w_θ^w,w_θ^l分别是偏好样本和非偏好样本的权重，ϵ是裁剪阈值。

3) SDPO 损失函数：SDPO 的最终损失函数如（49）所示：

$$\mathrm{L}_{SDPO}(\theta)=-E_{(c,x_0^w,x_0^l)\sim D}\left[\log\sigma\left(\frac{\beta}{\widetilde{w}_\theta}\left[\log\frac{\pi_\theta(x_0^w|c)}{\pi_{ref}(x_0^w|c)}-\log\frac{\pi_\theta(x_0^l|c)}{\pi_{ref}(x_0^l|c)}\right]\right)\right], \tag{49}$$

其中，σ是 sigmoid 函数，β是温度参数，π_{ref}是参考模型，D是偏好数据集。

SDPO 通过引入重要性采样和权重裁剪技术，有效缓解了扩散模型在偏好对齐过程中出现的不稳定性和偏差问题，显著提升了训练效率和生成质量，并在多个模态任务中展现出卓越的泛化能力。然而，该方法高度依赖于高质量的偏好数据，对噪声反馈较为敏感，且在复杂多模态任务和极端高分辨率场景下的扩展性能仍需进一步验证。

算法 15. DIFFUSION-Negative Preference Optimization（DIFFUSION-NPO）

输入：基础扩散模型权重θ，正偏好权重偏移η，负偏好权重偏移δ，条件输入c，无条件/负条件输入c'，CFG 强度ω，权重融合超参数α,β。

输出：经过负偏好优化引导的生成结果x。

1) 训练阶段：
   - 使用与正偏好优化相同的训练方法，但将偏好对顺序反转或奖励函数取反；
   - 对于 DPO 类方法，将偏好对$r=(x_0,x_1,c)$反转为$r_{NPO}=(x_1,x_0,c)$；
   - 对于基于奖励模型的方法，将奖励函数设为$R_{NPO}(x,c)=1-R(x,c)$。
2) 推理阶段：使用 CFG 公式，将正偏好模型$\theta_{pos}=\theta+\eta$和负偏好模型$\theta_{neg}=\theta+\alpha\eta+\beta\delta$结合，具体如公式（50）所示：

$$\epsilon_\theta^\omega=(\omega+1)\cdot\epsilon_{\theta_{pos}}(x_t,c,t)-\omega\cdot\epsilon_{\theta_{neg}}(x_t,c',t), \tag{50}$$

其中，ϵ_θ代表噪声预测网络；ω表示 CFG 引导强度；c为条件输入；c'则指无条件或负条件输入；η是正偏好权重偏移；δ为负偏好权重偏移；α和β为调控正负权重影响的超参数。

DIFFUSION-NPO 的优势在于其能够在不依赖额外数据或特殊训练策略的情况下，显著提升生成质量，同时具备出色的兼容性和可扩展性，适用于多种扩散模型及其优化变体。然而，其不足之处在于需存储和加载两个权重偏移（正/负），增加了存储和计算的开销。

算法 16. Smoothed Preference Optimization（SmPO-Diffusion）

输入：预训练的扩散模型；偏好数据集$D=\{(c,x_0^w,x_0^l)\}$，包含提示词c与胜败图像对；奖励模型用于生成平滑偏好标签；超参数α,γ,β。

输出：更新后的策略π_θ

1) 平滑偏好建模：采用平滑分布$\tilde{p}$替代原始二元分布，其定义详见公式（51）和（52）：

$$\tilde{p}(x_0^w|c)\propto p(x_0^w|c)^\alpha\cdot p(x_0^l|c)^{\gamma-\alpha}, \tag{51}$$

$$\tilde{p}(x_0^l|c)\propto p(x_0^w|c)^{\gamma-\alpha}\cdot p(x_0^l|c)^\alpha, \tag{52}$$

其中，x_0^w和x_0^l分别代表偏好对中的“胜者”和“败者”图像；α为权重因子，用于调控胜者和败者在分布中的相对贡献；γ为敏感度因子，负责调节混合分布对参数变化的敏感程度；$p(\cdot|c)$则表示原始图像生成的概率分布。

2) Renoise 反转优化：通过 DDIM 反转和 Renoise 步骤来估算潜在变量$\tilde{x}_t$，具体如公式（53）所示：

$$\tilde{x}_t=\sqrt{\frac{\alpha_t}{\alpha_{t-1}}}\hat{x}_{t-1}+\left(\sqrt{\frac{1-\alpha_t}{\alpha_t}}-\sqrt{\frac{1-\alpha_{t-1}}{\alpha_{t-1}}}\right)\epsilon_\theta^t(\hat{x}_t,c), \tag{53}$$

其中，$\hat{x}_t$是通过 DDIM 反转得到的估计值，ϵ_θ^t为噪声预测网络，α_t为噪声调度系数。

3) 损失函数：最终的损失函数定义如公式（54）所示：

$$\mathrm{L}(\theta)=-E_{t,D}\log\sigma\big(-(2\alpha-\gamma)\beta(\tilde{s}_\theta^t(x_0^w,c)-\tilde{s}_\theta^t(x_0^l,c))\big), \tag{54}$$

其中，$\tilde{s}_\theta^t$是基于 Renoise 估计的得分函数，用于衡量当前模型与参考模型在去噪过程中的差异；β为正则化系数。

SmPO-Diffusion 在偏好对齐任务中表现卓越，显著提升了图像质量和训练效率，并有效缓解了过优化及目标不一致的问题。然而，其性能仍然依赖于预训练奖励模型的质量，且可能受到数据集中潜在偏见的负面影响。

算法 17. Timestep-Segment Preference Optimization（TSPO）

输入：初始策略模型π_{ref}，提示分布D，预定义的偏好等级规则，超参数β,N,λ。参考图像、驱动音频、文本提示。

输出：更新后的策略π_θ生成的人体动画视频

1) 时间步分段策略：将总时间步范围$[0, T]$划分为两个区间：
   - 运动区间：$[T\cdot t_{min}, T]$，用于优化运动自然度；
   - 保真度区间：$[0, T\cdot t_{min}]$，用于优化视觉细节。
2) 专家 LoRA 模块：
   - Motion LoRA：在运动区间激活，优化时序一致性；
   - Fidelity LoRA：在保真度区间激活，增强结构保真度。
3) 偏好优化损失函数：对于每个维度 d（运动或保真度），损失函数如公式（55）所示：

$$\mathrm{L}_d=-\log\sigma\left(-\frac{\beta}{2}\left[\left(\mathrm{L}_\theta^{w_d}-\mathrm{L}_\theta^{l_d}\right)-\left(\mathrm{L}_{ref}^{w_d}-\mathrm{L}_{ref}^{l_d}\right)\right]\right), \tag{55}$$

其中，σ代表 sigmoid 函数；β是用于控制偏好强度的超参数；$\mathrm{L}_\theta^{w_d}$和$\mathrm{L}_\theta^{l_d}$分别表示策略模型在偏好样本和较差样本上的流匹配损失；$\mathrm{L}_{ref}^{w_d}$和$\mathrm{L}_{ref}^{l_d}$则分别代表参考模型在相同样本上的流匹配损失；w_d和l_d分别指在维度d上“胜出”和“较差”的样本。

TSPO 通过时间步分段与专家 LoRA 模块，有效解耦了运动与保真度的优化，在提升生成质量的同时实现了推理加速，展现出较强的实用性和可扩展性。然而，该方法依赖于高质量偏好数据的构建及人工标注，且其分段策略对时间步的划分较为敏感。

算法 18. Geometric-Averaged Preference Optimization（GAPO）

输入：初始策略模型π_θ；参考策略模型π_{ref}；包含软标签的偏好数据集$D=\{(x,y_1,y_2,\hat{p})\}$；超参数$\beta$。

输出：更新策略π_θ

1) 软偏好标签：定义$\hat{p}\in[0.5,1]$表示响应y_1优于y_2的估计概率，反映偏好的分布特性。
2) 加权几何平均策略：将策略输出似然按软标签加权几何平均，构造虚拟的“胜者”与“败者”策略，具体如公式（56）所示：

$$\bar{\pi}(y_w|x)\propto\pi(y_1|x)^{\hat{p}}\pi(y_2|x)^{1-\hat{p}},\quad \bar{\pi}(y_l|x)\propto\pi(y_1|x)^{1-\hat{p}}\pi(y_2|x)^{\hat{p}}. \tag{56}$$

3) GDPO 损失函数：将上述策略代入 DPO 损失中，得到的 GDPO 损失函数如公式（57）所示：

$$\mathrm{L}_{GDPO}=-E_{(x,y_1,y_2,\hat{p})}\left[\log\sigma\left(\beta(2\hat{p}-1)\log\frac{\pi_\theta(y_1|x)\pi_{ref}(y_2|x)}{\pi_{ref}(y_1|x)\pi_\theta(y_2|x)}\right)\right], \tag{57}$$

其中，β是控制 KL 正则强度的超参数，π_θ是待优化策略，π_{ref}是参考策略，σ是 sigmoid 函数。

GAPO 通过引入软标签和几何平均策略，有效缓解了 DPO 中的过优化和目标不匹配问题，显著提升了模型在多样化偏好数据下的对齐性能。然而，该方法的效果依赖于准确的软标签估计，若标签质量不佳或存在偏差，可能会影响最终结果；此外，超参数β的调整对性能影响较大，需谨慎进行设置。

算法 19. Full-Step-DPO

输入：超参数$\lambda_1, \lambda_2, \lambda_3, \beta$

输出：更新策略π_θ

1) 分步 DPO 损失：通过动态调节梯度权重，实现分步骤优化的效果，具体如公式（58）所示：

$$\nabla_\theta L=-\beta E_{(x,y^w,y^l)}\left[\sigma\left(\hat{r}_\theta(x,y^l)-\hat{r}_\theta(x,y^w)\right)\cdot\left(\sum_{i=1}^{K^w}\alpha_i^w\nabla_\theta \log \pi_\theta(s_i^w|x,s_{<i}^w)-\sum_{i=1}^{K^l}\alpha_i^l\nabla_\theta \log \pi_\theta(s_i^l|x,s_{<i}^l)\right)\right], \tag{58}$$

其中，s_i是推理过程中的第i步，K^w和K^l分别表示正样本和负样本的步数，$\hat{r}_\theta(x,y)=\beta \log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}$是隐式奖励函数，$\alpha_i$表示第$i$步的权重系数。

2) 动态权重系数：无需人工标注，即可自动评分，实现高效且可扩展的流程，具体如公式（59）所示：

$$\alpha_i=\begin{cases}\dfrac{e^{\gamma r_{s_i}}}{\sum_j e^{\gamma r_{s_j}}} & if\ s_i\in y^w\\ \dfrac{e^{-\gamma r_{s_i}}}{\sum_j e^{-\gamma r_{s_j}}} & if\ s_i\in y^l\end{cases}, \tag{59}$$

其中，r_{s_i}表示第i步的奖励值；γ是温度系数，控制奖励分布的锐度。

3) 自监督过程奖励模型：依据奖励的重要性调整各个步骤，提升对正确/错误步骤的敏感性，具体如公式（60）所示：

$$L_{PRM}=-\sum_{i=1}^{K}\left[y_{s_i}\log r_{s_i}+(1-y_{s_i})\log(1-r_{s_i})\right], \tag{60}$$

其中，y_{s_i}是步骤i对应的黄金标签，r_{s_i}是步骤i的 PRM 预测得分，K为总步数。

算法 20. Step-DPO

输入：问题x，前k-1个正确推理步骤$s_1,s_2,\ldots,s_{k-1}$，正确下一步s_{win}，错误下一步s_{lose}，参考模型π_{ref}，策略模型π_θ，超参数β。

输出：更新策略π_θ

在给定问题x及前k-1个正确推理步骤$s_{1\sim k-1}$的前提下，旨在最大化正确下一步s_{win}的概率，同时最小化错误下一步s_{lose}的概率，由此构建的 Step-DPO 损失函数如（61）所示：

$$L(\theta)=-E_{(x,s_{1\sim k-1},s_{win},s_{lose})\sim D}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(s_{win}|x;s_{1\sim k-1})}{\pi_{ref}(s_{win}|x;s_{1\sim k-1})}-\beta\log\frac{\pi_\theta(s_{lose}|x;s_{1\sim k-1})}{\pi_{ref}(s_{lose}|x;s_{1\sim k-1})}\right)\right], \tag{61}$$

其中，$s_{1\sim k\text{-}1}$为前k-1个正确推理步骤，s_{win}为正确的第k步，s_{lose}为错误的第k步，π_θ为待优化的策略模型，β为控制与参考模型偏离程度的超参数，D为步骤级偏好数据集。

Step-DPO 的优势在于其精细化的优化机制，能够显著提升模型在复杂推理任务中的表现，并兼具高数据效率和快速训练的特性。然而，该方法高度依赖于高质量步骤级标注数据的构建，对错误定位的准确性要求极高，且在非数学类长链推理任务中的泛化能力仍需进一步验证。

算法 21. LogicPO

输入：自然语言逻辑问题（包含上下文与查询）

输出：对应的一阶逻辑（FOL）程序

1) 监督微调阶段：通过逻辑标签匹配的 NL-FOL 样本对(S, FS)进行有监督训练，目标是最大化条件概率，具体如公式（62）所示：

$$\max_{\theta} E_{(S,FS)\sim D_{sft}}[\log P_\theta(FS|S)], \tag{62}$$

其中，S为输入的自然语言故事，FS为对应的 FOL 程序，θ为模型参数。

2) 偏好优化阶段：运用 DPO 或 KTO 方法在偏好数据集D_{pref}上进行进一步微调。DPO 的目标函数如公式（63）所示：

$$\mathrm{L}_{DPO}(\theta)=-E_{(s,FS^+,FS^-)}\left[\log\sigma\left(\beta\log\frac{P_\theta(FS^+|S)}{P_{ref}(FS^+|S)}-\beta\log\frac{P_\theta(FS^-|S)}{P_{ref}(FS^-|S)}\right)\right], \tag{63}$$

其中，FS^+为偏好样本（逻辑正确），FS^-为拒绝样本（存在逻辑或语法错误），P_{ref}为参考模型，β为控制偏离参考模型程度的超参数，σ为 sigmoid 函数。

LogicPO 在提升 FOL 翻译的逻辑一致性与语法正确性方面成效显著，尤其在减少谓词不一致和语法错误方面表现尤为突出。然而，该方法仍面临语义信息泄露和复杂逻辑表达不足的挑战。

算法 22. Multi-granularity Direct Preference Optimization（MDPO）

输入：数学问题x，前k步正确推理序列$s_{0\sim k\text{-}1}$，正确后续步骤y_w，错误后续步骤y_l，模型策略π_θ。

输出：更新策略π_θ

1) 目标统一：将数学问题转化为“给定问题与前k步，继续生成后续步骤”的文本补全任务，确保训练与推理目标一致。

2) 损失函数：MDPO 的损失函数如公式（64）所示：

$$-E_{(x,s_{0\sim k\text{-}1},y_w,y_l)\sim D}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w|(x,s_{0\sim k\text{-}1}))-\frac{\beta}{|y_l|}\log\pi_\theta(y_l|(x,s_{0\sim k\text{-}1}))-\gamma\right)\right], \tag{64}$$

其中，x代表数学问题；s_0至$s_{k\text{-}1}$表示前k步的正确推理步骤；y_w表示从第k步起直至正确的后续步骤序列；y_1表示从第k步起直至错误的后续步骤序列；β为奖励缩放系数；γ为奖励边际，旨在确保正确序列的奖励高于错误序列。

MDPO 通过多粒度监督机制显著增强了模型在数学推理任务中的准确性和鲁棒性，特别是在处理长链推理和复杂计算时表现出色。但 MDPO 的实施依赖于构建高质量的多粒度偏好数据，且其训练过程相对复杂。

算法 23. Pseudocode-style Planning Guided Preference Optimization（PGPO）

输入：任务指令u,专家轨迹数据集D,基础语言模型π_{base},最大迭代次数K。

输出：更新后的策略π_θ

1) 监督微调阶段：利用包含伪代码计划的专家轨迹数据，对基础模型进行精细调整，目标函数详见公式（65）：

$$\mathrm{L}_{SFT}(\theta)=-E_{(u,\tau')}\left[\sum_{j=1}^{n} log\,\pi_\theta(t_j|u,p,\tau_{j-1})+log\,\pi_\theta(a_j|u,p,\tau_{j-1},t_j)+log\,\pi_\theta(p|u)\right], \tag{65}$$

其中，u代表任务指令，p代表伪代码计划，τ'表示包含该计划的轨迹，t_j和a_j则分别指代第j步的思考和动作。

2) 轨迹收集与奖励构建：
   - 计划驱动奖励：基于智能体生成的计划与轨迹所获得的环境回报，构建优劣对比轨迹对$(p^w,\tau^w)\succ(p^l,\tau^l)$。
   - 计划遵循奖励：通过蒙特卡洛采样方法，估计智能体在第一步之后是否遵循既定计划，进而构建对比轨迹对$\tau^w_{2:n}\succ\tau^l_{2:m}$。

3) 偏好优化目标：融合 DPO 损失与 SFT 正则项，整体目标如公式（66）所示：

$$\min_{\pi_\theta}\left(\mathrm{L}_p+\mathrm{L}_f+\mathrm{L}_s\right), \tag{66}$$

其中，L_p代表计划驱动损失的 DPO 形式，旨在激励生成高质量的计划与轨迹；L_f则是计划遵循损失的 DPO 形式，用以增强智能体对计划的执行一致性；L_s作为 SFT 损失，有效防止策略出现过度偏离。

PGPO 通过引入结构化伪代码规划，显著提升了智能体的泛化能力和推理效率。其优势在于计划抽象性强、引导明确，能够有效减少无效探索和动作错误；然而，其不足之处在于依赖于高质量伪代码的生成及人工验证，且蒙特卡洛采样过程增加了计算开销。

算法 24. Self-Consistency Preference Optimization（ScPO）

输入：问题q，参考答案a，初始策略模型π_θ，采样响应组数G，超参数$k_0,\gamma,\delta_1,\delta_2$。

输出：更新后的策略π_θ

1) 在线重要性评估器：评估每个推理步骤的重要性，其公式表达如（67）所示：

$$d_{i,j}=\begin{cases}0, & if\,p_1\le p_2\\ \dfrac{p_1^2-p_2^2}{p_1^2\cdot l_{i,j}}, & if\,p_1>p_2\end{cases}, \tag{67}$$

其中，$p_1=P(a|q,S_i)$表示在完整推理路径下生成正确答案的概率；$p_2=P(a|q,\frac{S_i}{\{s_{i,j}\}})$则表示移除第$j$步后的概率；$l_{i,j}$代表第$j$步的长度。

2) 步骤级长度控制奖励函数：依据步骤的重要性和问题的难度，动态调整奖励值，具体计算方式如公式（68）所示：

$$r_{i,j}=\begin{cases}(1-k_1\sigma(\tilde{l}_{i,j}))(1-k_2\sigma(\tilde{n}_i)), & if\ a_i=a \\ -e^{-\frac{\rho\tilde{d}'_{i,j}}{k_0}}, & if\ a_i\neq a\end{cases}, \tag{68}$$

其中，k_1和k_2为动态惩罚系数，$\tilde{l}_{i,j}$和$\tilde{n}_i$为标准化后的步骤长度与步骤数，ρ代表问题难度，$\tilde{d}'_{i,j}$为归一化后的重要性分数。

3) 步骤级广义优势估计：引入折扣因子 γ，以计算累积优势，详细公式如（69）所示：

$$A_{i,j}=\sum_{n=0}^{k_i-j}\gamma^n\cdot\tilde{r}_{i,j+n}, \tag{69}$$

其中，$\tilde{r}_{i,j+n}$为归一化后的步骤奖励。

4) 难度自适应裁剪策略：根据问题的难度，调整 PPO 裁剪范围，具体方法如公式（70）所示：

$$\varepsilon_{low}=\varepsilon-\delta_1\cdot(1-\rho),\quad \varepsilon_{high}=\varepsilon+\delta_2\cdot\rho, \tag{70}$$

其中，δ_1、δ_2为调整幅度，ρ为难度系数。

SCPO 在提升推理效率方面表现卓越，能够有效区分关键步骤与冗余步骤，特别适用于复杂的推理任务。然而，其性能依赖于步骤重要性评估的准确性，若评估出现偏差，可能导致关键步骤被错误地压缩。

算法 25. Thinking Preference Optimization（ThinkPO）

输入：基础语言模型；长链思维训练数据$D_{sft}=\{(x,y)\}$；短链思维数据$D_{dpo}=\{(x,y_w,y_l)\}$；超参数：学习率、批次大小、β。

输出：更新后的策略π_θ

ThinkPO 的训练过程分为两个阶段：首先是推理 SFT 阶段，通过运用长链思维数据对基础模型进行监督微调，以构建初步的推理能力，具体如公式（71）所示；其次是推理 DPO 阶段，基于 DPO 目标函数，激励模型倾向于生成长链思维答案而非短链答案，其损失函数可表示为公式（72）所示：

$$\mathrm{L}_{SFT}(\theta)=-E_{(x,y)\sim D_{sft}}\left[\sum_{t=1}^{T}log\,\pi_\theta(y_t|x,y_{<t})\right], \tag{71}$$

其中，π_θ代表待训练的模型，x表示输入的问题，y对应于目标序列中的长链思维答案，y_t指的是答案序列中的第t个词元，$y_{<t}$则涵盖了第t个词元之前的所有词元，T为答案序列的总长度。

$$\mathrm{L}_{DPO}(\pi_\theta;\pi_{ref})=-E_{(x,y_w,y_l)\sim D_{dpo}}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\,log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right], \tag{72}$$

其中，π_θ为待优化的策略模型，即第一阶段 SFT 后的模型；π_{ref}为参考模型，通常与π_θ在初始化时相同，即 SFT 后的模型，但在 DPO 训练期间其参数被冻结；x为输入问题；y_w为长

链思维答案，即被选中的、偏好的答案；y_l为短链思维答案，即被拒绝的答案；β为控制模型偏离参考模型程度的超参数；σ为 sigmoid 函数。

ThinkPO 能够在无需额外标注长链数据的情况下，有效提升模型的推理能力，特别适合应用于资源受限的环境，并能够持续优化现有的 SFT 模型。然而，该方法对超参数具有较高的敏感性，因此需要精细调参以确保最佳效果。

算法 26. Tree Structured Direct Preference Optimization（TDPO）

输入：指令I，问题x，前一轮结构化信息SI，被选择的回答y_w，被拒绝的回答y_l，参考模型π_{ref}，超参数β。

输出：更新后的策略π_θ

1) 偏好树构建：通过多轮辩论提取结构，每个树的根节点包含问题、指令以及前一轮的结构化信息。正确和错误的回答分别作为被选择和被拒绝的子节点。
2) 训练目标：T-DPO 目标函数基于 DPO，通过对比正负样本的对数概率差异，引导模型偏好正确的推理路径。其公式如公式（73）所示：

$$L_{T\text{-}DPO}=-E_{(I,x,SI,y_w,y_l)\sim D_{T\text{-}DPO}}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_w|I,x,SI)}{\pi_{ref}(y_w|I,x,SI)}-\beta\,log\frac{\pi_\theta(y_l|I,x,SI)}{\pi_{ref}(y_l|I,x,SI)}\right)\right], \tag{73}$$

其中，I代表任务指令，x表示问题，SI指的是前一轮的结构化信息，y_w为被选中的正确回答，y_l为被排除的错误回答，π_θ是需要优化的学生模型策略，π_{ref}为参考模型策略，β是用于控制与参考策略偏离程度的超参数。

TDPO 能够高效利用多轮辩论中的结构化信息，显著提升模型推理的准确性和鲁棒性，尤其在数据稀缺的情况下仍能保持优异表现；然而，其性能受限于学生模型的基础能力，在处理极为复杂的任务时可能遭遇瓶颈。此外，训练过程依赖于高质量辩论数据的生成，导致计算成本相对较高。

算法 27. AIPO

输入：当前策略模型π_θ，参考模型π_{ref}，偏好数据集$D=\{(x,y_w,y_l)\}$，超参数β,α,λ。

输出：更新策略π_θ

1) 基础 DPO 目标重构：将原始的 DPO 损失重新表述为如下（74）所示：

$$L_{DPO}=-E_{(x,y_w,y_l)}\left[log\,\sigma\left(\beta(s_\theta-s_{ref})\right)\right], \tag{74}$$

其中，$s_\theta=log\frac{\pi_\theta(y_w|x)}{\pi_\theta(y_l|x)}$为策略模型对选中与拒绝响应的对数概率比，$s_{ref}=log\frac{\pi_{ref}(y_w|x)}{\pi_{ref}(y_l|x)}$为参考模型对选中与拒绝响应的对数概率比，$\beta$为控制偏离参考模型程度的超参数。

2) 引入一致性感知项α：在 sref 前引入系数$\alpha(\alpha>0)$，构建α-DPO 损失，具体如（75）所示：

$$L_{\alpha\text{-}DPO}=-E_{(x,y_w,y_l)}\left[log\,\sigma\left(\beta(s_\theta-(1+\alpha)s_{ref})\right)\right], \tag{75}$$

该损失相当于在 Bradley-Terry 模型中引入动态奖励边界$\alpha\beta\cdot s_{ref}$，增强对参考模型偏好的响应。

3) 稳定性增强：为防止策略模型输出分布出现剧烈波动，引入负对数似然损失作为正则项，具体如（76）所示：

$$L_{NLL}=-\frac{1}{|y_w|}\log\pi_\theta(y_w|x), \tag{76}$$

其中$|y_w|$是选中响应的长度，用于归一化。

4) AIPO 总损失：AIPO 的损失函数如公式（77）所示：

$$L_{AIPO}=L_{\alpha\text{-}DPO}+\lambda\cdot L_{NLL}, \tag{77}$$

其中λ是平衡两项权重的超参数。

AIPO 通过引入参考模型的偏好一致性作为动态奖励边界，有效缓解了迭代训练中的长度利用问题，并在多个基准测试中展现出卓越性能。其显著优势在于能够自适应调整奖励边界，从而提升训练的稳定性和模型的整体性能。然而，AIPO 仍依赖于参考模型的质量，且在每轮迭代中未对超参数进行调整，这可能会限制其在异构模型或动态数据环境中的泛化能力。

算法 28. Critical-token Direct Preference Optimization (C-DPO)

输入：超参数 $\lambda_1, \lambda_2, \lambda_3, \beta$

输出：更新策略 π_θ

1) 关键标记识别：运用两种模型进行识别，即正向模型P^p，其基于正确样本进行训练；负向模型P^n，则基于错误样本进行训练。对于负样本集中的每一个标记y_t，通过计算其得分s_t，如（78）所示，从而判定该标记是否属于关键标记：

$$\log s_t=(1+\beta)\log P^p(y_t|x,y_{<t})-\beta\log P^n(y_t|x,y_{<t})-\log Z, \tag{78}$$

其中，β为缩放参数，$P^p(\cdot)$是正向模型所给出的概率，$P^n(\cdot)$是负向模型所给出的概率，Z是用于配分函数的归一化常数。

2) 目标函数：基于 DPO 的目标函数引入 token 级奖励机制，具体如（79）和（80）所示：

$$\phi_s(x,y,s)=\gamma\sum_{t=1}^{T}(1-s_t)\log\frac{\pi_\theta(y_t|x,y_{<t})}{\pi_{ref}(y_t|x,y_{<t})}, \tag{79}$$

$$L_{cDPO}=-\sum_{i=1}^{M}\log\sigma\big(\phi(x_i,y_i^p)-\phi_s(x_i,y_i^n,s_i^n)\big), \tag{80}$$

其中，y_i^p是正样本（正确推理），y_i^n是负样本（错误推理），s_i^n是负样本中每个标记的关键分数向量。

算法 29. Hallucination-Targeted Direct Preference Optimization（HDPO）

输入：图像I，问题x，正样本回复y_c和负样本回复y_r（源自 VDH/LCH/MCH 构造策略）。

输出：更新策略π_θ

1) 构建三种幻觉目标偏好对：
   - 视觉分散幻觉（VDH）：保留注意力分数最低的视觉 token，构建视觉误导负样本。
   - 长上下文幻觉（LCH）：截断长回复并引导模型续写，模拟长文本幻觉现象。

- 多模态冲突幻觉（MCH）：在问题前添加与图像相冲突的信息，构造冲突场景负样本。

2) 损失函数：将三种幻觉目标偏好对得到的结果代入 DPO 的损失函数，具体如公式（81）所示：

$$L_d=-E_D\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_c|x)}{\pi_{ref}(y_c|x)}-\beta\,log\frac{\pi_\theta(y_r|x)}{\pi_{ref}(y_r|x)}\right)\right], \tag{81}$$

其中，y_c代表优选回复（正样本），y_r代表拒绝回复（负样本），β为控制偏离参考模型程度的超参数。

HDPO 的优势在于其精准的幻觉建模和高效的数据处理能力，即使在数据量有限的情况下，也能显著增强抗幻觉性能，并在各类任务中展现出稳定的表现。然而，其构建过程依赖于外部模型和启发式策略，因此在可扩展性和通用性方面仍存在一定的局限性。

算法 30. Credit Assignment Policy Optimization（CAPO）

输入：问题文本q；待优化的策略模型$\pi_{\theta_{old}}$；用于评估的生成式过程奖励模型$\pi_{LLM\text{-}as\text{-}GenPRM}$；生成的评估数量$k$；奖励与惩罚的权重$W_{whole}$和$W_{process}$。

输出：更新策略π_θ

1) 过程奖励生成：使用 LLM 作为生成式过程奖励模型，对策略模型生成的回答进行多轮评判，提取错误步骤索引，具体如公式（82）所示：

$$\{c_j\}_{j=1}^{k}\sim\pi_{LLM\text{-}as\text{-}GenPRM}(\cdot|I,q,y_i), \tag{82}$$

2) 细粒度奖励计算：对每个 token 的奖励进行惩罚调整，具体如公式（83）所示：

$$R_t^i=r_v\cdot W_{whole}-I(t\in T_{err}^i)\cdot W_{process}, \tag{83}$$

其中，r_v是基于规则的响应级奖励机制（正确时奖励值为 1，错误时奖励值为 0）；W_{whole}和$W_{process}$则分别代表整体奖励权重和过程惩罚权重；T_{err}^i是所有错误步骤中所包含的 token 索引集合。

3) 优势估计与策略优化：使用组内归一化计算每个 token 的优势，如公式（84）所示，并基于 PPO 目标函数进行策略更新：

$$\widehat{A}_t^i=\frac{R_t^i-mean(\{R_{t'}^i\})}{std(\{R_{t'}^i\})}. \tag{84}$$

CAPO 的优势在于其实现了可验证且细粒度的信用分配机制，无需人工标注过程标签，同时能显著提升模型在数学和通用推理任务上的表现。然而，该方法依赖于外部的大型语言模型作为评判标准，这可能会引发计算成本增加和评判一致性方面的挑战。

算法 31. Probability-Consistent Preference Optimization（PCPO）

输入：初始模型M_0，提示集合$D=\{(x_i,y_i^*)\}$，超参数α,β,N,k。

输出：优化后的模型序列$M_1,M_2,\ldots,M_T$

1) 候选对构建：针对每个问题，利用当前模型生成多个响应，并根据答案的正确性将其划分为正例集Y_w和负例集Y_l，然后通过 Levenshtein 距离筛选出候选对集合C_t^{pairs}。

2) 偏好对选择：计算每对候选响应的“配对加权分数”s_w，选取分数最高的正例-负例对作为训练数据。其中，Token 一致性分数的计算方法如公式（85）所示，配对加权分数的计算方法如公式（86）所示：

$$c_t(y_w|y_l)=exp\left(-|\log P_w(y_t|x,y_{<t})-\log P_l(y_t|x,y_{<t})|\right), \tag{85}$$

$$s_w(y_w|y_l)=\frac{\sum_t c_t(y_w|y_l)}{l_{y_l}}, \tag{86}$$

其中，P_w和P_l分别表示在当前位置t的正例和负例的条件概率；y_t是匹配的 token；l_{y_l}是负例的 token 长度，用于归一化处理。

3) PCPO 损失函数：PCPO 损失函数的定义如公式（87）所示：

$$\mathrm{L}_{PCPO}=-s_w\log\sigma\left(\beta\log\frac{M_\theta(y^+|x)}{M_t(y^+|x)}-\beta\log\frac{M_\theta(y^-|x)}{M_t(y^-|x)}\right)-\frac{\alpha s_w}{|y^+|}\log M_\theta(y^+|x). \tag{87}$$

PCPO 通过引入概率一致性机制，显著提升了偏好数据的质量及训练效果，尤其在复杂推理任务中表现尤为出色。然而，该方法依赖于真实答案来构建偏好对，计算 token 概率与匹配过程亦产生了额外的计算开销。

算法 32. Cognitive Preference Optimization（CogPO）

输入：策略模型π_θ，参考模型π_{ref}，微任务类型对应的β值β_S,β_M,β_L。

输出：更新策略π_θ

1) CogPO 的目标函数如公式（88）所示：

$$\mathrm{L}_{CogPO}=-E_{(x,r_w,r_l)\sim D}\log\sigma(M_{\beta_{CogPO}}(x,r_w,r_l)), \tag{88}$$

$$M_{\beta_{CogPO}}(x,r_w,r_l)=\beta_{CogPO}\left(\log\frac{\pi_\theta(r_w|x)}{\pi_{ref}(r_w|x)}-\log\frac{\pi_\theta(r_l|x)}{\pi_{ref}(r_l|x)}\right), \tag{89}$$

其中，x代表输入问题；r_w表示被选中的推理路径（符合模型的认知能力）；r_l表示被拒绝的推理路径（不适合或存在错误）；σ为 sigmoid 函数；$M_{\beta_{CogPO}}$代表偏好边际，其定义如公式（89）所示；β_{CogPO}根据微任务类型动态选取，其取值分别为β_S,β_M,β_L。

2) 微任务划分：

- 小差距任务：正确但复杂度不匹配的路径对；
- 中差距任务：正确但不适合 vs.错误的路径对；
- 大差距任务：正确且适合 vs.错误的路径对。

CogPO 的优点在于其能显著提升小模型在复杂推理任务中的表现，特别是在通过难度感知的β调节机制后，进一步增强了训练的精细度和适应性。然而，其局限性在于依赖大模型来生成高质量的训练数据，并且对于小模型自身的认知上限依然无法实现突破。

算法 33. Input-Output Preference Optimization（IOPO）

输入：原始指令x_1，原始响应y_1，约束修改后的指令x_2，对应新指令的响应y_2，参考模型π_{ref}，超参数β。

输出：更新策略π_θ

1) 数据构造：构建两组输入-输出对：

- 匹配组$G_1=\{(x_1,y_1),(x_2,y_2)\}$
- 不匹配组$G_2=\{(x_1,y_2),(x_2,y_1)\}$

其中，x_2是在x_1的基础上通过"增、删、改"约束生成的变体，y_1和y_2分别为对应指令的正确响应。

2) 偏好概率建模：使用公式（90）建模组间偏好概率：

$$p(G_1 \succ G_2)=\sigma\left(\frac{\lambda}{2}(\Pi_1+\Pi_2)\right), \tag{90}$$

其中，Π_1和Π_2的定义分别如公式（91）和（92）所示：

$$\Pi_1=2\beta\, log\frac{\pi_\theta(y_1|x_1)}{\pi_{ref}(y_1|x_1)}-\beta\, log\frac{\pi_\theta(y_2|x_1)}{\pi_{ref}(y_2|x_1)}-\beta\, log\frac{\pi_\theta(y_1|x_2)}{\pi_{ref}(y_1|x_2)}, \tag{91}$$

$$\Pi_2=2\beta\, log\frac{\pi_\theta(y_2|x_2)}{\pi_{ref}(y_2|x_2)}-\beta\, log\frac{\pi_\theta(y_1|x_2)}{\pi_{ref}(y_1|x_2)}-\beta\, log\frac{\pi_\theta(y_2|x_1)}{\pi_{ref}(y_2|x_1)}. \tag{92}$$

3) 损失函数：损失函数具体如公式（93）所示：

$$\mathrm{L}_{IOPO}=-E_{(x_1,y_1,x_2,y_2)}\left[log\,\sigma\left(\frac{\Pi_1+\Pi_2}{2}\right)\right]. \tag{93}$$

IOPO 通过联合建模输入与输出偏好，显著增强了对复杂指令中多约束条件的感知与遵循能力。实验结果显示，其在多个基准测试中均优于 DPO 等现有方法。然而，IOPO 也伴随着更高的计算和内存开销，并且数据构造过程更为复杂。

算法 34. Direct Multi-Preference Optimization（DMPO）

输入：聚合的细粒度偏好数据集$D=\{(k,x^k,y_w^k,y_l^k)\}$，参考策略模型$\pi_{ref}$，子偏好数量$\kappa$，选择预算$\lambda$。

输出：目标策略模型π_θ

1) PD 选择原则：选择 PD 值最负（即共识最高）的样本子集进行 DPO 训练，具体如公式（94）所示：

$$\widetilde{D}=arg\,top\,\lambda\{-\Delta\phi_k(z)|z\in D\}. \tag{94}$$

2) PD 估计与长度偏差修正：使用小规模代理模型训练各子偏好的奖励模型，并通过长度平衡采样与长度惩罚项来缓解长度偏差对 PD 估计的影响。

3) 目标函数：DMPO 目标函数如公式（95）所示：

$$\mathrm{L}_{DMPO}(\theta)=-E_{z\sim D}\left[log\,\sigma\left(\kappa M_\theta(z)+\Delta\phi_\kappa(z)\right)\right], \tag{95}$$

其中，$M_\theta(z)=\beta\, log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\, log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}$表示偏好边际；$\Delta\phi_k(z)=\phi_k(x,y_w)-\phi_k(x,y_l)$为偏好分歧项；$\phi_k(x,y)=-\sum_{k'\neq k} r_{k'}(x,y)$表示除当前子偏好外其他所有偏好的负奖励之和；$\sigma$代表 sigmoid 函数；$\kappa$代表子偏好数量；$\beta$为 KL 惩罚系数。

DMPO 能够高效利用细粒度偏好信号，借助理论驱动的数据筛选策略，在冲突数据中精准识别高质量样本，从而显著提升对齐效果及训练效率。然而，该方法依赖于子偏好奖励模型的准确估计，若奖励模型存在偏差或训练不足，可能会影响 PD 估计的可靠性。

算法 35. Confidence-Reward Driven Preference Optimization（CRPO）

输入：参考模型π_{ref}，源句$x(i)$，奖励模型R。

输出：偏好数据$D_{\succ}^{(i)}=(x^{(i)},y_w,y_l)$

CRPO 提出了两种评分机制：$CR+$和$CR\times$，用于评估每个句子对在训练中的价值，两种方法具体如下所示：

1) $CR+$（加法形式）：加法形式如公式（96）所示：

$$CR+=K\cdot[R(x,y_w)-R(x,y_l)]+[\log\pi_{ref}(y_l|x)-\log\pi_{ref}(y_w|x)], \tag{96}$$

其中，$R(x,y)$表示奖励模型对句子y在源句x下的评分；$\pi_{ref}(y|x)$代表参考策略生成句子y的概率；K是一个超参数，用于平衡奖励值与置信度之间的量级差异；y_w为优选句子，y_l则为劣选句子。

2) $CR\times$（乘法形式）：通过乘法自然融合奖励差异与置信度差异，无需额外调参，具体如公式（97）所示：

$$CR\times=[R(x,y_w)-R(x,y_l)]\cdot[\log\pi_{ref}(y_l|x)-\log\pi_{ref}(y_w|x)]. \tag{97}$$

CRPO 的优势在于其能够高效识别并利用困难样本，从而提升数据选择的质量和训练效率。该技术在多种翻译方向和模型架构上均展现出卓越的泛化能力。然而，其不足之处在于目前仅选取 CR-Score 最高的句子对，可能遗漏其他虽分数略低但质量同样优秀的数据。

算法 36. Confidence-guided Preference Optimization（ConfPO）

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，策略模型π_θ，超参数β,γ。

输出：更新策略π_θ

1) 关键词元选择机制：通过特定的选择函数$s(y_i)$来确定是否对某一词元进行优化，依据模型对词元的预测置信度（即概率）进行判断，具体如公式（98）所示：

$$s(y_i)=\mathbf{1}(\pi_\theta(y_i|x,y_{<i})\le\tau), \tag{98}$$

其中，$\pi_\theta(y_i|x,y_{<i})$表示当前策略模型π_θ在给定提示x和历史词元$y_{<i}$的条件下，预测当前词元y_i的概率；τ为置信度阈值，若模型对词元y_i的置信度低于或等于此阈值，则该词元将被选中；$\mathbf{1}(\cdot)$为指示函数，当括号内条件成立时取值为 1，否则为 0。因此，$s(y_i)$是一个二值函数，1 表示选中，0 表示不选。

2) 自适应阈值设定：ConfPO 并未采用固定的全局阈值，而是针对每个训练样本中的回答序列，动态地计算相应的阈值，以便更好地适应不同内容和长度的回答。具体计算方法如公式（99）所示：

$$\tau=\frac{1}{|y|}\sum_{i=1}^{|y|}\pi_\theta(y_i|x,y_{<i}), \tag{99}$$

其中，$|y|$表示响应y的总词元数。该公式将阈值τ定义为当前模型对整个响应序列y中所有词元的平均预测概率。这表明，对于一个回答，仅当词元的置信度低于该回答的平均水平时，才会被选中进行优化。

3) 目标函数设计：ConfPO 在 SimPO 目标函数的基础上，引入了词元选择函数，专门针对置信度低于阈值的词元进行奖励计算。其目标函数具体如公式（100）所示：

$$L_{ConfPO}(\pi_\theta)=-E_{(x,y^w,y^l)\sim D}\left[\log\sigma\left(\frac{\beta}{|y_s^w|}\sum_{i=1}^{|y^w|}s(y_i^w)\log\pi_\theta(y_i^w|x,y_{<i}^w)-\frac{\beta}{|y_s^l|}\sum_{i=1}^{|y^l|}s(y_i^l)\log\pi_\theta(y_i^l|x,y_{<i}^l)-\gamma\right)\right], \tag{100}$$

其中，γ作为目标间隔超参数，严格要求偏好响应的奖励值必须至少比非偏好响应的奖励值高出γ；$|y_s^w|$和$|y_s^l|$分别代表在y^w和y^l中被选中的词元总数（即$s(y_i)=1$的词元个数）。通过利用选中词元的数量进行归一化处理，旨在消除因选中词元数量差异导致的偏差，从而确保奖励值能够进行公正的比较。该目标函数的核心目标在于最大化偏好响应中选中词元的平均对数概率，同时最小化非偏好响应中选中词元的平均对数概率。

ConfPO 的显著优势在于其轻量且高效的设计，无需依赖额外模型或进行多次前向传播，仅通过模型内部的置信度信号便能实现精确的词元选择。在多个基准测试中，其表现一致优于均匀优化方法，并有效缓解了过优化问题。然而，其阈值策略的普适性以及在各类模型与任务上的泛化能力仍有待进一步验证。此外，对于长文本或复杂推理任务中词元重要性分布的假设，可能并不总是适用。

算法 37. Self-Consistency Preference Optimization（ScPO）

输入：初始模型M_0，未标注的种子问题集D_0，超参数α,β,k,τ,T。

输出：优化后的模型M_T

1) 偏好构建：针对每个问题x，随机采样k个回答$y\sim x$，依据最终答案的投票数$V(y)$来选取正例y^+（得票数最高）和负例y^-（得票数最低）。仅当$V(y^+)\geq\tau$时，保留该对样本。

2) 损失函数：ScPO 加权损失函数如公式（2）所示：

$$L_{ScPO}(y^+,y^-|x)=-w(x)\log\sigma\left(\beta\log\frac{M_\theta(y^+|x)}{M_t(y^+|x)}-\beta\log\frac{M_\theta(y^-|x)}{M_t(y^-|x)}\right)-\frac{\alpha w(x)}{|y^+|}\log M_\theta(y^+|x), \tag{101}$$

其中，$w(x)=\frac{(V(y^+)-V(y^-))}{k}$代表权重，用以反映偏好的质量（即置信度）；$\sigma$指的是 sigmoid 函数；$\beta$和$\alpha$作为超参数，分别调控偏好优化与监督损失的强度；$M_t$表示当前迭代过程中的参考模型；$M_\theta$则是指待优化的模型参数。

3) 迭代训练：从初始模型 M0 出发，每轮利用当前模型生成问题和偏好对，并通过 LScPO 进行模型更新，迭代进行（通常设置为 T=2 轮）。

ScPO 能够在无需黄金标签的情况下，显著提升模型在复杂推理任务中的性能，甚至在某些任务上能与监督学习方法相媲美，且不会增加推理成本。然而，这种方法依赖于模型自身的采样一致性，如果初始模型的一致性较差，可能会对训练效果产生不利影响。

算法 38. Rank Preference Optimization（RankPO）

输入：查询x；候选对(y_w,y_l)，其中y_w为优选候选，y_l为劣选候选；参考模型π_{ref}；超参数：温度τ，对齐强度β，损失函数f。

输出：更新后的策略π_θ

1) 相似度定义：相似度的定义如公式（102）所示：

$$sim(z_x,z_y)=\text{余弦相似度}(z_x,z_y), \tag{102}$$

其中，z_x和z_y分别代表查询和候选的嵌入表示。

2) 损失函数：对于查询x和一对候选y_w（优于）与y_l（劣于），优化模型以使得查询与优选候选的嵌入相似度高于与劣选候选的相似度。损失函数如公式（103）所示：

$$L_{RankPO}=E_{(x,y_w,y_l)}[f(\beta\cdot\Delta^{\pi_\theta}-\beta\cdot\Delta^{\pi_{ref}})], \tag{103}$$

其中，$\Delta^{\pi}=\frac{sim^{\pi}(z_x,z_{y_w})}{\tau}-\frac{sim^{\pi}(z_x,z_{y_l})}{\tau}$；$\pi_\theta$为待优化策略模型，$\pi_{ref}$为参考模型；$\beta$是控制对齐强度的系数，$\tau$是温度参数；$f$是损失函数形式。

RankPO 在排序任务中展现出卓越的平衡能力，既能够高效地对齐 AI 或人工的偏好，又能有效缓解灾难性遗忘问题，非常适合应用于需要持续适应和知识保留的实际系统。然而，其性能表现高度依赖于对比学习阶段的嵌入质量，引入了更高的计算成本。

算法 39. Weighted Preference Optimization（WPO）

输入：初始策略模型π_θ，离策略偏好数据集$D=\{(x,y_w,y_l)\}$，参考模型π_{ref}，超参数β。

输出：更新后的策略π_θ

1) 权重计算：使用长度归一化的序列概率作为权重，具体计算如公式（104）所示：

$$w(x,y)=exp\left(\frac{1}{|y|}\sum_{t=1}^{|y|}log\,\pi_\theta(y_t|x,y_{<t})\right), \tag{104}$$

其中，$|y|$是输出序列的令牌数，$\pi_\theta(y_t|x,y_{<t})$是当前策略模型在给定上下文下生成令牌y_t的概率。

2) 损失函数：WPO 的损失函数如公式（105）所示：

$$L_{WPO}=-E_{(x,y_w,y_l)\sim D}\left[w(x,y_w)\cdot w(x,y_l)\cdot log\,p(y_w\succ y_l|x)\right], \tag{105}$$

其中，$p(y_w\succ y_l|x)$是基于 Bradley-Terry 模型的偏好概率；$w(x,y)$是权重，且在反向传播过程中被分离。

WPO 有效缓解了离策略训练中的分布差异问题，提升了训练的稳定性和最终性能，且无需额外采样成本。然而，该方法仍无法完全弥合离策略与在策略方法之间的性能差距，并且在模型置信度较低时可能引入偏差。

算法 40. 2-Dimensional Direct Preference Optimization (2D-DPO)

输入：超参数集 $\lambda_1,\lambda_2,\lambda_3,\beta$

输出：更新策略 π_θ

1) 加权分段奖励机制：第k段的加权奖励值$r_{w,k}$和 $r_{l,k}$分别如式（106）所示：

$$r_{w,k}=W\cdot\overrightarrow{r_{w,k}},\quad r_{l,k}=W\cdot\overrightarrow{r_{l,k}}, \tag{106}$$

其中，$W=[w_1,w_2,\ldots,w_A]$是所有维度的加权向量，满足$\sum_{j=1}^{A}w_j=1$，$F_{w,k}=[r_{w,k,1},r_{w,k,2},\cdots,r_{w,k,A}]$表

示第k段在A个维度上的得分向量。

2) 等效温度调节：为保持与原始 DPO 的一致性，2D-DPO 对每个段落的等效温度进行相应调整，具体如式（107）和（108）所示：

$$\beta_{eff,k}=\frac{\beta}{r_{w,k}}(for\ preferred\ paragraphs), \tag{107}$$

$$\beta_{eff,k}=\frac{\beta}{r_{l,k}}(for\ inferior\ paragraphs). \tag{108}$$

3) 段落筛选：为提升训练效率，2D-DPO 系统将精心筛选高评分回答中得分最高的 N 段，同时从低评分回答中挑选得分最低的 N 段，以此进行成对优化，具体如式（109）所示：

$$N=min\,(S_w,S_l), \tag{109}$$

其中，S_w和S_l分别代表优选段落和次优段落的数量。

4) 2D-DPO 目标函数：目标函数如式（110）所示：

$$L_{2D\text{-}DPO}=-E_{(\tau_w,\tau_l)}\left[\sum_{k=0}^{N\text{-}1} log\,\sigma\left(\beta\sum_{t=n_k}^{n_k+l_k} r_{w,k}\,log\frac{\pi_\theta(y_t^w|x_t^w)}{\pi_{ref}(y_t^w|x_t^w)}-\beta\sum_{t=n_k}^{n_k+l_k} r_{l,k}\,log\frac{\pi_\theta(y_t^l|x_t^l)}{\pi_{ref}(y_t^l|x_t^l)}\right)\right]. \tag{110}$$

算法 41. InCo-DPO

输入：策略模型π_θ，参考模型π_{ref}，强外部模型π_{strong}，指令数据集{D}，超参数：前缀长度y_p，温度T,β。

输出：更新策略π_θ

1) 引入“续写”机制，以合成高质量偏好数据:

- 强模型生成前缀y_p，策略模型π_θ完成后缀y_s，最终形成完整响应$y=[y_p,y_s]$；
- 通过调节前缀长度$|y_p|$和采样温度T，动态平衡奖励与一致性。

2) 量化策略一致性与数据质量：一致性权重用于衡量响应与政策模型之间的分布一致性，如公式（111）所示，而奖励模型$r(x,y)$则用于评估响应质量：

$$w(x,y)=exp\left(\frac{1}{|y|}\sum_{t=1}^{|y|} log\frac{\pi_\theta(y_t|x,y_{<t})}{\sum_{v\square V}\pi_\theta(v|x,y_{<t})^2}\right), \tag{111}$$

其中，$\pi_\theta(y_t|x,y_{<t})$表示策略模型对位置t处实际标记y_t的预测概率；$\sum_{v\square V}\pi_\theta(v|x,y_{<t})^2$表示整个词汇表$V$中各词汇输出概率的平方总和，旨在进行归一化处理；权重$w(x,y)\in[0,1]$值越高，表明响应与策略模型之间的一致性越强。

3) 如式（112）所示，优化目标仍为 DPO 损失函数：

$$\mathrm{L}_{DPO}(\pi_\theta,\pi_{ref})=-E_{(x,y_w,y_l)}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\,log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right], \tag{112}$$

其中，y_w,y_l分别代表优选和消极响应；β代表 KL 惩罚系数，其作用在于调节模型与参考模

型之间的偏差程度。

InCo-DPO 通过低成本的“前缀注入”操作，有效突破了传统策略模型的固有局限。该方法不仅充分发挥了外部模型的智能优势，还展现出卓越的通用性，显著提升了不同模型及 DPO 变体的性能表现。然而，这种方法也存在明显短板：首先，它依赖于强大的外部模型来生成高质量前缀，这无疑会增加系统的复杂度和计算成本；其次，其性能受前缀长度、采样温度等超参数的影响较大，需经过精细调整才能找到最佳平衡点。

算法 42. Few-Shot Preference Optimization（FSPO）

输入：每个用户的偏好数据集$D_i=\{(x,y_1,y_2,c)\}$，可选的用户描述 $y_{S^{(i)}}$。

输出：更新策略π_θ

1) 元学习目标：将每位用户视作一项独立任务，旨在最小化全体用户上的期望损失，具体表述如公式（113）所示：

$$\min_{\theta} E_{S^{(i)}\sim S}\left[E_{(x,y_1,y_2,c)\sim D_i}\left[\mathrm{L}^{\theta}_{pref}(x,y_1,y_2,c)\right]\right], \tag{113}$$

其中，$S(i)$表示用户，D_i是该用户的偏好数据，$\mathrm{L}^{\theta}_{pref}$是偏好损失函数。

2) 偏好损失函数：偏好损失函数具体如公式（114）和（115）所示：

$$\mathrm{L}^{\theta}_{pref}=\left\|h^{y_w,y_l}_{\pi_\theta}-(2\beta)^{-1}\right\|^2_2, \tag{114}$$

$$h^{y_w,y_l}_{\pi_\theta}=log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}, \tag{115}$$

其中，y_w和y_l分别是偏好和非偏好响应，π_θ是当前策略模型，π_{ref}是参考模型，β是控制正则化强度的超参数。

3) 用户描述链式思考（COT）：在推理阶段，模型首先生成用户描述，再基于该描述生成响应，以提升个性化和可解释性。

FSPO 在个性化响应生成方面表现卓越，尤其在少样本场景中，能够高效捕捉用户偏好，并在从合成数据到真实用户的迁移过程中展现出优秀的泛化能力。然而，该方法存在可能强化用户偏见或导致信息茧房的风险，且其性能受合成数据的多样性与结构质量的制约。

算法 43. Self-Augmented Preference Optimization（SAPO）

输入：初始 SFT 数据集$D=\{(x,y^+)\}$，当前策略模型π_θ，EMA 模型π_{EMA}，回放缓冲区B。

输出：更新后的策略π_θ

1) 分段级监督：将优选回复y^+随机截断为三段：A、B、C，如公式（116）所示，其中B为待替换段，由 EMA 模型基于A段生成新段B'，构成负样本，如公式（117）所示：

$$y^+=A\oplus B\oplus C, \tag{116}$$

$$y^-=A\oplus B'\oplus C. \tag{117}$$

2) EMA 模型更新：EMA 模型的更新过程如公式（118）所示：

$$\theta_{EMA}\leftarrow\alpha\cdot\theta_{EMA}+(1-\alpha)\cdot\theta, \tag{118}$$

其中，θ为当前策略参数；α为衰减系数，用于平滑模型更新。

3) 回放缓冲区与采样：采用 FIFO 结构的回放缓冲区存储训练样本，支持课程式学习，逐步将简单样本替换为复杂样本，以提升训练的稳定性。

4) 损失函数：SAPO 可兼容 DPO 或 ORPO 等损失函数。使用 DPO 损失的情况如公式（119）所示：

$$L_{DPO}=-E_{(x,y^+,y^-)}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y^+|x)}{\pi_{ref}(y^+|x)}-\beta\log\frac{\pi_\theta(y^-|x)}{\pi_{ref}(y^-|x)}\right)\right], \tag{119}$$

其中，β为 KL 正则化系数，π_{ref}为参考模型，σ为 sigmoid 函数。

SAPO 的显著优势在于其卓越的数据高效性和训练动态性，它能够在无需依赖外部奖励模型或配对数据的前提下，实现持续的自我优化。然而，其表现依旧受制于初始 SFT 数据的质量，同时训练过程对超参数的敏感性较高，这可能导致在某些特定任务或领域中，出现过度拟合或收敛不稳定的现象。

算法 44. Self-Generated Preference Optimization（SGPO）

输入：初始策略模型π_θ，监督微调数据集S_P，用于改进器训练的小规模 SFT 数据集S_R，外部 LLMπ_ϕ。

输出：更新后的策略π_θ

1) 改进器训练：利用外部大型语言模型生成目标改进响应，构建训练数据集 DR，旨在最小化损失函数，具体公式如（120）所示：

$$\theta^*=\arg\max_\theta E_{(x,\hat{y}_{\theta_0},\tilde{y})\sim D_R}\left[\log\pi_\theta(\tilde{y}|r(x,\hat{y}_{\theta_0}))\right], \tag{120}$$

其中，π_θ代表当前策略模型，$\hat{y}_{\theta_0}$为初始策略生成的响应，$\tilde{y}$为外部大型语言模型生成的改进响应，r则为改进提示模板。

2) 偏好数据生成与优化：通过训练好的改进器生成偏好对$D_P=\{(x,\tilde{y}_{\theta^*},\hat{y}_{\theta^*})\}$，其中改进响应标记为“选中”，策略响应标记为“拒绝”。优化目标详见公式（121）：

$$\theta^{**}=\arg\min_\theta E_{(x,\tilde{y}_{\theta^*},\hat{y}_{\theta^*})\sim D_P}\left[-\log\sigma\left(\beta\log\frac{\pi_\theta(\tilde{y}_{\theta^*}|x)}{\pi_{\theta^*}(\tilde{y}_{\theta^*}|x)}-\beta\log\frac{\pi_\theta(\hat{y}_{\theta^*}|x)}{\pi_{\theta^*}(\hat{y}_{\theta^*}|x)}\right)\right], \tag{121}$$

其中，σ代表 sigmoid 函数，β是用于控制与参考策略偏离程度的超参数，π_{θ^*}则是指改进后的策略模型。

SGPO 成功实现了完全自生成的在线偏好优化，显著减少了对人工标注数据的依赖。此外，通过策略-改进器一体化设计，有效提升了训练的一致性和效率。然而，该方法仍需依赖外部 LLM 生成初始改进目标，且在多次迭代过程中可能面临改进器性能退化的风险。

算法 45. Zeroth-Order Preference Optimisation（ZOPrO）

输入：初始策略模型π_{ref}，初始奖励模型R_0，提示数据集D_{prompt}，迭代次数N，超参数：学习率η，扰动幅度ε，衰减系数α。

输出：更新后的策略π_θ

1) 基础 SPSA 梯度：基础 SPSA 梯度如公式（122）所示：

$$\hat{g}_i \approx \frac{J(\theta_i+\varepsilon_i z_i)-J(\theta_i-\varepsilon_i z_i)}{2\varepsilon_i} z_i, \tag{122}$$

其中，J代表目标函数，θ_i为第i步的策略模型参数，ε_i表示扰动幅度，z_i则为扰动方向向量。

2) 自适应扰动策略：ZOPrO 以 SPSA 框架为基础，结合 RLOO 目标函数，并引入结构化扰动策略。自适应扰动策略的具体表达式如公式（123）所示：

$$z_i=\alpha_i \Delta\pi_t+\sqrt{1-\alpha_i^2}\cdot\frac{u_i}{\|u_i\|}\|\Delta\pi_t\|, \tag{123}$$

其中，$\Delta\pi_t=\pi_\theta^{(t)}-\pi_\theta^{(t-1)}$表示上一轮策略模型参数的变化；$u_i \sim N(0,I_d)$并投影到$\Delta R_t$的正交空间中；$\alpha_i$为衰减系数，用于控制历史更新影响的权重；$\Delta R_t=R_\phi^{(t)}-R_\phi^{(t-1)}$表示奖励模型参数的变化。

ZOPrO 在减少内存占用和加快单次迭代速度方面表现卓越，特别适合应用于资源受限或梯度信息不可获取的环境。然而，其每一步的奖励提升幅度较小，收敛速度依然不及一阶方法，并且对超参数的设置较为敏感。

算法 46. Calibrated Direct Preference Optimization (Cal-DPO)

输入：超参数$\lambda_1, \lambda_2, \lambda_3, \beta$

输出：更新策略π_θ

1) 隐性奖励：隐性奖励的定义与 DPO 一致，具体如式（124）所示：

$$\hat{r}_\theta(x,y)=log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}. \tag{124}$$

2) 校准损失：通过引入平方损失校准项，隐式奖励的尺度被强制调整至接近真实奖励的尺度，具体如公式（125）所示：

$$L_{Cal}(\theta;x,y)=\left(log\frac{\pi_\theta(y|x)}{\pi_{ref}(y|x)}-\frac{r(x,y)}{\beta}\right)^2, \tag{125}$$

其中：$r(x,y)$是真实奖励，用来调整对数比值的期望；β代表温度系数，主要用于调节 KL 散度的权重。

3) Cal-DPO 损失函数如式（126）所示：

$$\begin{aligned} L_{Cal\text{-}DPO}(\theta;x,y_w,y_l) &= -log\,\sigma\left(log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right) \\ &+\left(log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\frac{1}{2\beta}\right)^2+\left(log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}+\frac{1}{2\beta}\right)^2. \end{aligned} \tag{126}$$

算法 47. Balanced Preference Optimization（BPO）

输入：输入序列x，选定响应y_w，拒绝响应y_l，参考策略模型π_{ref}，当前策略模型π_θ，间隙适配器参数α，温度参数β，偏好数据集D。

输出：更新策略π_θ

1) 平衡奖励边际：平衡奖励边际的具体公式如（127）所示：

$$\rho_\theta^b = min(r_w, -\alpha r_l), \tag{127}$$

其中，$r_w = \beta log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}$表示选定响应相对于参考策略的对数概率比，用以衡量其质量提升的程度；$r_l = \beta log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}$表示拒绝响应相对于参考策略的对数概率比，用以衡量其质量下降的程度；$\alpha \in (0,1]$代表间隙适配器参数，用于控制对拒绝响应的惩罚强度；$min(\cdot)$用于动态选择当前需要优化的薄弱环节。

2) 间隙适配器机制：

- 当$r_w < -\alpha r_l$时：优先优化选定响应，提升r_w；
- 当$r_w \geq -\alpha r_l$时：优先抑制拒绝响应，降低r_l；
- α值越小，对拒绝响应的惩罚越宽松；α值越大，对选定响应的关注越集中。

3) 损失函数：最终损失函数如公式（128）所示：

$$L(\theta) = -E_{(x,y_w,y_l)\sim D}\left[f\left(min\left(\beta log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}, -\alpha\beta log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right)\right], \tag{128}$$

其中，$f(\cdot)$为偏好优化函数，负责将奖励差异转换为概率值；β为温度参数，用于调控策略模型与参考模型之间的偏离程度；D为一个包含(x,y_w,y_l)三元组的偏好数据集。

算法 48. Linear Preference Optimization（LPO）

输入：提示x，优选回答y_w，拒绝回答y_l，参考模型π_{ref}，待优化的策略模型π_θ，超参数β,λ,r_1,r_2。

输出：更新策略π_θ

1) 梯度解耦与绝对差损失：LPO 采用绝对差函数替换 DPO 中的 log-sigmoid 函数，损失函数定义如公式（129）所示：

$$L_{LPO} = 2\beta\left|x_1 - x_2 - \frac{1}{2\beta}\right| + \lambda \cdot max(0, -x_1), \tag{129}$$

其中，$x_1 = \frac{1}{len_w} log \frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}$是选择响应的长度归一化对数概率比；$x_2 = \frac{1}{len_l} log \frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}$是拒绝响应的长度归一化对数概率比；$\beta$是用于调节偏移量的超参数；$\lambda$则表示正样本正则项的强度。

2) 梯度分离与可控下降：借助 Straight-Through Estimator（STE）实现梯度路径的有效分离，具体过程如公式（130）所示，进而得出最终损失，详见于公式（131）：

$$\begin{cases} L_{LPO\text{-}ste}^{x_1} = r_1 \cdot 2\beta|x_1 - x_2.detach() - \frac{1}{2\beta}| + \lambda \cdot max(0, -x_1) \\ L_{LPO\text{-}ste}^{x_2} = r_2 \cdot 2\beta|x_1.detach() - x_2 - \frac{1}{2\beta}| + \lambda \cdot max(0, -x_1.detach()) \end{cases}, \tag{130}$$

$$L_{LPO\text{-}ste} = \frac{2}{r_1 + r_2}\left(r_1 \cdot L_{LPO\text{-}ste}^{x_1} + r_2 \cdot L_{LPO\text{-}ste}^{x_2}\right), \tag{131}$$

其中，r_1和r_2分别控制选择响应和拒绝响应的梯度下降速率。

LPO 在梯度解耦、训练稳定性和可控性方面明显优于 DPO，尤其在数学推理和语音任务中表现出色。然而，LPO 引入了更多超参数，导致调优复杂度提升；此外，其依赖 STE 技术可能会引起梯度估计的偏差。

算法 49. Proximalized Preference Optimization（PRO）

输入：反馈数据集D，参考模型π_{ref}，初始策略模型π_θ，超参数α,β。

输出：更新后的策略π_θ

PRO 的损失函数由优化器和正则项两部分组成，其中优化器的具体形式如公式（132）所示，正则项则如公式（133）所示。此外，系统引入了超响应机制，将未标注的响应聚合为一个"超响应"H，从而使得正则项能够在压缩的响应空间y_H上高效计算，避免了枚举所有响应的繁琐过程：

$$-\beta E_{y\sim\hat{\mu}}[\hat{s}(y)\cdot log\,\pi_\theta(y)], \tag{132}$$

其中，β表示控制偏好优化的强度，$\hat{\mu}$代表经验响应分布，$\hat{s}(y)$是响应y的偏好得分，$\pi_\theta(y)$则为模型对响应y的概率。

$$\frac{\alpha}{2}E_{y_1,y_2\sim\mu}\left[D_{KL}\left(\mathrm{B}\left(\frac{1}{2}\right)\|\mathrm{B}\left(\sigma\left(r_\theta(y_1)-r_\theta(y_2)\right)\right)\right)\right], \tag{133}$$

其中，α代表正则化强度，μ表示响应分布（包括超响应H），$r_\theta(y)=\beta\,log\frac{\pi_\theta(y)}{\pi_{ref}(y)}$定义为隐式奖励，$\sigma$为 sigmoid 函数，B则代表伯努利分布。

PRO 的优势在于其能够统一支持多种反馈类型，有效缓解似然不确定性和奖励破解问题，并在实验中展现出优于或与现有方法相媲美的性能表现。然而，PRO 的引入也带来了额外的超参数，导致调优成本相对较高。

算法 50. ReLU-based Preference Optimization（RePO）

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，初始策略模型π_θ，目标奖励间隔$\gamma\in[0,1]$。

输出：更新后的策略π_θ

1) 隐式奖励间隔：RePO 隐式奖励间隔具体计算公式如（134）所示：

$$M_\theta=\frac{log\,\pi_\theta(y_w|x)}{|y_w|}-\frac{log\,\pi_\theta(y_l|x)}{|y_l|}, \tag{134}$$

其中，π_θ是当前策略模型，y_w和y_l分别代表偏好和非偏好响应，$|y|$表示响应的长度。

2) 损失函数：RePO 损失函数如公式（135）所示：

$$\mathrm{L}_{RePO}(\pi_\theta)=E_{(x,y_w,y_l)\in D}[ReLU(-(M_\theta-\gamma))], \tag{135}$$

其中，γ是目标奖励间隔阈值，用于调节模型对样本的筛选强度；当$M_\theta\geq\gamma$时，梯度为零，模型停止更新；反之，模型则继续进行优化。

RePO 仅需单一超参数即可达到竞争性性能，并通过 ReLU 机制自动筛选简单样本，集中优化复杂样本，从而提升训练的稳定性。然而，该机制对"未充分区分"的样本赋予相同的梯度权重，未能深入区分样本的难度层次，限制了模型对高价值样本的深度挖掘潜力。

算法 51. Robust Preference Optimization via Dynamic Target Margins（γ-PO）

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，初始目标边界γ_0，正则化系数τ，基础对齐方法及其参数。

输出：更新后的策略π_θ

1) 目标函数：γ-PO 的目标函数如公式（136）所示：

$$L_{\gamma\text{-}PO}=-E_D\left[\log\sigma(r_w-r_l-\gamma_i)\right], \tag{136}$$

其中，r_w和r_l分别代表偏好响应和非偏好响应的奖励值；γ_i是每个样本对的动态目标边界；σ指的是 sigmoid 函数。

2) 优化目标：动态边界γ_i的优化目标如公式（137）所示：

$$\gamma_i=\arg\min_{\gamma_i}\left[-E_D\left[\log\sigma(r_w-r_l-\gamma_i)\right]+\tau\cdot D_{KL}(p_i\|p_0)\right], \tag{137}$$

其中，$p_i=\frac{\gamma_i}{\sum_j\gamma_j}$表示归一化的边界分布；$p_0$代表均匀分布，用作正则化目标；$\tau$是用于调控边界分布与均匀分布之间差异的正则化系数。

3) 边界调整机制：

- 当奖励差异$m=r_w-r_l$较大时，γ_i增加，专注于高置信样本的学习；
- 当m值较小时，γ_i降低，减少对模糊样本的依赖；
- 通过引入 KL 散度正则项，防止边界值过大或过小，确保训练过程的稳定性。

γ-PO 的优点在于其自适应边界机制能够有效提升模型对噪声数据的鲁棒性，并且作为即插即用的模块，几乎不会增加计算开销。然而，γ-PO 引入了额外的超参数τ，其调优过程对性能影响较大。

算法 52. Vote-based Preference Optimization（VPO）

输入：上下文x，响应对(y_1,y_2)，对应票数(v_1,v_2)，参考模型π_{ref}，超参数c,β。

输出：更新后的策略π_θ

1) 目标偏好概率建模：采用贝叶斯最小均方误差（MMSE）估计器来计算目标偏好概率，具体计算方法如公式（138）所示：

$$\hat{\theta}_{MSE}(v_1,v_2)=\frac{v_1+c}{v_1+v_2+2c}, \tag{138}$$

其中，v_1和v_2分别为两个响应所获得的票数；c为超参数，用于控制先验分布的影响（通常设定为 1）；$\hat{\theta}_{MSE}$表示响应 1 优于响应 2 的估计概率。

2) 损失函数：VPO 基于 DPO 扩展的损失函数如公式（139）所示：

$$\max_\theta E_D\left[p(Y_1)\log\sigma(r_1-r_2)+p(Y_2)\log\sigma(r_2-r_1)\right], \tag{139}$$

其中，$r_i=\beta\log\frac{\pi_\theta(y_i|x)}{\pi_{ref}(y_i|x)}$代表隐式奖励函数；$p(Y_i)=\hat{\theta}_{MMSE}$是 VPO 估计的目标概率。

VPO 通过引入投票信息，显著增强了对主观偏好的建模能力，并在实验中展现出更优的生成质量和更高的训练稳定性，尤其在处理争议性样本时表现尤为突出。然而，其效果受到投票数据质量和数量的制约，在投票信息缺失或噪声较大的情况下，其性能可能受到限制。

算法 53. AlphaPO

输入：当前策略模型π_θ，参考模型π_{ref}，偏好数据集$D=\{(x,y_w,y_l)\}$，超参数β,α,γ。

输出：更新策略π_θ

1) 奖励函数定义：AlphaPO 所采用的奖励函数如公式（140）所示：

$$r_\alpha(y;x)=\beta\left(\frac{1-\pi_\theta(y|x)^{-\frac{\alpha}{|y|}}}{\alpha}\right), \tag{140}$$

其中，$|y|$是响应y的长度（token 数量）；β是奖励缩放系数；α是控制奖励函数形状的关键参数。

2) 损失函数：基于 Bradley-Terry 偏好模型，AlphaPO 的损失函数如公式（141）所示：

$$L_{AlphaPO}=-E_{(x,y_w,y_l)\sim D}\left[log\,\sigma\left(-\frac{\beta}{\alpha}\pi_\theta(y_w|x)^{-\frac{\alpha}{|y_w|}}+\frac{\beta}{\alpha}\pi_\theta(y_l|x)^{-\frac{\alpha}{|y_l|}}-\gamma\right)\right], \tag{141}$$

其中，y_w是偏好响应，y_l是非偏好响应，γ是边际项，用于增强偏好与非偏好响应之间的区分度。

AlphaPO 的主要优点在于其通过α参数灵活调节奖励函数形状，有效缓解了似然偏移和过度优化问题，提升了模型在复杂指令任务上的表现。然而，该方法引入了额外的超参数α，增加了调优的复杂性，且在某些模型上的提升不如其他模型显著，表明其性能可能受模型结构和数据质量的影响。

算法 54. InfoPO

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，参考模型π_{ref}，初始策略模型π_θ。

输出：更新策略π_θ

1) InfoPO 的目标函数如公式（142）所示：

$$\mathrm{L}_{InfoPO}(\theta;x,y_w,y_l)=-\,log\,\pi_\theta(y_w|x)+\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}, \tag{142}$$

其中，x代表输入提示，y_w表示选中（偏好）响应，y_l表示拒绝（非偏好）响应。

2) 梯度分析：

- 第一项旨在提升模型对选中响应的预测准确性，通过增加负对数似然（NLL）损失来实现；
- 第二项则通过参考模型的概率归一化，限制对拒绝响应的梯度幅度，从而防止模型过度更新。

InfoPO 的优势在于其简洁高效的设计，无需依赖 BT 假设，能够稳定地进行训练，并显著提升推理任务的性能。然而，其不足之处在于目前仅对 NWJ 这一种互信息估计器进行了探索，未来需进一步研究其他估计器的适用性。

算法 55. Maximum a Posteriori Preference Optimization（MaPPO）

输入：初始策略模型π_θ，参考模型π_{ref}，偏好数据集$D=\{(y_w,y_l,x)\}$，先验奖励差异$\Delta_r=r_w-r_l$，超参数β。

输出：更新策略π_θ

MaPPO 在 DPO 的 MLE 损失基础上引入了一个先验项，构建了最大后验优化目标。其损失函数具体形式如公式（143）所示：

$$L_{MaP}(\theta)=E_{(y_w,y_l,x)\sim D}\left[-\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\Delta_r\beta\log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right], \tag{143}$$

其中，σ为 sigmoid 函数，β是 KL 正则化权重，π_θ为待优化策略模型，π_{ref}为参考模型，$\Delta_r=r_w-r_l$是先验奖励差异，用于调节对次选响应的惩罚强度。

MaPPO 的优势在于其通过引入先验奖励信息，有效缓解了 DPO 中的置信度退化问题，从而提升了模型的校准能力和对齐效果，并在多种模型规模与任务中展现出一致且稳健的表现。然而，其性能高度依赖于先验奖励模型的质量，若奖励模型存在偏差或噪声，可能会对优化效果产生不利影响。

算法 56. Trust Region Direct Preference Optimization（TR-DPO）

输入：参考模型π_{ref}，训练数据$D=\{(x,y_w,y_l)\}$，超参数β,α,τ。

输出：更新策略π_θ

1) 软更新策略：在每个训练步骤中，参考策略π_{ref}的参数会通过加权平均的方式，平滑地融入当前训练策略π_θ的参数。其更新公式如（144）所示：

$$\pi_{ref}\leftarrow sg\left(\alpha\pi_\theta+(1-\alpha)\pi_{ref}^{prev}\right), \tag{144}$$

其中，$\alpha\in[0,1]$是权重系数，控制当前策略π_θ对参考策略的融合程度；sg表示停止梯度操作，确保参考策略不被直接优化。

2) 硬更新策略：每隔τ步，直接将参考策略替换为当前策略，以实现策略的阶段性重置，更新公式如（145）所示：

$$\pi_{ref}\leftarrow sg(\pi_\theta). \tag{145}$$

3) 损失函数：TR-DPO 采用与 DPO 相同的损失函数，如（146）所示，但参考策略π_{ref}在训练过程中是动态调整的：

$$L_{TR\text{-}DPO}=-E_{(x,y_w,y_l)\sim D}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_w|x)\pi_{ref}(y_l|x)}{\pi_{ref}(y_w|x)\pi_\theta(y_l|x)}\right)\right], \tag{146}$$

其中，β是控制对齐强度的超参数，σ是 sigmoid 函数，y_w和y_l分别表示偏好和非偏好响应。

TR-DPO 通过动态更新参考策略，有效缓解了过优化问题，显著提升了模型在高度偏离初始策略时的鲁棒性和生成质量。尤其在通用助手任务中，其表现远超传统 DPO。然而，该方法对超参数较为敏感，需针对不同任务和模型规模进行精细调优，这在一定程度上增加了训练的复杂性和计算开销。

算法 57. Pre-DPO

输入：初始策略模型π_{SFT}，偏好数据集$D=\{(x_i,y_i^+,y_i^-)\}$，偏好优化方法M。

输出：更新策略 $\pi_{Pre\text{-}DPO}$

1) 初始化参考模型：根据使用的偏好优化方法是否为基于参考模型的方法，设置参考模型为 SFT 模型或不设置参考模型，具体如公式（147）所示：

$$\pi_{ref}=\begin{cases}\pi_{SFT} & \text{若 } M \text{ 为基于参考模型的方法}\\ None & \text{若 } M \text{ 为无参考模型方法}\end{cases}, \tag{147}$$

其中M为偏好优化方法。

2) 第一轮偏好优化：使用偏好优化方法M对初始策略π_{SFT}进行训练，得到优化后的策略π_M，具体如公式（148）所示：

$$\pi_M=M(\pi_{SFT};\pi_{ref}). \tag{148}$$

3) 设置引导参考模型：将第一轮优化得到的模型π_M设置为引导参考模型，具体如公式（149）所示：

$$\pi_{guide}=\pi_M. \tag{149}$$

4) 使用引导参考模型进行 DPO 优化：使用引导参考模型π_{guide}对初始策略π_{SFT}再次进行 DPO 优化，得到最终模型$\pi_{Pre\text{-}DPO}$，具体如公式（150）所示：

$$\pi_{Pre\text{-}DPO}=M_{DPO}\left(\pi_{SFT};\pi_{guide}\right). \tag{150}$$

借助引入引导参考模型，Pre-DPO 显著提升了 DPO 的数据利用率和模型性能，在多项基准测试中均呈现出了稳定的性能表现，且无需依赖外部模型或额外数据进行支持。然而，这种方法在提升性能的同时，也相应增加了训练所需的轮次，进而提升了计算成本。此外，它对引导参考模型的质量要求较高，若第一轮优化未能达到理想效果，可能会对最终的性能产生负面影响。

算法 58. Odds-Ratio Preference Optimization（ORPO）

输入：初始预训练模型π_0；成对偏好数据集$\{(x,\hat{y},y)\}$，其中$\hat{y}$为优选回复，y为被拒回复；超参数β。

输出：更新后的策略π_θ

ORPO 的损失函数由 KL 正则项和偏好对齐项两部分组成，其中 KL 正则项旨在约束模型，确保其不偏离原始预训练分布；偏好对齐项则通过几率比的对数形式，提升优选回复的生成概率，同时抑制被拒回复的产生。具体公式如下（151）所示：

$$\mathrm{L}_{ORPO}=KL(\pi_\theta\|\pi_0)-\beta\, log_2\frac{\pi_\theta(\hat{y}|x)}{\pi_\theta(y|x)}, \tag{151}$$

其中，π_θ表示当前待优化的策略模型，π_0代表预训练模型（即初始策略），x为输入提示，$\hat{y}$为优选回复，y为被拒回复，β则为控制偏好信号强度的超参数。

ORPO 训练流程简洁高效，无需维护参考模型，有效降低了内存与计算需求，同时在偏好对齐和策略稳定性之间实现了良好平衡。然而，ORPO 对偏好数据的质量依赖性较强，若优选样本与被拒样本的区分不够明确，可能导致优化目标模糊。

算法 59. Simple Preference Optimization（SimPO）

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，其中x为输入提示，y_w为获胜响应，y_l为失败响应；策略模型 π_θ；超参数β与γ。

输出：更新后的策略π_θ

1) 奖励设计：SimPO 的奖励设计如公式（152）所示：

$$r_{SimPO}(x,y)=\frac{\beta}{|y|}\log\pi_\theta(y|x), \tag{152}$$

其中，$|y|$表示响应y的长度（即 token 数量），$\pi_\theta(y|x)$代表策略模型在给定x条件下生成y的概率，β为奖励缩放系数。

2) 目标函数：SimPO 的目标函数如公式（153）所示：

$$\mathrm{L}_{simPO}(\pi_\theta)=-E_{(x,y_w,y_l)}\left[\log\sigma\left(\frac{\beta}{|y_w|}\log\pi_\theta(y_w|x)-\frac{\beta}{|y_l|}\log\pi_\theta(y_l|x)-\gamma\right)\right], \tag{153}$$

其中，y_w代表获胜响应；y_l代表失败响应；γ表示目标奖励间隔，用于调控奖励差异的最小阈值；σ为 sigmoid 函数。

SimPO 的主要优势在于其简洁性、高效性和卓越性能：它无需依赖参考模型，从而显著降低了计算和内存开销，并在多个基准测试中明显超越 DPO 及其变体。然而，SimPO 也存在一些局限性，其目标间隔γ需要手动调优，缺乏理论上的自动确定机制。

算法 60. Token-level Direct Preference Optimization（T-DPO）

输入：当前策略模型的词元 logits：$\pi_\theta(\cdot|[x,y^{<t}])$；参考模型的词元 logits：$\pi_{ref}(\cdot|[x,y^{<t}])$；偏好对$(y_w,y_l)$的词元索引；超参数$\beta$（KL 惩罚系数），$\alpha$（KL 平衡系数）。

输出：更新后的策略π_θ

1) 目标函数：T-DPO 在词元级别优化的目标函数，具体如公式（154）所示：

$$\max_{\pi_\theta} E_{x,y^{<t}\sim D,z\sim\pi_\theta(\cdot|[x,y^{<t}])}\left[A_{\pi_{ref}}([x,y^{<t}],z)-\beta D_{KL}\left(\pi_\theta(\cdot|[x,y^{<t}])\|\pi_{ref}(\cdot|[x,y^{<t}])\right)\right], \tag{154}$$

其中，$A_{\pi_{ref}}$为优势函数，用于衡量在状态$[x,y^{<t}]$下选择词元z的相对价值；β为控制模型偏离参考模型程度的超参数；D_{KL}为反向 KL 散度，旨在防止模型过度偏离参考模型。

2) 模型重构：基于 Bradley-Terry 模型的词元级重构，详细表达式如公式（155）所示：

$$P_{BT}(y_1\succ y_2|x)=\sigma\big(u(x,y_1,y_2)-\delta(x,y_1,y_2)\big), \tag{155}$$

其中，$u=\beta\log\frac{\pi_\theta(y_1|x)}{\pi_{ref}(y_1|x)}-\beta\log\frac{\pi_\theta(y_2|x)}{\pi_{ref}(y_2|x)}$表示奖励差异；$\delta=\beta\big(D_{SeqKL}(x,y_2;\pi_{ref}\|\pi_\theta)-D_{SeqKL}(x,y_1;\pi_{ref}\|\pi_\theta)\big)$代表前向 KL 散度差异；$D_{SeqKL}$则是指序列 KL 散度，即对所有词元位置的 KL 散度进行求和。

3) 损失函数：T-DPO 的最终损失函数，详细呈现如公式（156）所示：

$$\mathrm{L}_{T\text{-}DPO}=-E_{(x,y_w,y_l)\sim D}\big[\log\sigma\big(u(x,y_w,y_l)-\alpha\cdot\delta_2(x,y_w,y_l)\big)\big], \tag{156}$$

其中，α是用于控制 KL 散度平衡的超参数；在δ_2中，采用了梯度截断技术，仅针对非偏好响应的 KL 散度进行梯度更新。

T-DPO 在提升对齐效果和生成多样性方面表现卓越，特别是在控制 KL 散度增长和防止模型退化方面，相较于 DPO 方法具有显著优势。然而，其训练过程较为复杂，超参数的调优敏感性较高，且计算开销略大于 DPO。

算法 61. TGDPO（Token-Level Guided Direct Preference Optimization）

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，参考策略模型π_{ref}，初始策略模型π_θ，词元级奖励函数$\hat{r}(s_t,a_t)$，超参数β,α。

输出：更新后的策略π_θ

1) 词元级 PPO 目标重构：将序列级 PPO 目标细化为词元级优化问题，并引入词元奖励函数$\hat{r}(s_t,a_t)$和缩放函数$f(\cdot)$，构建优化目标，具体如公式（157）所示：

$$\max_{\pi_\theta} E_{s_t\sim D,a_t\sim\pi_\theta(\cdot|s_t)}\left[\frac{r_\phi(s_t,a_t)}{\beta\cdot f(\hat{r}(s_t,a_t))}-\log\frac{\pi_\theta(a_t|s_t)}{\pi_{ref}(a_t|s_t)}\right], \tag{157}$$

其中，s_t表示当前状态（上下文），a_t为生成的词元，π_θ为待优化策略，π_{ref}为参考策略，β为控制偏离程度的超参数，$f(\cdot)$为基于词元奖励$\hat{r}(s_t,a_t)$的缩放函数。

2) 最优策略与奖励表示：通过求解上述优化问题，得出词元级最优策略及其相应的奖励表示，详细如公式（158）所示：

$$\frac{r_\phi(s_t,a_t)}{f(\hat{r}(s_t,a_t))}=\beta\log\frac{\pi_{\theta_t}(a_t|s_t)}{\pi_{ref}(a_t|s_t)}+\beta\log Z(s_t), \tag{158}$$

其中，$Z(s_t)$为配分函数，与策略无关。

3) TGDPO 损失函数：结合 Bradley-Terry 偏好模型，构建可计算的词元级损失函数，具体形式如公式（159）所示：

$$\mathrm{L}_{TGDPO}(\pi_\theta)=-E_{(x,y_w,y_l)\sim D}\left[\log\sigma\left(\sum_{t=0}^{T_w-1}\beta f_w(\hat{r}_w^t)\log\frac{\pi_\theta(y_w^t|s_w^t)}{\pi_{ref}(y_w^t|s_w^t)}-\sum_{t=0}^{T_l-1}\beta f_l(\hat{r}_l^t)\log\frac{\pi_\theta(y_l^t|s_l^t)}{\pi_{ref}(y_l^t|s_l^t)}\right)\right], \tag{159}$$

其中，$f_w=1+\alpha\hat{r}_w^t$，$f_l=1-\alpha\hat{r}_l^t$，α为控制奖励影响强度的超参数。

TGDPO 通过引入词元级奖励引导机制，显著提升了 DPO 的训练效率和模型性能。特别是在损失收敛后，仍能维持高质量策略，有效避免了传统方法中常见的策略退化问题。然而，该方法对词元奖励的质量仍存在一定依赖性，且引入了额外的超参数，可能会增加调优的成本。

算法 62. FocalPO

输入：偏好数据集$D=\{(x,y_w,y_l)\}$，参考模型π_{ref}，待优化策略模型π_θ，超参数$\gamma\geq0$，温度参数β。

输出：更新策略π_θ

FocalPO 在 DPO 损失基础上引入一个动态调节因子p^γ，以增强对高置信度正确样本的关注。具体的损失函数公式如（160）所示：

$$\mathrm{L}_{FocalPO}=-E_{(x,y_w,y_l)\sim D}\left[p(y_w\succ y_l|x)^\gamma\cdot\log p\,(y_w\succ y_l|x)\right], \tag{160}$$

其中，x代表输入提示；y_w和y_l分别表示优选回答和非优选回答；$p(y_w\succ y_l|x)$表示模型判断y_w优于y_l的概率，该概率由隐式奖励模型计算得出；γ为聚焦超参数，用于调控对高置信度样本的强调程度；D则为偏好数据集。

FocalPO 的优势在于其能够明确且动态地调整样本权重，从而有效提升了对易学习样本的利用效率，并在多个基准测试中显著超越了 DPO 及其变体。然而，其性能依然受到超参数γ的影响，且尚未在不同模型与任务组合中进行广泛测试，可能存在一定的泛化局限性。

算法 63. Exploratory Preference Optimization（XPO）

输入：迭代次数T，KL 正则化系数β>0，探索系数α>0，初始参考策略π_{ref}，策略类别Π。

输出：更新后的策略π_θ

XPO 的核心是对标准在线 DPO 目标函数的修改，具体优化目标如公式（161）所示：

$$\pi^{(t+1)}=\arg\min_{\pi\in\Pi}\left\{\alpha\sum_{i=1}^{t}\log\pi(\tilde{\tau}^{(i)})-\sum_{(\tau_+,\tau_-)\in D_{pref}^{(t)}}\log\left[\sigma\left(\beta\log\frac{\pi(\tau_+)}{\pi_{ref}(\tau_+)}-\beta\log\frac{\pi(\tau_-)}{\pi_{ref}(\tau_-)}\right)\right]\right\}, \tag{161}$$

其中，$\pi^{(t+1)}$代表第t+1轮更新后的策略（语言模型）；α为探索系数，用于控制探索奖励项的权重；$\tilde{\tau}^{(i)}$是通过参考策略π_{ref}采样的轨迹（响应）；$D_{pref}^{(t)}$是截至第t轮所积累的偏好数据集，包含正例τ_+和负例τ_-；σ为 Sigmoid 函数，负责将差异转换为概率；β为 KL 正则化系数，用于调控策略与参考策略之间的偏离程度。

算法 64. LongMab-PO

输入：长上下文C，问题q，真实答案y^*，LLM 模型M，片段数n，每轮选中片段数K，总轮数T。

输出：偏好数据集$D=\{(C,q,y^+,y^-)\}$，其中y^+和y^-分别为奖励最高和最低的响应

1) 片段划分：将长上下文C划分为n个等长片段，公式如（162）所示：

$$C_{chunk}=\{C_1,C_2,\ldots,C_n\}, \tag{162}$$

2) UCB 片段选择：在第t轮 rollout 中，依据上置信界（UCB）得分，选择前K个片段，具体如公式（163）所示：

$$UCB_t(C_i)=\mu_i(t)+\alpha\cdot\sqrt{\frac{2\ln t}{n_i(t)+\epsilon}}, \tag{163}$$

其中，$\mu_i(t)$代表片段C_i的期望奖励，$n_i(t)$表示片段C_i被选中的次数，α为探索权重超参数，而ϵ是一个小常数，用于防止除零错误。

3) 响应生成与奖励计算：利用选定的片段$\tilde{C}(\pi_t)$生成响应$\tilde{y}_t$，并计算相应奖励，具体如公式（164）所示：

$$r(\tilde{y}_t)=\frac{SubEM(\tilde{y}_t,y^*)+F1(Ans(\tilde{y}_t),y^*)}{2}, \tag{164}$$

其中，y^*为真实答案，$Ans(\cdot)$用于提取响应中的答案字符串。

4) UCB 统计量更新：根据当前响应的奖励，更新选中片段的期望奖励及选择次数，具体如公式（165）所示：

$$\mu_i(t+1)=\frac{1}{t}\left(\mu_i(t)\cdot(t-1)+r(\tilde{y}_t)\right),\quad n_i(t+1)=n_i(t)+1, \tag{165}$$

LongMab-PO 通过 MAB 机制有效提升了长上下文理解中的关键信息召回率和响应多样性，显著优于静态采样方法。然而，其计算开销较大，且对初始片段奖励估计和超参数较为敏感，可能在实际部署中增加调优负担。

算法 65. Short-to-Long Preference Optimization（LongPO）

输入：短上下文模型π_S，长文本语料C_L，目标上下文长度L。

输出：更新策略π_θ

1) 构建短长偏好数据：针对同一指令I_L，分别输入长上下文$x_L=[C_L;I_L]$和其压缩后的短上下文$x_S=[C_S;I_L]$，通过短上下文模型π_S生成回答y_L（被拒绝）和y_S（被选择），从而构成四元组(x_S, x_L, y_S, y_L)。
2) 短长偏好学习目标：采用 DPO 风格的目标函数，激励模型在长上下文情境中更偏好生成短上下文下的高质量回应，具体表述如公式（166）所示：

$$\mathrm{L}_{LongPO}=-E\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_S|x_L)}{\pi_S(y_S|x_S)}-\beta\,log\frac{\pi_\theta(y_L|x_L)}{\pi_S(y_L|x_S)}\right)\right], \tag{166}$$

其中，π_θ代表待优化的策略模型；π_S则是原始短上下文参考模型；β是一个用于控制偏好强度的超参数。

3) 短长 KL 约束：该约束要求模型在长上下文x_L下的输出分布与在短上下文x_S下的分布保持一致，具体如公式（167）所示：

$$C'=\beta\cdot D_{KL}[\pi_\theta(y|x_L)\|\pi_S(y|x_S)], \tag{167}$$

该约束被隐式地融入上述目标函数中，确保模型在扩展上下文时不丢失短上下文能力。

LongPO 的优势在于无需依赖外部标注即可实现长上下文能力的自我演进，同时能完全保留短上下文性能。在多个长文本基准测试中，其表现甚至超越了更大规模的模型。然而，该方法依赖于模型自身生成偏好数据，若初始短上下文模型的质量不佳，可能会限制其最终性能的上限。

算法 66. Lightweight Post-optimization（LiPO）

输入：原始动作块序列$q_{new}(t)$，上一块优化轨迹$q_{post}(t)$，延迟时间t_d，混合时间t_b，扰动边界$\bar{\epsilon}_b,\bar{\epsilon}_p$。

输出：平滑后的关节轨迹$q_{opt}(t)=q_{ref}(t)+\epsilon(t)$

1) 参考轨迹构建与线性混合：在推理延迟窗口内冻结轨迹，随后在混合过程中对前后两个动作块进行线性插值，具体操作如公式（168）所示：

$$q_{ref}(t)=\begin{cases} q_{post}(t), & t\in[0,t_d] \\ (1-\alpha(t))q_{post}(t)+\alpha(t)q_{new}(t), & t\in(t_d,t_b] \\ q_{new}(t), & t\in(t_b,T] \end{cases}, \tag{168}$$

其中，$q_{post}(t)$为上一块优化后的轨迹；$q_{new}(t)$为新预测的动作块；t_d为推理延迟时间；t_b为混合区结束时间；$\alpha(t)=\frac{t-t_d}{t_b-t_d}$为线性混合权重。

2) 加加速度最小化优化：在参考轨迹的基础上，通过优化扰动$\epsilon(t)$，实现加加速度的最小化。具体过程详见于公式（169），相应的约束条件则如公式（170）所示：

$$\min_{\epsilon}\int_0^T\left\|\frac{d^3}{dt^3}(q_{ref}(t)+\epsilon(t))\right\|^2 dt, \tag{169}$$

$$\text{约束}=\begin{cases}\epsilon(t)=0, & t\in[0,t_d]\\ \|\epsilon(t)\|_\infty\le\bar{\epsilon}_b, & t\in(t_d,t_b]\\ \|\epsilon(t)\|_\infty\le\bar{\epsilon}_p, & t\in(t_b,T]\end{cases}, \tag{170}$$

其中，$\bar{\epsilon}_b>\bar{\epsilon}_p$允许在混合区域内拥有更大的调整空间。

3) 高频率控制插值：使用五次样条插值将优化后的低频率轨迹转换为高频率控制指令，确保位置、速度、加速度的连续性。

LiPO 的优势在于其轻量化设计、易于集成、显著提升运动平滑性，且不改变原策略结构；然而，其不足之处在于未充分考虑机器人动力学约束，这可能导致在高动态任务中限制其性能，同时对于任务空间误差的控制相对保守。

算法 67. Smoothed Preference Optimization（SmPO-Diffusion）

输入：短上下文x_{short}，长上下文x_{long}，偏好对(y_w,y_l)，策略模型π_θ，参考模型π_{ref}。

输出：更新后的策略π_θ

1) 短上下文偏好优化损失：采用标准的偏好优化目标，基于短上下文x_{short}及偏好对(y_w,y_l)进行优化，具体公式如（171）所示：

$$\mathrm{L}_{short}=E_{x\sim D_{x_{short}},y_w,y_l\sim D_y}\left[f\left(\eta\cdot\left[r_\phi(x,y_w)-r_\phi(x,y_l)-\gamma\right]\right)\right], \tag{171}$$

其中，$r_\phi(x,y)$代表奖励函数，通常由策略模型π_θ隐式定义；$f(\cdot)$属于凸函数；η和γ则是算法的超参数。

2) 短到长奖励对齐损失：旨在激励模型在短上下文与长上下文情境下，对同一回答y赋予一致的奖励分数，具体如公式（172）所示：

$$\mathrm{L}_{SoLo\text{-}RA}=\alpha\cdot E_{x\sim D_x,y\sim D_y}s\left(\left|r_\phi(x_{short},y)-r_\phi(x_{long},y)\right|\right), \tag{172}$$

其中，$s(\cdot)$是$f(\cdot)$对应的上界函数；α是平衡两项损失的系数；实际中通常仅对优选回答y_w进行对齐，以提升稳定性和效率。

SoLoPO 在理论和实验层面均展现出卓越性能，能够显著增强模型在长上下文理解和推理方面的能力，同时大幅降低训练成本。然而，该方法仍需依赖人工设定压缩率及对齐系数，且在应对生成式长文本任务时的有效性仍需进一步验证。

算法 68. Self-Exploring Preference-Incentive Online Preference Optimization（SE-POPO）

输入：初始策略π_{ref}，采样器初始策略π_{sam}^1，奖励范围R_{max}，区间数K，每区间迭代数T，超参数α,β。

输出：更新后的策略π_θ

1) 偏好驱动的探索目标：算法通过优化以下目标函数来实现，该函数中特别引入了偏好激励项，具体形式如公式（173）所示：

$$\max_{\pi\in\Pi}\sum_{s=1}^{t}\log\sigma\left(\beta\log\frac{\pi(y_s^w|x_s)}{\pi_{ref}(y_s^w|x_s)}-\beta\log\frac{\pi(y_s^l|x_s)}{\pi_{ref}(y_s^l|x_s)}\right)+\alpha E_{y'\sim\pi_{sam}(\cdot|x)}\left[\sigma\left(-\beta\log\frac{\pi(y'|x)}{\pi_{ref}(y'|x)}\right)\right], \tag{173}$$

其中，π代表待优化的策略模型；π_{ref}为参考策略；π_{sam}作为采样器策略，用于生成对比响应；β是温度参数，用于调控 KL 正则化的强度；α为探索系数，负责调节探索的强度；σ为 Sigmoid 函数，用于构建偏好概率模型；y_s^w和y_s^l分别表示偏好数据中的胜者和败者响应。

2) 自更新采样器机制：SE-POPO 将训练过程划分为多个阶段，每个阶段均采用当前策略作为采样器，并在阶段结束时将采样器更新为当前策略的输出结果。该机制能够逐步提升采样器的性能，确保其生成的响应与当前策略保持一致，有效避免因响应差异过大所导致的信息稀疏问题。

SE-POPO 首次成功实现了对奖励范围的多项式依赖，显著提升了样本效率。其实用变体通过简化目标函数，进一步减少了计算开销。然而，该算法在具体实现中仍存在对生成长度的潜在偏好，并且依赖于在线采样，这可能导致在实际部署中受到推理效率的限制。

算法 69. Mixture of Experts for Dual Preference Optimization（MidPO）

输入：基础模型π_{ref}；安全性和帮助性偏好数据集D_s,D_h；预训练奖励模型r_s,r_h；路由器超参数。

输出：更新策略π_θ

1) 单偏好增强专家训练：采用 SPE-DPO 损失分别对安全性专家E_θ^s和帮助性专家E_θ^h进行训练，具体如公式（174）所示：

$$\mathrm{L}_{SPE\text{-}DPO}=-E\left[\log\sigma\left(\beta\log\frac{E_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\log\frac{E_\theta(y_l|x)}{\pi_{ref}(y_l|x)}-\big(r(y_w,x)-r(y_l,x)\big)\right)\right], \tag{174}$$

其中，$r(y,x)$是奖励模型的输出，β是 KL 控制参数，σ是 sigmoid 函数，y_w和y_l分别代表偏好响应和非偏好响应。

2) 动态路由机制：通过两个线性路由器R_s和R_h计算专家权重，具体如公式（175）所示：

$$\alpha=\sigma(R_s(x)),\quad \beta=\sigma(R_h(x)), \tag{175}$$

其中，$R(x)=e(UV+b_1)Z+b_2$，其中e是隐藏状态，U、V、Z是低秩矩阵，b_1、b_2是偏置。

3) MoE 模型输出：将两个专家的 LoRA 权重与基础模型相结合，具体如公式（176）所示：

$$o_m=W_0e+\alpha\cdot B_sA_se+\beta\cdot B_hA_he, \tag{176}$$

其中，B_sA_s和B_hA_h分别代表安全性和帮助性专家的 LoRA 权重。

4) 双偏好优化损失：在训练路由器时，使用带有 L1 正则的 DPO 损失，损失函数具体如公式（177）所示：

$$\mathrm{L}_{dual}=-E\left[\log\sigma\left(\beta\log\frac{\pi_\theta^m(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\log\frac{\pi_\theta^m(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right]+\lambda\big(\|\alpha\|_1+\|1-\beta\|_1\big). \tag{177}$$

MidPO 的优势在于其模块化设计及动态路由机制，这不仅有效平衡了安全性与辅助性，还在多个基准测试中超越了现有方法。然而，其不足之处在于专家训练依赖于预定义的奖励模型，同时路由器结构也增加了模型的复杂度和推理所需的时间。

算法 70. Adaptive Multi-objective Preference Optimization（AMoPO）

输入：查询x；偏好数据集$D=\{(x,y_w,y_l,d)\}$，其中$d=[d_1,...,d_K]$为各维度分数；策略模型π_θ。

输出：更新策略π_θ

1) 多目标优化目标：目标函数为最大化加权后的多维度偏好概率，具体如（178）所示：

$$max\sum_{k=1}^{K}\rho(\alpha_k|x_k^*)\,log\,\pi_k^*(y_w\succ y_l|x_k^*), \tag{178}$$

其中，K代表偏好维度的数量；α_k是第k维的权重，由策略ρ生成；$x_k^*=f(x,d_k)$是根据维度d_k重构的提示；π_k^*为第k维的偏好对齐函数。

2) AMoPO 损失函数：AMoPO 损失函数如（179）所示：

$$\mathrm{L}_{AMoPO}=-E_{(x,y_w,y_l)\sim D}\left[\sum_{k=1}^{K}\alpha_k\,log\,\sigma\left(\frac{\beta}{|y_w|}log\,\pi_\theta(y_w|x_k^*)-\frac{\beta}{|y_l|}log\,\pi_\theta(y_l|x_k^*)\right)\right], \tag{179}$$

其中，β代表缩放系数；$|y_w|$和$|y_l|$分别表示较好答案和较差答案的回答长度；σ指的是 Sigmoid 函数；π_θ则是指策略模型。

3) 自适应权重分配机制：针对每个维度k，计算生成概率的均值μ_k和方差σ_k^2，构建高斯分布并进行权重采样，具体如公式（180）所示：

$$\mu_k=\frac{1}{T}\sum_{t=1}^{T}p_{k,t},\quad \sigma_k^2=\frac{1}{T}\sum_{t=1}^{T}(p_{k,t}-\mu_k)^2, \tag{180}$$

其中，$p_{k,t}=\pi_\theta(y_t|x_k^*,y_{<t})$表示第$t$个 token 的生成概率。

AMoPO 的优势在于无需额外训练奖励模型或参考模型，从而显著降低了计算成本。此外，通过动态权重机制，它实现了多目标的自适应平衡，在多个基准测试中的表现超越了现有方法。然而，其局限性在于对模型规模的敏感性，小模型上的提升效果有限。

算法 71. Multi-Objective Preference Optimization（MOPO）

输入：离线偏好数据集$D=\{(x_i,y_i,y_i',I(y_i,y_i'))\}_{i=1}^N$；参考策略$\pi_{ref}$；约束阈值初始化参数$\beta$；正则化系数$\tau$、下界约束参数$\epsilon$、更新间隔$t_0$；批量大小$M$、学习率$\eta$、训练轮数$T$。

输出：更新策略π_θ

1) 问题建模：MOPO 将多目标对齐问题建模为一个带有 KL 正则化约束的优化问题，具体形式如公式（181）所示：

$$\max_{\pi}E[p(y\succ y'|x)]-\tau KL(\pi\|\pi_{ref})\quad s.t.\quad E[q(y\succ y'|x)]\geq b, \tag{181}$$

其中，$p(y\succ y'|x)$表示主目标的偏好概率；$q(y\succ y'|x)$代表次目标的偏好向量；τ为 KL 正则化系数；π_{ref}作为参考策略；b则是次目标的约束阈值。

2) 重要性采样比优化：通过引入重要性采样比$\rho(y)=\frac{\pi(y)}{\pi_{ref}(y)}$，将原始问题转化为对$\rho$的优化，并推导出其对偶形式，具体如公式（182）所示：

$$\rho_{\lambda}^{*}(y)=exp\left(\tau^{-1}E[p+\lambda^{T}q]-1\right), \tag{182}$$

其中λ是拉格朗日乘子，用于平衡约束违反程度。

3) 下界约束估计：为避免偏好估计误差导致的约束违反，MOPO 通过优化一个对抗性分布π_k来估计偏好的下界，具体如公式（183）所示：

$$\min_{\pi_k} E[\rho(y)q_k] \quad s.t. \quad KL(\pi_k \| \pi_{ref}) \leq \epsilon. \tag{183}$$

4) 策略提取：利用行为克隆技术，从最优重要性采样比中提取策略，具体如公式（184）所示：

$$\max_{\pi} E_{y\sim\pi_{ref}}\left[\rho^{*}(y)\log\pi(y)\right]. \tag{184}$$

MOPO 能够直接从偏好数据中学习多目标策略，无需依赖奖励模型或进行标量化假设，且在实际应用中展现出优异的帕累托前沿逼近能力及超参数鲁棒性。然而，由于多目标优化问题的固有复杂性，训练过程中可能会出现不稳定或收敛速度较慢的情况。

算法 72. Contrastive Preference Optimization（CPO）

输入：源输入x，偏好输出y_w，非偏好输出y_l，策略模型π_θ，均匀先验策略U，温度参数β。

输出：更新策略π_θ

1) 数据构建：构建三元组数据集 $D=\{(x^{(i)},y_w^{(i)},y_l^{(i)})\}_{i=1}^{N}$，其中$x$为输入，$y_w$为偏好输出，$y_l$为非偏好输出。

2) 偏好优化损失：偏好优化损失具体如公式（185）所示：

$$L_{prefer}(\pi_\theta;U)=-E_{(x,y_w,y_l)\sim D}\left[\log\sigma\left(\beta\log\pi_\theta(y_w|x)-\beta\log\pi_\theta(y_l|x)\right)\right], \tag{185}$$

其中，U为均匀先验策略；β为温度系数，控制偏好强度；σ为 sigmoid 函数。

3) 行为克隆正则项：行为克隆正则项如公式（186）所示，用于约束模型输出不偏离教师模型的分布：

$$\mathrm{L}_{NLL}=-E_{(x,y_w)\sim D}\left[\log\pi_\theta(y_w|x)\right]. \tag{186}$$

4) 总目标函数：总目标函数如公式（187）所示：

$$J_{NLL}=\min_{\theta} L_{prefer}(\pi_\theta,U)+\mathrm{L}_{NLL}. \tag{187}$$

CPO 在减少计算开销和提升生成质量方面表现出潜力，尤其适用于资源受限的场景。然而，其效果高度依赖于偏好评分的准确性，若评分机制存在偏差，可能导致模型学习到错误的偏好。此外，CPO 对教师模型的依赖也限制了其在无监督或弱监督设置下的泛化能力。

算法 73. Contrastive Preference Optimization（CPO）

输入：策略模型π_θ，参考模型π_{ref}，偏好回答y^w与非偏好回答y^l，创造力维度得分$\delta^w,\nu^w,\xi^w,\gamma^w$，注入权重$\lambda_d$、$\lambda_n$、$\lambda_s$、$\lambda_q$。

输出：更新策略π_θ

CrPO 的目标函数如公式（188）所示：

$$\mathrm{L}_{CrPO}=-E_{(x,y^w,y^l)}\left[(\lambda_d\delta^w+\lambda_n v^w+\lambda_s\xi^w+\lambda_q\gamma^w)\cdot\mathrm{L}_{DPO}\right], \tag{188}$$

其中，L_{DPO}为基础 DPO 的损失函数；δ^w代表多样性得分，该得分基于回答与其他回答之间的语义距离；v^w表示新颖性得分，该得分基于回答与参考语料在词汇语义分布上的差异；ξ^w为惊喜性得分，该得分基于回答的困惑度；γ^w则是质量得分，由奖励模型提供；λ_d、λ_n、λ_s、λ_q分别为各维度的注入权重。

CrPO 的优势在于其模块化设计，能够灵活地平衡多个创造力维度，显著提升生成内容的新颖性和多样性，并在多个评估基准上超越现有模型。然而，其不足之处在于对计算资源的需求较高，且可能引入安全风险。

算法 74. FairPO

输入：数据集$D=\{(x_i,y_i)\}$，预训练参考模型参数$\hat{w}$，特权标签集P，非特权标签集$\overline{P}$，超参数$\beta,\epsilon,\eta_w,\eta_\alpha$。

输出：优化后的模型参数$\{w_t\}_{t\in T}$

1) 特权标签的偏好损失：
   - 若$y_{il}=+1$，存在混淆负样本$k\in S_{il}^{neg}$，采用 DPO 风格损失如公式（189）所示：

$$h_w(x_i,l,k)=log\frac{m(x_i;w_l)}{m(x_i;\hat{w}_l)}-log\frac{m(x_i;w_k)}{m(x_i;\hat{w}_k)}, \tag{189}$$

     损失为：$\mathrm{L}_P=-log\,\sigma(\beta\cdot h_w(x_i,l,k))$
   - 若$y_{il}=0$，存在混淆正样本$k\in S_{il}^{pos}$，其损失相似但偏好方向相反。
2) 非特权标签的约束损失:使用铰链损失限制其性能不低于参考模型,具体如公式（190）所示：

$$\mathrm{L}_{\overline{P}}=E_{(x_i,j)}\left[max\,(0,\mathrm{l}(w_j)-\mathrm{l}(\hat{w}_j)-\epsilon)\right], \tag{190}$$

其中l为 BCE 损失。

3) GRPO 平衡机制：总体目标如公式（191）所示：

$$\min_{w}\max_{\alpha\in\Delta_1}[\,\alpha_P\mathrm{L}_P+\alpha_{\overline{P}}\mathrm{L}_{\overline{P}}], \tag{191}$$

其中α为自适应权重，通过镜像上升法更新。

FairPO 的创新之处在于将偏好学习与群体鲁棒优化相结合，显著提升了模型对稀有或敏感标签的判别能力，并在多个基准测试中验证了其有效性。然而，该方法依赖于动态构建的混淆集合以及参考模型的质量，这可能导致训练过程不稳定或调参复杂度增加。

算法 75. Expectation Confirmation Preference Optimization（ECPO）

输入：对话状态s_t，系统响应p_t，用户期望目标i^E，对话历史h_t，满意度阈值λ。

输出：更新策略π_θ

1) 前向期望确认：在每一轮对话中，用户模拟器根据系统响应p_t和期望目标i^E，生成满意度评分r_t和解释$CONF_t$，具体如公式（192）所示：

$$\{CONF_t,r_t\}=U(I_{ect}(i^E,h_t,p_t)), \tag{192}$$

其中，h_t是对话历史，I_{ect}是期望确认指令。

2) 后向期望推导：对于满意度低于阈值λ的响应，使用*Rewriter*模型基于$CONF_t$生成改进响应$\tilde{p}_t$，如公式（193）所示：

$$\tilde{p}_t= Rewriter(I_{bed}(s_t,p_t,CONF_t)),\quad r_t\leq\lambda, \tag{193}$$

3) 偏好优化：使用生成的偏好对$(p_t,\tilde{p}_t)$构建数据集D_{pre}，并采用 DPO 优化方法进行训练，具体如公式（194）所示：

$$\mathrm{L}_{DPO}=E_{(s_t,p_t,\tilde{p}_t)}\left[-\log\sigma\left(\beta\log\frac{\pi_\theta(\tilde{p}_t|s_t)}{\pi_{ref}(\tilde{p}_t|s_t)}-\beta\log\frac{\pi_\theta(p_t|s_t)}{\pi_{ref}(p_t|s_t)}\right)\right], \tag{194}$$

其中，β是控制偏离程度的超参数。

ECPO 通过模拟用户期望确认过程，实现了低成本、高质量的轮次级别偏好优化，显著提升了对话推荐系统的交互质量和用户满意度。然而，其性能仍然依赖于用户模拟器的真实性，存在与现实用户分布不一致的风险，可能导致在实际部署中引入偏差。

算法 76. S-DPO

输入：用户历史交互序列H_u，候选物品集合$C=\{i_1,i_2,...,i_N\}$，正样本物品i_p，多个负样本物品集合$I_d\subset\frac{C}{\{ip\}}$，参考模型$\pi_{ref}$，超参数$\beta$。

输出：更新策略π_θ

1) 多负样本偏好分布建模：使用 Plackett-Luce 模型对用户偏好进行建模，假设存在一个隐式奖励函数$r(x_u,e)$，用以表示在提示x_u下响应e的奖励值。多负样本偏好分布定义如（195）所示：

$$p^*(e_p\succ e_d,\forall e_d\in\varepsilon_d|x_u)=\frac{exp\,(r(x_u,e_p))}{\sum_{j=1}^{K}exp\,(r(x_u,e_j))}, \tag{195}$$

其中，x_u代表由用户历史交互序列构成的提示，e_p表示偏好物品的标题，e_d则是多个负样本标题的集合，K指的是候选物品的总数（涵盖正样本及所有负样本）。

2) 奖励函数参数化：根据 DPO 的理论，奖励函数$r(x_u,e)$可参数化为（196）所示：

$$r(x_u,e)=\beta\log\frac{\pi_\theta(e|x_u)}{\pi_{ref}(e|x_u)}+\beta\log Z(x_u), \tag{196}$$

其中，$\pi_\theta(e|x_u)$表示当前策略模型在提示x_u下生成响应e的概率，$\pi_{ref}(e|x_u)$则代表参考模型的生成概率，β是一个超参数，用于控制策略相对于参考模型的偏离程度，$Z(x_u)$为配分函数，确保概率的归一化。

3) S-DPO 损失函数：将参数化后的奖励函数融入偏好分布，从而得出 S-DPO 损失如（197）所示：

$$L_{S.DPO}=-E_{(x_u,e_u,\varepsilon_d)}\left[\log\sigma\left(-\log\sum_{e_d\square\varepsilon_d}exp\left(\beta\log\frac{\pi_\theta(e_d|x_u)}{\pi_{ref}(e_d|x_u)}-\beta\log\frac{\pi_\theta(e_p|x_u)}{\pi_{ref}(e_p|x_u)}\right)\right)\right], \tag{197}$$

其中σ为 sigmoid 函数。

S-DPO 通过引入多负样本和软最大排序损失，显著提升了语言模型在推荐任务中的排序性能和收敛效率，尤其在挖掘困难负样本方面表现尤为突出。然而，该方法也面临一个显著问题：随着负样本数量的增加，训练成本逐步上升，且在大规模候选集场景下的扩展性仍需进一步探讨。

算法 77. DSpO（Direct Sorted Portfolio Optimization）

输入：原始多频股票数据$X=\{x_1,x_2,\ldots,x_N\}$，其中$x_i=\left(x_i^{hf}\in R^{T\times a},x_i^{lf}\in R^b\right)$，分别表示高频和低频特征。

输出：排序分数向量$S\in R^N$

1) 多频数据融合模块：将高频数据通过 CNN 和 Transformer 编码为嵌入向量，低频数据通过线性投影融合，使用交叉注意力机制生成统一表示，具体如公式（198）所示：

$$o_i=Softmax\left(\frac{\left(e_i^{lf}W_q^{stock}\right)\left(e_i^{hf}W_k^{stock}\right)^{\mathrm{T}}}{\sqrt{D}}\right)\left(e_i^{hf}W_v^{stock}\right), \tag{198}$$

其中，$e_i^{hf}\in R^{T\times D}$为高频嵌入，$e_i^{lf}\in R^{1\times D}$为低频嵌入，$W_q,W_k,W_v$为注意力权重矩阵，$D$为嵌入维度。

2) 跨股票 Transformer 模块：建模所有股票间的依赖关系如公式（199）所示：

$$R=Softmax\left(\frac{\left(OW_q^{intra}\right)\left(OW_k^{intra}\right)^{\mathrm{T}}}{\sqrt{D}}\right)\left(OW_v^{intra}\right), \tag{199}$$

其中$O=[o_1,\ldots,o_N]\in R^{N\times D}$是所有股票的融合表示，输出$R$用于后续排序。

3) 单调逻辑回归损失（MonLR）：最大化预测排序与真实排序一致的概率，具体如公式（200）所示：

$$L=\sum_{i=1}^{k}\sum_{j=1}^{k}\log\left(1+exp\left(-tanh\left(f(x_i)-f(x_j)\right)\cdot tanh\left(y_i-y_j\right)\right)\right), \tag{200}$$

其中，$f(x_i)$为模型预测分数，y_i为真实收益，$tanh$用于平滑符号函数。

4) 跨截面子采样策略：从全市场随机选取k只股票构成训练样本，提升样本多样性与训练效率。

DSpO 的优势在于其端到端的设计，能够有效整合多频数据，显著提升排序准确性和投资组合收益。此外，通过子采样和 MonLR 损失函数的应用，进一步增强了模型的鲁棒性。然而，DSpO 对投资规模较为敏感，在大资金场景下，由于未充分考虑流动性等因素，其性能会受到影响。

算法 78. Multi-Agent Reasoning System Preference Optimization（Mars-PO）

输入：多智能体系统$\{M_1,M_2,...,M_K\}$，训练数据集$D=\{(x_i,y_i)\}$，奖励模型R，超参数α,β,N,T（温度采样参数）。

输出：优化后的各智能体模型$M_1,M_2,...,M_K$

1) 响应样本生成：每个智能体M_k对输入问题x_i生成N个响应$y_i^n=(c_i^n,a_i^n)$，其中c_i^n为推理链，a_i^n为答案。根据答案的正确性，将样本划分为正样本集G_k^w和负样本集G_k^l。
2) 偏好对构建：利用奖励模型对所有智能体的正样本进行评分，选取得分最高的样本构成混合正样本集G^w，并将其与各智能体的负样本集G_k^l进行配对。
3) 混合偏好优化：针对每个智能体，采用以下混合损失函数进行优化，总损失函数如公式（201）所示，DPO 损失函数如公式（202）所示，NLL 损失函数如公式（203）所示：

$$\mathrm{L}=\mathrm{L}_{DPO}+\alpha\mathrm{L}_{NLL}, \tag{201}$$

$$\mathrm{L}_{DPO}=-\log\sigma\left(\beta\log\frac{M_\theta(c_i^w,a_i^w|x_i)}{M_k(c_i^w,a_i^w|x_i)}-\beta\log\frac{M_\theta(c_i^l,a_i^l|x_i)}{M_k(c_i^l,a_i^l|x_i)}\right), \tag{202}$$

$$\mathrm{L}_{NLL}=\frac{\log M_\theta(c_i^w,a_i^w|x_i)}{|c_i^w+y_i^w|}, \tag{203}$$

其中，M_θ代表待优化的模型参数，M_k为原始模型，c_i^w和a_i^w分别表示正样本的推理链与答案，c_i^l和a_i^l则分别对应负样本的推理链与答案，β是用于控制偏好强度的超参数，α为 NLL 损失的权重系数。

Mars-PO 通过多智能体协作和混合样本策略，显著提升了模型在数学推理任务中的性能，尤其在处理复杂多步推理时表现出色且鲁棒。然而，该方法依赖于高质量奖励模型和多个智能体的输出，导致计算成本较高。

算法 79. Group Distributional Preference Optimization (GDPO)

输入：超参数$\lambda_1,\lambda_2,\lambda_3,\beta$

输出：更新策略π_θ

1) 如公式（204）所示，GDPO 的整体损失函数包含两个部分：信念校准损失和信念条件偏好对齐损失：

$$L_{gdpo}(x,p_B^*,y_B;\theta)=L_{cal}(p_\theta(b|x),p_B^*)+E_{b_c,y_c,y_r}L_{pref}(y_c\succ y_r,b_c,x), \tag{204}$$

其中，p_B^*表示目标信念的分布，y_B表示与每个信念相对应的响应集合，b_c是从信念集合中采样得到的信念，y_c和y_r分别是从响应集合中抽取的“选择”和“拒绝”两类响应。

2) 信念校准损失如公式（205）所示：

$$L_{cal}(p_\theta(b|x),p_B^*)=KL(p_\theta(b_{[0]}|x),p_B^*)-\log p_\theta(b|x), \tag{205}$$

其中，$p_\theta(b_{[0]}|x)$表示模型对首个信念标记的预测概率分布情况；$-\log p_\theta(b|x)$表示负对数似然，能有效促使模型准确预测信念。

3) 信念偏好对齐的损失函数如公式（206）所示：

$$L_{pref}(y_c\succ y_r,b_c,x)=-\log\sigma\left(\beta\log\frac{p_\theta(y_c|x,b_c)}{p_{ref}(y_c|x,b_c)}-\beta\log\frac{p_\theta(y_r|x,b_c)}{p_{ref}(y_r|x,b_c)}\right), \tag{206}$$

其中，β是控制对齐强度的温度超参数。

算法 80. Group Robust Preference Optimization（GRPO）

输入：数据集$\bar{D}=\cup_{g=1}^{K} D_g$,，其中每个$D_g$包含三元组$(x_g,y_w,y_l)$；群体大小为$N_g$；初始策略参数$\theta^{(0)}$，初始群体权重$\alpha^{(0)}$；学习率$\eta_\theta$、$\eta_\alpha$，投影算子$P$。

输出：更新策略π_θ

1) 目标函数：GRPO 的核心在于它的优化目标，具体如公式（207）所示：

$$\min_{\pi} \max_{\alpha\in\Delta_K} \sum_{g=1}^{K} \alpha_g \cdot \mathrm{L}_{DPO}(\pi,D_g), \tag{207}$$

其中，π代表待优化的策略；α_g表示群体 g 的权重，属于概率单纯形Δ_K；L_{DPO}为 DPO 损失函数，用于衡量模型对偏好数据的拟合程度；D_g为第g个群体的偏好数据集。

2) 交替更新机制：

- 更新群体权重α：使用指数加权法，对损失较高的群体赋予更大权重，如公式（208）所示：

$$\alpha_g^{(t)} \propto \alpha_g^{(t\text{-}1)} \cdot exp\left(\eta_\alpha \cdot l(\pi_{\theta^{(t\text{-}1)}};D_g)\right). \tag{208}$$

- 更新策略参数θ：使用加权梯度下降，关注高权重群体的梯度，具体如公式（209）所示：

$$\theta^{(t)}=P\left(\theta^{(t\text{-}1)}\text{-}\eta_\theta \cdot \alpha_g^{(t)} \nabla_\theta l(\pi_{\theta^{(t\text{-}1)}};D_g)\right). \tag{209}$$

GRPO 的优势在于能够显著提升最差群体的性能，缩小群体间的性能差距，增强模型在多样化用户群体中的公平性和鲁棒性。然而，其不足之处在于可能会牺牲整体平均性能，并且在群体数据平衡、偏好差异较小的情况下，其优势并不显著。

算法 81. Multi-Agent Contrastive Preference Optimization（MACPO）

输入：弱教师模型集合$\{W_k\}_{k=1}^{K}$，强学生模型S，初始训练集$D=D_1\cup D_2$，查询集Q_{w2s}，负行为数据集。

输出：优化后的强学生模型S^T，优化后的弱教师模型集合$\{W_k^T\}_{k=1}^{K}$。

1) 互增强正行为构建：弱教师生成候选回答，如公式（210）所示；基于学生模型S计算困惑度，并筛选高质量正样本，如公式（211）所示：

$$G_{all}=\left\{(y_{i,k})_{k=1}^{K} | y_i \sim W_k(x_i), x_i \in Q_{w2s}\right\}, \tag{210}$$

$$G_S^{pos}=\left\{y_{i,k} | \arg\min_k (ppl_{i,k}), y_{i,k} \in G_{all}\right\}, \tag{211}$$

其中，W_k表示第k个弱教师，$y_{i,k}$是其对问题x_i的回答；$ppl_{i,k}$是学生模型对$y_{i,k}$的困惑度。

2) 难负样本构建：通过利用负行为数据进行微调，构建出负教师W_k^{neg}和负学生S^{neg}，从而生成自适应的负样本。具体过程如公式（212）所示：

$$G_S^{neg}=\{y_i|y_i\sim S^{neg}(x_i)\},\quad G_{W_k}^{neg}=\{y_i|y_i\sim W_k^{neg}(x_i)\}. \tag{212}$$

3) 对比偏好对构建与优化：构建对比偏好对，如公式（213）所示；据此，利用 DPO 损失进行优化，得到优化函数，如公式（214）所示：

$$D_*^{cp}=\{(x_i,(y_i^{pos},y_i^{neg}))|y_i^{pos}\in G_*^{pos},y_i^{neg}\in G_*^{neg}\}, \tag{213}$$

$$\mathrm{L}_{po}=\mathrm{L}_{dpo}+\gamma\mathrm{L}_{sft}, \tag{214}$$

其中，*表示学生S或教师W_k；L_{dpo}为原始 DPO 损失，L_{sft}为监督微调损失，γ为权重超参数。

MACPO 通过多智能体协同与对比学习的策略，有效缓解了弱监督环境下的噪声问题及自对齐过程中的模型崩溃现象，显著提升了强模型的对齐性能。然而，该方法依赖于多个教师模型的参与和负样本的构造，导致计算成本相对较高。

算法 82. Iterative Reasoning Preference Optimization（IRPO）

输入：初始模型M_0，训练集$D=\{(x_i, y_i)\}$（包含问题与对应正确答案），每轮生成样本数N，每问题配对数量K，损失权重α和β，迭代轮数T。

输出：训练后的最终模型M_T

1) 生成候选推理链及答案：针对每一个训练问题x_i，利用当前模型M_t生成N个包含推理步骤c_i^n及最终答案y_i^n的响应。

2) 构建偏好对：依据答案的正确性（奖励r_i^n），将响应分为正例集G_i^w和负例集G_i^l，并为每个问题构建K组正负配对(c_i^w,y_i^w)和(c_i^l,y_i^l)。

3) 优化 DPO+NLL 损失：通过如下联合损失函数更新模型参数，具体如公式（215）所示：

$$\mathrm{L}_{DPO+NLL}=-\log\sigma\left(\beta\log\frac{M_\theta(c_i^w,y_i^w|x_i)}{M_t(c_i^w,y_i^w|x_i)}-\beta\log\frac{M_\theta(c_i^l,y_i^l|x_i)}{M_t(c_i^l,y_i^l|x_i)}\right)-\alpha\frac{\log M_\theta(c_i^w,y_i^w|x_i)}{|c_i^w|+|y_i^w|}, \tag{215}$$

其中，M_θ代表待优化的当前模型；M_t则为上一轮迭代的模型（即参考模型）；σ表示 sigmoid 函数；β是 DPO 损失的温度系数；α为 NLL 损失的权重系数；$|c_i^w|+|y_i^w|$则表示正例响应的总长度，用于归一化处理。

IRPO 的优势在于其无需额外数据或人工标注，仅需通过迭代自生成与偏好优化，即可显著增强模型的推理能力，并在多个任务中展现出一致且卓越的表现。然而，该方法依赖于明确的正确答案判断标准，且在迭代后期，性能提升可能会逐渐趋于饱和。

算法 83. EVOLVE

输入：初始预训练模型π_θ，偏好数据集D，精炼模板z，迭代轮数T。

输出：更新策略π_θ

1) SFT 阶段优化：在标准的 SFT 损失基础上，增加对精炼能力的训练，具体如公式（216）所示：

$$\mathrm{L}_{EVOLVE\text{-}SFT}(\pi_\theta)=-E_{(x,y_w,y_l)\sim D}[\log\pi_\theta(y_w|x)+\log\pi_\theta(y_w|x,y_l,z)], \tag{216}$$

其中，x为输入，y_w为优选回答，y_l为劣选回答，z为精炼模板。

2) 偏好训练阶段优化：结合 DPO 损失与自我精炼损失，具体如公式（217）所示：

$$L_{EVOLVE\text{-}PT}(\pi_\theta;\pi_{ref})=\alpha L_{SR}(\pi_\theta;\pi_{ref})+(1-\alpha)L_{DPO}(\pi_\theta;\pi_{ref}), \tag{217}$$

其中，L_{SR}为自我精炼损失，π_{ref}为参考模型，α为平衡系数。

3) 推理阶段：运用自我精炼策略，逐步完善回答，具体如公式（218）所示：

$$y_n \sim \pi_\theta(\cdot|x,y_{n-1},z). \tag{218}$$

EVOLVE 通过迭代式自我精炼显著提升了小模型的性能，甚至在多项任务中超越了大型模型。然而，该方法依赖于高质量的初始数据和精炼模板的设计，并且在数学推理等专业任务上的提升仍存在局限性。

算法 84. Reverse Preference Optimization（RPO）

输入：原始指令x；两个回答y_i和y_j，满足$S_{y_i}\neq S_{y_j}$；约束遵循状态集合S_{y_i}和S_{y_j}。

输出：更新后的策略π_θ

1) 差异度量：与采用总得分差的方法不同，RPO 计算在每个约束条件下遵循情况不同的数量，具体如公式（219）所示，用于量化两个响应y_i和y_j之间的真实差异程度：

$$g=\sum_{k=1}^{|S_x|} I[S_{y_i}(k)\neq S_{y_j}(k)], \tag{219}$$

其中，$|S_x|$表示指令x中包含的约束总数量；$S_y(k)$为一个布尔函数，用于判断响应y是否满足第k个约束（满足时值为 1，否则为 0）；$I[\cdot]$为指示函数，当内部条件成立时取值为 1，否则为 0；g 代表两个响应在约束遵循上的差异个数，即实际的“差距”。

2) 约束反转：针对选定的响应y_i，将其所有未遵循的约束进行反转，从而生成一个新的指令x_{S_i}。这一操作确保y_i成为新指令下的理想响应，具体过程如公式（220）所示：

$$x_{S_i}=ReverseConstraints(x,\{k|S_{y_i}(k)=0\}), \tag{220}$$

其中，$\{k|S_{y_i}(k)=0\}$为响应y_i未能遵循的所有约束的索引集合；$ReverseConstraints()$是一个函数，用于将指定集合中的约束语义进行反转；x_{S_i}为反转后的新指令。对于该指令，y_i完美地遵循了所有约束，即$S_{y_i}=\vec{1}$。

3) 自适应边际：为了在优化过程中凸显不同偏好之间的差异程度，RPO 在损失函数中引入了一个与差异数量g成正比的边际项，具体形式如公式（221）所示：

$$Margin=\gamma g, \tag{221}$$

其中，g表示上述计算所得的差异数量；γ是一个超参数，用于调节边际的缩放强度；$Margin$代表自适应边际。差异越显著的偏好对，在优化过程中要求其对数概率差距也相应增大，从而使模型的优化方向更加明确且稳健。

4) 目标函数：RPO 的优化目标建立在 DPO 的 Bradley-Terry 模型基础之上，但在此基础上引入了反转后的指令以及自适应边际项，其损失函数具体形式如公式（222）所示：

$$L_{RPO}=-E_{(x_{S_i},y_i,y_j,g)\sim D}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(y_i|x_{S_i})}{\pi_{ref}(y_i|x_{S_i})}-\beta\log\frac{\pi_\theta(y_j|x_{S_i})}{\pi_{ref}(y_j|x_{S_i})}-\gamma g\right)\right], \tag{222}$$

其中，y_i和y_j为两个回答，其中y_i在反转后成为完美回答；β和γ为超参数，分别控制策略偏离和差异权重。

算法 85. Self-Play Preference Optimization（SPPO）

输入：初始策略π_0；偏好数据集D（包含三元组(x,y_w,y_l)）；学习率η，正则化系数β；迭代次数T。

输出：更新后的策略π_θ

1) 目标：寻找最优策略π^*，如公式（223）所示，使得其在对阵任何策略 π'时的期望胜率最大：

$$\pi^*=\arg\max_{\pi}\min_{\pi'}E_{y_1\sim\pi,y_2\sim\pi'}[P(y_1\succ y_2|x)], \tag{223}$$

其中，$P(y_1\succ y_2|x)$表示在上下文x下，响应y_1优于y_2的概率。

2) 损失函数：SPPO 的损失函数如公式（224）所示：

$$l_{SPPO}(r_\pi,x,y^w,y^l,\pi_t)=\frac{1}{2}\left(r_\pi(x,y^w)-1+\log Z_{\pi_t}(x)\right)^2+\frac{1}{2}\left(r_\pi(x,y^l)+\log Z_{\pi_t}(x)\right)^2, \tag{224}$$

其中，$r_\pi(x,y)=\beta\log\frac{\pi(y|x)}{\pi_t(y|x)}$表示重参数化奖励；$Z_{\pi_t}(x)=\sum_y\pi_t(y|x)\exp(\eta\cdot P(y\succ\pi_t|x))$代表配分函数；$\eta$为学习率，$\beta$为正则化系数；$y^w$和$y^l$分别代表偏好对中的优胜响应和劣汰响应。

SPPO 的优势在于其能够处理非传递性偏好，且无需显式奖励模型，从而简化了训练流程。然而，其损失函数的设计较为复杂，依赖于配分函数的估计，这可能会在高维响应空间中带来计算上的挑战。

算法 86. Ra-DPO

输入：初始策略模型π_θ，参考模型π_{ref}，偏好数据集$D=\{(x,y_w,y_l)\}$，超参数α,β,μ,η。

输出：更新策略π_θ

1) 目标函数设计：Ra-DPO 在 token 级别上最大化风险感知优势函数，同时约束与参考模型之间的 KL 散度，具体公式如（225）所示：

$$\max_{\pi_\theta}E\left[\widetilde{A}_{\pi_{ref}}([x,y^{<t}],z)-\beta D_{KL}(\pi_\theta\|\pi_{ref})\right], \tag{225}$$

其中，$\widetilde{A}_{\pi_{ref}}$为风险感知优势函数，$z$为当前token动作。

2) 风险感知优势函数：风险感知优势函数的具体计算方法如式（226）所示：

$$\widetilde{A}_\pi([x,y^{<t}],z)=\widetilde{Q}_\pi([x,y^{<t}],z)-\Phi^\mu(\widetilde{V}_\pi([x,y^{<t}])), \tag{226}$$

其中，$\widetilde{Q}_{\pi}$为风险感知动作值函数，$\widetilde{V}_{\pi}$为风险感知状态值函数，Φ^{μ}为嵌套风险度量函数，μ为风险控制参数。

3) 损失函数：Ra-DPO 的损失函数在 DPO 基础上引入序列风险比率差δ，具体如公式（227）所示：

$$L_{Ra\text{-}DPO}=-E[\log\sigma(u-\delta)], \tag{227}$$

其中，$u=\beta\log\frac{\pi_{\theta}(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\log\frac{\pi_{\theta}(y_l|x)}{\pi_{ref}(y_l|x)}$，$\delta=\beta D_{SeqRR}(x,y_l)-\beta D_{SeqRR}(x,y_w)$，其中$D_{SeqRR}$作为序列风险比率，用于衡量每一步风险累积情况。

Ra-DPO 的优势在于其能够在 token 级别动态感知并精准控制风险，有效平衡模型的性能与稳定性，尤其在抑制模型漂移方面表现尤为突出。然而，该方法依赖于风险度量的选择，且计算复杂度较高，可能并不适用于所有类型的风险控制任务。总体而言，Ra-DPO 为风险敏感的语言生成任务提供了坚实的理论基础和实用的工具支持。

算法 87. Ex-Ante Reasoning Preference Optimization（ERPO）

输入：用户查询x，预定义安全规则集合R，参考模型π_{ref}，训练数据集：安全数据集D_{safe}和通用数据集$D_{general}$。

输出：更新策略π_{θ}

ERPO 分为三个阶段，其核心内容与公式如下：

1) 监督微调阶段：目标函数如公式（228）所示：

$$L_{SFT}(\theta)=-E_{(z^+,y^+,y^-,x)}\left[\log p_{\theta}(z^+\oplus y^+|x\oplus prefix(y^-))\right], \tag{228}$$

其中，x代表用户输入；z^+表示正向推理路径；y^+代表安全回复；y^-代表有害回复；$prefix(y^-)$为有害回复的前缀，旨在提升模型对有害路径的识别与纠正能力。

2) 偏好优化阶段：使用 DPO 目标函数，具体如公式（229）所示：

$$L_{DPO}(\pi_{\theta};\pi_{ref})=-E_{(x,s^w,s^l)}\left[\log\sigma\left(\beta\log\frac{\pi_{\theta}(s^w|x)}{\pi_{ref}(s^w|x)}-\beta\log\frac{\pi_{\theta}(s^l|x)}{\pi_{ref}(s^l|x)}\right)\right], \tag{229}$$

其中，s^w代表优选与劣选序列（包含推理与回复过程）；π_{θ}和π_{ref}分别指代当前模型与参考模型；β则是用于控制偏离参考模型程度的惩罚系数。

3) 长度控制迭代优化：通过迭代 DPO 优化，控制推理长度，提升推理效率。定义长度奖励如公式（230）所示：

$$R(z)=\frac{L(z^+_{summ})}{L(z)}, \tag{230}$$

其中，$L(\cdot)$表示 token 长度，优选更简洁的推理路径。

ERPO 通过显式推理机制显著增强了模型在复杂安全场景中的鲁棒性，尤其在科学领域有害内容识别方面表现尤为出色，同时仍保持出色的通用性。然而，该方法不可避免地引入了额外的推理延迟。

算法 88. Reject Preference Optimization（RejPO）

输入：用户前缀P_p；用户长期兴趣表示L；用户短期兴趣表示S；去毒专家模型（用于训练阶段排序）。

输出：生成补全列表$G=\{g_1,g_2,...,g_N\}$，其中所有g_i的生成概率均高于[Reject]标记

在训练过程中，由“去毒专家”模型对生成结果排序，并在低于阈值时插入 [Reject] 标记，优化其排序位置，RejPO 的损失函数如公式（231）所示：

$$L_{RejPO}=-\left(\sum_{(y_+,y_r)} log\,\sigma\left[log\,P(y_+|P_p,S,L)-log\,P(y_r|P_p,S,L)\right]+\sum_{(y_r,y_-)} log\,\sigma\left[log\,P(y_r|P_p,S,L)-log\,P(y_-|P_p,S,L)\right]\right), \quad (231)$$

其中，y_r表示特殊标记[Reject]；y_+代表经“去毒专家”提升后的生成结果；y_-则代表被“去毒专家”抑制的生成结果；P_p、S、L分别代表前缀、短期兴趣和长期兴趣；σ为 sigmoid 函数，用于实现概率归一化。

RejPO 实现了端到端的自适应去毒功能，无需依赖后处理模型，从而显著降低了推理延迟，同时确保生成内容的相关性。然而，其训练过程依赖于高质量的“去毒专家”模型进行排序，若该专家模型存在偏差，可能会影响[Reject]标记的插入准确性。

算法 89. Trust Region Preference Approximation（TRPA）

输入：初始策略模型π_{ref}，提示分布D，预定义的偏好等级规则，超参数β,N,λ。

输出：更新后的策略π_θ

1) 偏好构建：依据预定义的规则，将回答划分为四个偏好等级，并构建偏好对比对(y_1,y_2)，其中y_1优于y_2。
2) KTPO 技术：KTPO 技术的具体形式如公式（232）所示：

$$\beta(y)=\begin{cases} N\cdot\beta, & if\ y\ 属于偏好等级\ 1 \\ \beta, & 否则 \end{cases}, \quad (232)$$

其中，$N>1$为放大因子，用于提升高偏好回答的优化强度。

3) 损失函数：TRPA 的损失函数具体如公式（233）所示：

$$L_{TRPA}(\pi_\theta;\pi_{ref})=-E_{(x,y_1,y_2)\sim D_{old}}\left[log\,\sigma\left(\beta(y_1)\,log\frac{\pi_\theta(y_1|x)}{\pi_{ref}(y_1|x)}-\beta(y_1)\,log\frac{\pi_\theta(y_2|x)}{\pi_{ref}(y_2|x)}\right)\right]+\lambda E_{x\sim D}\left[D_{KL}\left(\pi_{\theta_{old}}(\cdot|x)\|\pi_\theta(\cdot|x)\right)\right], \quad (233)$$

其中，π_θ为待优化的策略模型，π_{ref}为参考策略（即初始模型），D_{old}为由旧策略$\pi_{\theta_{old}}$生成的数据集，$\beta(y_1)$为 KTPO 系数，用于对高偏好等级的回答赋予更大的权重，λ为 KL 正则化系数。

TRPA 无需设计复杂的奖励函数，有效避免了奖励破解问题，并具备理论上的单调改进保证。此外，在实验中，TRPA 展现出卓越的稳定性和推理性能。然而，其偏好等级仍然依赖于人工规则设计，这可能会限制其泛化能力。

算法 90. Length Controlled Preference Optimization（LCPO）

输入：原始大型推理模型π_θ，训练问题集合D，优选回答y_w与被拒绝回答y_l的配对数据，超参数λ。

输出：更新策略π_θ

目标函数形式：LCPO 采用 Bradley-Terry 损失框架，其目标函数定义如公式（234）所示：

$$L_{LCPO}=-\lambda \log \sigma\left(\log\left(\frac{p_\theta(y_w|x)}{1-p_\theta(y_w|x)}\right)-\log\left(\frac{p_\theta(y_l|x)}{1-p_\theta(y_l|x)}\right)\right), \tag{234}$$

其中，$p_\theta(y|x)=exp\left(\frac{1}{|y|}\log \pi_\theta(y|x)\right)$表示模型在输入$x$条件下生成序列$y$的归一化概率；$y_w$代表优选的（较短且准确的）回答；$y_l$则是被拒绝的（较长或冗余的）回答；$\sigma$为 sigmoid 函数；$\lambda$是用于控制偏好对齐强度的超参数。

LCPO 的优势在于仅需极少的训练样本和训练步数，即可实现超过 50%的输出长度压缩，同时保持高推理精度，展现出显著的计算效率优势。然而，其性能高度依赖于训练数据的质量和多样性，且在非数学类任务上的泛化能力仍需进一步验证。

算法 91. Omni-DPO

输入：输入提示x，偏好对(y_w,y_l)，参考模型π_{ref}，质量分数S_w,S_l，超参数$\beta,\eta,\gamma,\tau_{ref},\tau_{good},\lambda$。

输出：更新策略 π_θ

1) 质量权重w_{qual}：根据偏好对中正负样本的专家评分差异进行动态加权，具体如（235）所示：

$$w_{qual}=\sigma(\eta\cdot(S_w-S_l)), \tag{235}$$

其中S_w,S_l分别为正负样本的质量分数，η为缩放因子，σ为 sigmoid 函数。

2) 性能权重w_{perf}：基于模型当前对样本的拟合程度，调整权重，如（236）所示，以避免对已学好的样本进行过度训练：

$$w_{perf}=\left[1-\sigma\left(\frac{\beta}{|y_w|}\log \pi_\theta(y_w|x)-\frac{\beta}{|y_l|}\log \pi_\theta(y_l|x)-\tau_{ref}\right)\right]^\gamma, \tag{236}$$

其中β为 DPO 温度参数，$|y|$为响应长度，τ_{ref}为统一性能阈值，γ控制权重衰减强度。

3) 校准负对数似然损失$L_{c\text{-}NLLL}$：针对高质量却难以拟合的正样本，实施强化学习，具体如（237）所示：

$$\mathrm{L}_{c\text{-}NLL}=-1(\log \pi_{ref}(y_w|x)>\log \pi_\theta(y_w|x))\cdot 1(S_w\geq\tau_{good})\cdot\frac{\log \pi_\theta(y_w|x)}{|y_w|}, \tag{237}$$

其中τ_{good}为高质量样本阈值。

4) 总损失函数：Omni-DPO 的损失函数如（238）所示：

$$\mathrm{L}_{Omni\text{-}DPO}=-E\left[w_{qual}\cdot w_{perf}\cdot \log \sigma(\Delta_r)\right]+\lambda \mathrm{L}_{c\text{-}NLL}, \tag{238}$$

其中Δr为隐式奖励边际，λ为 c-NLL 损失的权重系数。

算法 92. Negative-aware Preference Optimization（NaPO）

输入：用户历史交互序列s_u，正样本物品y_u^+，随机负样本集合$\widehat{\mathrm{E}}_u$，批次训练数据B。

输出：更新策略π_θ

1) 批次内负样本共享：在训练批次中，通过相似性筛选，共享负样本的 log 概率值，以扩展负样本集合，具体如公式（239）所示：

$$\mathrm{E}_u=\widehat{\mathrm{E}}_u\cup S_u, \tag{239}$$

其中，$\hat{E}_u$是随机选取的负样本，S_u则源自相似序列的共享负样本。

2) 动态奖励边际调整：根据辅助序列推荐器输出的置信度，动态调整奖励边际 γ，具体如公式（240）所示：

$$\gamma_{batch}=\left[1+\alpha\left(E_{u,y_u^-}\left[conf(s_u,y_u^-)\right]-R_0\right)\right]\gamma_0, \tag{240}$$

其中，$conf(s_u,y_u^-)=\frac{1}{2}(1\text{-}SR\text{-}SCORE(s_u,y_u^-))$表示负样本的置信度；$R_0$为动态更新的基准置信度；$\alpha$为调整系数，$\gamma_0$为初始边际值。

3) 优化目标：优化目标如公式（241）所示：

$$L_{NAPO}=-E_{(x_u,y_u^+,E_u)}\left[\log\sigma\left(\log p\,(y_u^+\succ_u E_u|x_u)-\gamma\right)\right], \tag{241}$$

其中，$p(y_u^+\succ_u E_u|x_u)=\frac{exp\,(H_\theta(y_u^+|x_u))}{\Sigma_{y_u^-\in E_u}exp\,(H_\theta(y_u^-|x_u))}$，$H_\theta(y|x)=\beta\log\pi_\theta\,(y|x)$是模型输出的对数概率。

NaPO 通过高效的负样本共享和动态边际机制，显著提升了推荐系统的性能及偏差控制能力，且无需引入参考模型，计算效率极高；然而，其超参数数量较多，需精细调优，这可能会限制其易用性和泛化能力。

算法 93. Negative Preference Optimization（NPO）

输入：预训练模型参数θ，遗忘数据集D_f，保留数据集D_r，超参数γ。

输出：更新后的策略参数θ

1) 遗忘损失设计：NPO 将遗忘数据视为“负偏好”，通过优化过程使其在模型输出中的概率降低，进而实现知识的擦除。

2) 目标函数：NPO 的优化目标由两部分构成：遗忘损失和保留损失，具体形式如公式（242）所示：

$$l_u(\theta;D_f,D_r)=l_f(\theta;D_f)+\gamma l_r(\theta;D_r), \tag{242}$$

其中，D_f代表遗忘数据集，D_r代表保留数据集；l_f为遗忘损失，旨在激励模型降低对D_f的输出概率；l_r为保留损失，通常采用交叉熵损失形式，以维持模型在D_r上的性能；γ为权衡超参数，用于调节遗忘与保留之间的平衡。

NPO 提供了一种轻量级且高效的遗忘机制，能够在不进行全模型重训练的情况下实现定向知识删除，并在多个任务中展现出优异的效用保持能力。然而，NPO 对下游微调高度敏感，其遗忘效果易被后续训练所破坏，存在“再学习”的风险。

算法 94. Hybrid Preference Optimization（HPO）

输入：离线数据集D_{off}（大小为N_{off}）；采样策略π_{samp}；超参数$\alpha>0$，$\gamma\in N$；参考策略π_{ref}；在线探索轮数T。

输出：更新策略π_θ

1) 混合数据集构建：在每个在线轮次t中，HPO 从离线数据集D_{off}中采样γ对偏好数据，并将其与在线收集的偏好数据合并，形成混合数据集$D_{hyb}^{(t)}$。

2) 乐观探索机制：通过从采样策略$\tilde{\pi}^{(t)}$中生成$t+\gamma$条轨迹，构建乐观数据集$D_{opt}^{(t)}$，以鼓励策略生成多样化的响应。

3) 策略更新目标函数：HPO 的目标函数如公式（243）所示：

$$\pi^{(t+1)}=\arg\min_{\pi\in\Pi}\left[\alpha\sum_{\tau\in D_{opt}^{(t)}}\log\pi(\tau)-\sum_{(\tau^{+},\tau^{-})\in D_{hyb}^{(t)}}\log\sigma\left(\beta\log\frac{\pi(\tau^{+})}{\pi_{ref}(\tau^{+})}-\beta\log\frac{\pi(\tau^{-})}{\pi_{ref}(\tau^{-})}\right)\right], \tag{243}$$

其中，α代表乐观探索的强度；β是 KL 正则化的系数；π_{ref}为参考策略；σ表示 sigmoid 函数；$\log\frac{\pi(\tau)}{\pi_{ref}(\tau)}$是轨迹$\tau$的对数概率比，用于衡量策略相对于参考策略的偏离程度。

HPO 的优势在于其理论上的样本效率高以及对离线数据覆盖性要求的降低，这使得其在实际应用中展现出更强的灵活性和可扩展性；然而，其性能表现高度依赖于超参数的精准调优，这可能会引入一定的成本和延迟。

算法 95. Implicit Model Fusion via Preference Optimization（InfiFPO）

输入：枢轴模型M^p；多个源模型$\{\mathrm{M}_i^s\}_{i=1}^N$；偏好数据集$D_p=\{(x,y_w,y_l)\}$；超参数$\beta$，融合策略$\gamma_i$。

输出：优化后的枢轴模型M^p

1) 目标函数推导：从 FuseRLHF 目标出发，通过拉格朗日松弛得到如下无约束优化目标如公式（244）所示：

$$\arg\max_{\mathrm{M}^p}E_{x,y\sim D}[r(x,y)]-\beta\sum_{i=1}^{N}\gamma_i D_{SKL}[\mathrm{M}^p(y|x)\|\mathrm{M}_i^s(y|x)], \tag{244}$$

其中，M^p代表待优化的枢轴模型；M_i^s表示第i个源模型；γ_i为源模型的融合权重；D_{SKL}指的是序列级 KL 散度；β则是用于控制对齐强度的超参数。

2) 融合源模型构建：融合源模型定义为各源模型序列概率的加权几何平均，具体如公式（245）所示：

$$\mathrm{M}_{fu}^s(y|x)=\prod_{i=1}^{N}[Clip(\mathrm{M}_i^s(y|x))]^{\gamma_i}, \tag{245}$$

其中，$Clip$作为概率裁剪函数，旨在限制源模型对错误响应的过度影响。

3) InfiFPO 损失函数：最终优化目标如公式（246）所示：

$$\mathrm{L}_{InfiFPO}=-E_{(x,y_w,y_l)}\left[\log\sigma\left(\beta\log\frac{\mathrm{M}^p(y_w|x)}{\mathrm{M}_{fu}^{s,clip}(y_w|x)}-\beta\log\frac{\mathrm{M}^p(y_l|x)}{\mathrm{M}_{fu}^{s,clip}(y_l|x)}\right)\right], \tag{246}$$

其中，σ为 sigmoid 函数，y_w和y_l分别代表偏好响应和非偏好响应。

InfiFPO 在众多基准测试中展现出卓越性能，能够高效整合多源模型的优势，显著增强模型的整体表现，同时有效规避了传统融合方法中的词汇对齐难题。然而，该技术仍依赖于

DPO 等基础优化框架，其理论解释性存在局限，且在大规模模型及数据集上的泛化能力尚未得到充分验证。

算法 96. Score Entropy Policy Optimization（SEPO）

输入：预训练离散扩散模型参数θ_{pre}，奖励函数R_t，训练步数S，每轮训练周期K，KL 权重α，裁剪阈值ϵ。

输出：更新后的策略π_θ

1) 自归一化重要性采样：SEPO 采用自归一化重要性采样方法，有效降低了梯度估计的方差，具体如公式（247）所示：

$$q_t^\theta(y) \approx \frac{1}{M}\sum_{i=1}^{M} q_t^\theta(y|z_i)\cdot\frac{q_t^\theta(z_i)}{q_t^\theta(z_i|y)}, \tag{247}$$

其中，z_i是从$q_t^\theta(\cdot|y)$中采样的邻居状态，用于估计边际概率$q_t^\theta(y)$。

2) 梯度裁剪：通过比例$r_{x,y}^t$将其裁剪至区间$[1-\epsilon,1+\epsilon]$，以稳定训练过程，比例的具体计算如公式（248）所示：

$$r_{x,y}^t = \frac{q_t^\theta(y)}{q_t^{\theta_{old}}(y)}\cdot\frac{s_{\theta_{old}}(x,T\text{-}t)_y}{s_\theta(x,T\text{-}t)_y}. \tag{248}$$

3) 目标函数：SEPO 的目标函数详细表达如公式（249）所示：

$$\mathrm{l}_t(\theta) = -E_{x\sim q_t^\theta}[R_t(x)] + \alpha\cdot KL(q_t^\theta \| q_t^{\theta_{pre}}), \tag{249}$$

其中，$R_t(x)$是奖励函数，q_t^θ是当前策略分布，$q_t^{\theta_{pre}}$是预训练模型分布，α是 KL 散度权重。

SEPO 在不可微奖励处理和任务通用性方面展现出显著优势。实验结果显示，其在语言建模和 DNA 序列生成任务中的表现均超越现有方法。然而，SEPO 的 SNIS 步骤引入了额外的计算复杂度，并且在采样过程中需多次迭代，这可能导致训练速度有所减缓。

算法 97. Direct Judgement Preference Optimization（DJPO）

输入：任务类型与评分标准p，任务输入i，待评估的响应r或响应对$\{r_1,r_2\}$。

输出：最终判断j

1) 三种偏好数据构建：
   - CoT Critique（DCoT）：包含推理链和最终判断，用于提升推理能力；
   - Standard Judgement（DStd）：仅包含最终判断，强化对关键判断信号的学习；
   - Response Deduction（DDed）：根据评价反推原始响应，增强对响应质量的理解。
2) 训练目标：结合 SFT 损失和 DPO 损失，具体如公式（250）所示：

$$\mathrm{L}_{DPO+SFT} = -\frac{\log M_s(y_i^w|x_i)}{|y_i^w|+|x_i|} - \log\sigma\left(\beta\frac{M_s(y_i^w|x_i)}{M_{ref}(y_i^w|x_i)} - \beta\frac{M_s(y_i^l|x_i)}{M_{ref}(y_i^l|x_i)}\right), \tag{250}$$

其中，M_s代表学生模型（即待训练的评判模型）；M_{ref}代表参考模型（其参数固定不变）；y_i^w表示正样本（即判断正确的评价）；y_i^l表示负样本（即判断错误的评价）。

DJPO 通过多任务偏好优化显著提升了评判模型的综合能力，尤其在抗偏差和任务适应性方面表现突出。然而，其训练数据依赖强教师模型生成，成本较高。

算法 98. Dual Preference Optimization（DuPO）

输入：初始任务输入x，双任务T_{cd}。

输出：更新策略π_θ

DuPO 的核心在于其广义二元框架。该框架将初始任务输入x分解为已知分量x_k和未知分量x_u，并同步构建相应的对偶任务T_{cd}。通过原始输出y和已知信息x_k，系统能够重构未知分量x_u，此重构质量被用作自监督奖励信号，以优化原始任务。该奖励信号由公式（251）表示，而目标函数则由公式（252）给出：

$$r(y) \propto exp\left(-\lambda \cdot d(x_u, T_{cd}(y, x_k))\right), \tag{251}$$

$$J(\theta) = E_{y \sim \pi_\theta(y|x)}[r(y)], \tag{252}$$

其中，$d(\cdot)$是与域相关的距离度量；λ用于调控奖励对重构误差的敏感度；T_{cd}是一个双任务模型，用于从y和x_k中重建x_u。

算法 99. Contrastive Preference Optimization（CPO）

输入：前缀x，正样本续写y_1，负样本续写集合$\{y_2, \ldots, y_K\}$，参考模型π_{ref}，温度参数β。

输出：更新策略π_θ

目标函数：通过对比正样本（真实续写）与负样本（自动生成），使模型更倾向于生成高质量序列，目标函数具体如公式（253）所示：

$$\mathrm{L}_{CPO}(\pi_\theta, \pi_{ref}) = E_{y_i \sim A}\left[log \frac{exp\left(\beta\, log \frac{\pi_\theta(y_1|x)}{\pi_{ref}(y_1|x)}\right)}{\sum_{j=1}^{K} exp\left(\beta\, log \frac{\pi_\theta(y_j|x)}{\pi_{ref}(y_j|x)}\right)}\right], \tag{253}$$

其中，y_1代表真实续写（正样本），而y_2至y_K则为负样本，这些负样本是从分布A中采样获得的；β是一个温度超参数，用于调控对比强度；K表示总样本数，其中包含 1 个正样本和K-1 个负样本。

ConPO 的优势在于无需人工标注、计算成本较低，且能有效引入序列级监督，从而提升生成质量和指令遵循能力。然而，其性能极大地依赖于负样本的质量，若负样本中噪声过多，性能将受影响。

算法 100. Adaptive Design via Policy Tree（ADAPT）

输入：历史数据集、效用函数定义、树深度限制、最小节点样本数。

输出：优化后的策略树结构、各叶子节点对应的决策动作、预期效用估计值。

1) 策略树结构：策略树T由内部节点（分裂规则）和叶子节点（决策动作）组成。设第m个叶子节点的决策动作为a_m，其对应的数据子集为D_m。
2) 目标函数：算法最大化期望效用$U(T)$，定义如（254）所示：

$$U(T)=\sum_{m=1}^{M}\sum_{i\square D_m}E[Y_i(a_m)], \tag{254}$$

其中，M是叶子节点总数，$Y_i(a_m)$表示个体i在动作a_m下的结果变量，$E[\cdot]$表示期望值，通常通过历史数据估计。

3) 分裂准则：选择分裂特征j和阈值c以最大化分裂后的效用增益，具体如公式（255）所示：

$$\Delta U(j,c)=U(T_{left})+U(T_{right})-U(T_{parent}), \tag{255}$$

其中，T_{left}和T_{right}为分裂后的左右子树，T_{parent}为分裂前节点对应的子树。

4) 停止条件：当节点样本数低于预设阈值n_{min}或效用增益ΔU小于临界值ϵ时停止分裂，以防止过拟合。

ADAPT 的显著优势在于其高度的可解释性，能够灵活应对数据分布的变化，并展示透明的决策路径。然而，该模型对初始参数（如分裂阈值）较为敏感，在数据稀疏的情况下可能表现不稳定。此外，其计算复杂度会随着树深度的增加而显著上升，这可能会限制其在超大规模场景中的实际应用。

算法 101. Interactive Policy Optimization（IPO）

输入：初始策略π_θ；任务-环境对数据集$D=\{(q,e)\}$；组大小n，最大步数max_steps；超参数ε, β。

输出：更新策略π_θ

1) 概率比率计算：概率比率计算具体如公式（256）所示：

$$Pr_t(\theta)=\frac{\pi_\theta(\phi_t,a_t|\tau_{i,<t})}{\pi_{\theta old}(\phi_t,a_t|\tau_{i,<t})}, \tag{256}$$

其中，ϕ_t和a_t分别代表第t步的思考和动作，$\tau_{i,<t}$则是指轨迹中第t步之前的所有历史信息。

2) 目标函数：IPO 的目标函数具体如公式（257）所示：

$$J_{IPO}(\theta)=E\left[\frac{1}{n}\sum_{i=1}^{n}\frac{1}{|\tau_i|}\sum_{t=1}^{|\tau_i|}min\left[P_{r_t}(\theta)\widehat{A}_{i,t},\ clip(P_{r_t}(\theta),1-\varepsilon,1+\varepsilon)\widehat{A}_{i,t}\right]\right]-\beta D_{KL}[\pi_\theta\|\pi_{ref}], \tag{257}$$

其中，$\widehat{A}_{i,t}=\frac{r_i-\mu_r}{\sigma_r}$是组内归一化优势，$\mu_r$和$\sigma_r$分别代表组内奖励的均值和标准差，$\beta$是 KL 散度惩罚系数，旨在防止策略过度偏离参考模型。

IPO 的优势在于其能够通过在线交互自主探索环境，从而减少对人工先验知识的依赖，并在稀疏奖励条件下仍能有效优化策略，展现出卓越的泛化能力。然而，IPO 依赖于大量并行环境交互，导致计算成本较高。

算法 102. DreamBoothDPO

输入：超参数 $\lambda_1, \lambda_2, \lambda_3, \beta$

输出：更新策列 π_θ

1) 评分函数：综合图像相似度（IS）与文本相似度(TS)来评估图像x的质量，具体如公式（258）所示：

$$S(x)=\lambda\cdot TS(x)+(1-\lambda)\cdot IS(x), \tag{258}$$

其中，$S(x)$表示图像x的合成评分，λ是用于控制 TS 和 IS 重量的称重参数，$TS(x)=cos(CLIP\text{-}I(x),CLIP\text{-}T(c))$表示图像与文本提示之间的相似度，$IS(x)=\frac{1}{|C|}\sum_{x_i\square C}cos(CLIP\text{-}I(x),CLIP\text{-}I(x_i))$用于评估图像与参考概念图像集之间的相似程度。

2) 角度筛选：选取图像对，以提升或至少不显著影响 IS 和 TS 指标，具体如（259）所示：

$$C_1<atan2(\Delta IS,\Delta TS)<C_2, \tag{259}$$

其中：$\Delta IS=IS(x_w)-IS(x_l)$，$\Delta TS=TS(x_w)-TS(x_l)$；$atan2(\cdot)$是一个四象限反正切函数，用于返回角度值，$C_1$和$C_2$是控制优化方向的角度上下限。

算法 103. DreamDPO

输入：超参数$\lambda_1,\lambda_2,\lambda_3,\beta$

输出：更新策略π_θ

1) 成对噪声图像的构建方法具体如式（260）所示：

$$x_t^1=\alpha_t x_0+\sigma_t\epsilon^1,\quad x_t^2=\alpha_t x_0+\sigma_t\epsilon^2, \tag{260}$$

其中，x_0是渲染的 RGB 图像，α_t和σ_t是扩散过程中的超参数，ϵ^1和ϵ^2是两种不同的高斯噪声。

2) 由偏好驱动的损失函数如公式（261）所示，而经过分段优化梯度处理后得到的最终版本则如公式（262）所示：

$$L_{Reward}=-E_t\left[w(t)\left(\left\|\epsilon^{win}-\epsilon_\phi(x_t^{win};y,t)\right\|_2^2-\left\|\epsilon^{lose}-\epsilon_\phi\left(x_t^{lose};y,t\right)\right\|_2^2\right)\right], \tag{261}$$

$$\nabla_\theta L^{Reward^*}=\begin{cases}E_t\left[w(t)\cdot\epsilon_\phi(x_t^{win};y,t)\cdot\dfrac{\partial x}{\partial\theta}\right], & s_{gap}<\tau\\ E_t\left[w(t)\cdot\left(\Delta_t^{win}-\Delta_t^{lose}\right)\cdot\dfrac{\partial x}{\partial\theta}\right], & otherwise\end{cases}, \tag{262}$$

其中，$\Delta_t^{win}=\epsilon_\phi(x_t^{win};y,t)-\epsilon^{win}$和$\Delta_t^{lose}=\epsilon_\phi(x_t^{lose};y,t)-\epsilon^{lose}$，其中，$x_t^{win}$是更符合偏好样本，而$x_t^{lose}$则是与偏好样本匹配度较低的样本；$\epsilon_\phi$是扩散模型在预训练阶段的噪声预测器；$w(t)$是时间步长权重函数；$s_{gap}=r(x_t^{win},y)-r(x_t^{lose},y)$表示偏好分数的差距；$\tau$是阈值。

算法 104. DecoupledESC

输入：对话上下文$c_t=(u_0,a_0,\ldots,u_t)$，策略规划器模型LLM_{SP}，回复生成器模型LLM_{RG}。

输出：支持策略$s_t\sim LLM_{SP}(s|c_t)$，情感支持回复$a_t\sim LLM_{RG}(a|c_t,s_t)$。

1) 策略规划器训练：使用 DPO 优化策略选择，目标函数如公式（263）所示：

$$\mathrm{L}_{SP\text{-}dpo}=-E_{(c,s_c,s_r)\sim D_{SP\text{-}dpo}}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(s_c|c)}{\pi_{ref}(s_c|c)}-\beta\,log\frac{\pi_\theta(s_r|c)}{\pi_{ref}(s_r|c)}\right)\right], \tag{263}$$

其中，c为对话上下文；s_c为优选策略，s_r为拒绝策略；β为控制偏好强度的超参数。

2) 回复生成器训练：使用 DPO 优化回复质量，目标函数如公式（264）所示：

$$L_{RG\text{-}dpo}=-E_{(c,s,a_c,a_r)\sim D_{RG\text{-}dpo}}\left[\log\sigma\left(\beta\log\frac{\pi_\theta(a_c|c,s)}{\pi_{ref}(a_c|c,s)}-\beta\log\frac{\pi_\theta(a_r|c,s)}{\pi_{ref}(a_r|c,s)}\right)\right], \tag{264}$$

其中，a_c为优选回复，a_r为拒绝回复；s为策略输入。

DecoupledESC 通过任务解耦与分阶段优化，显著提升了情感支持的质量及策略多样性，有效缓解了偏好偏差与心理错误。然而，该方法依赖于高质量的解耦偏好数据构建，且尚未在大规模模型上验证其扩展性。

算法 105. Contrastive Preference Optimization（CPO）

输入：对话历史d，原始当前语句u_k，解析器DP，初始 DCM 模型DCM_β。

输出：优化后的 DCM 模型DCM_{β^*}，澄清后的语句u_c。

1) 偏好对构建：
   - 对每个对话历史d，DCM 生成多个澄清候选$\{u_c^j\}$；
   - 解析器计算每个澄清的贡献分数$e_c^j=\log p_\theta(y|d,u_c^j)$，其中$y$是真实关系；
   - 选取$e_c^+>e_k$的澄清作为正样本u_c^+，$e_c^-<e_k$的作为负样本u_c^-，其中e_k是原始语句的得分。
2) 贡献差距计算：定义贡献差距如公式（265）所示，反映正负样本对解析贡献的差异程度：

$$g=e_c^+-e_c^-. \tag{265}$$

3) 加权偏好优化损失函数：加权偏好优化损失函数如公式（266）和（267）所示：

$$L_{cpo}=-\frac{1}{N'}\sum_{i=1}^{N'}\left[\sigma(\mu g^i)\log\sigma(f(u_c^{i+},u_c^{i-},d^i))+(1-\sigma(\mu g^i))\log\sigma(f(u_c^{i-},u_c^{i+},d^i))\right], \tag{266}$$

$$f(u^+,u^-,d)=\eta\left(\log\frac{p_{\beta^*}(u^+|d)}{p_\beta(u^+|d)}-\log\frac{p_{\beta^*}(u^-|d)}{p_\beta(u^-|d)}\right), \tag{267}$$

其中，σ代表 sigmoid 函数，μ为缩放因子，η为 DPO 超参数，β^*是待优化的 DCM 参数，而β则是初始参数。

CaPO 通过引入贡献差距机制，有效提升了澄清生成的质量及其对解析器的适配性，显著降低了错误澄清所带来的负面影响。然而，该方法依然依赖于自动构建的偏好数据，其质量受制于初始澄清模型的性能。

算法 106. VideoDPO

输入：预训练视频扩散模型，文本提示集，OmniScore 评分模型。

输出：对齐后的视频扩散模型，偏好对齐后的生成视频。

1) OmniScore 评分系统：
   - 包括帧内质量、帧间质量和语义对齐；
   - 使用预训练模型对各维度评分并加权聚合。

2) 偏好对生成：

- 对每个提示 p，生成 N 个视频$\{v_1,\ldots,v_N\}$，计算每个视频的 OmniScore $s_i=S(v_i,p)$。
- 选择得分最高和最低的视频构成偏好对(v^W,v^L)。

3) 重加权损失函数：定义样本频率$p(s)$，计算偏好对概率如（268）所示，定义重加权因子如（269）所示，从而得到损失函数如（270）所示：

$$prob(s^W,s^L)=\sqrt{p(s^W)\cdot p(s^L)}, \tag{268}$$

$$w_{pair}=\left(\frac{\beta}{prob(s^W,s^L)}\right)^{\alpha}, \tag{269}$$

$$L_{video}=L_{DPO}(p,v^W,v^L)\cdot w_{pair}, \tag{270}$$

其中β为常数，α为超参数，L_{DPO}为原始 DPO 损失。

算法 107. Knowledge-aware Preference Optimization（KnowPO）

输入：问题q，检索上下文τ（可能包含冲突或无关知识），模型参数知识α（即无上下文时的回答）。

输出：模型生成的回答y

1) 指令微调阶段：通过使用构建的冲突上下文和无关上下文对模型进行监督微调，使其能够在不同情境中准确选择合适的知识源。

2) 偏好优化阶段：采用 DPO 目标函数对模型进行优化，使其更倾向于生成正样本回答，从而避免负样本错误。KnowPO 的目标函数如公式（271）所示：

$$\mathrm{L}_{DPO}(\pi_\theta;\pi_{ref})=-E_{(x,y_w,y_l)\sim D}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\,log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right], \tag{271}$$

其中，π_θ代表待优化的策略模型；π_{ref}为参考模型；x为输入问题与上下文拼接而成的提示；y_w为正样本回答（即理想行为）；y_l为负样本回答（即错误行为）；β为控制偏离参考模型程度的超参数；σ为 sigmoid 函数。

3) 偏好数据构建：

- 正样本：在冲突情境中运用冲突知识进行回应，在无关情境中则使用参数知识进行回答。
- 负样本：模拟两类错误类型：一是上下文忽略，即在冲突情境中仍然采用参数知识；二是上下文过度包含，即在无关情境中错误地运用上下文信息。

KnowPO 通过系统构建知识冲突数据集并引入负信号优化，显著增强了模型在 RAG 中的知识选择能力，并在多个数据集和基模型上展现出卓越的泛化性能。然而，该方法依赖于高质量的知识冲突数据构建，尽管偏好平衡策略行之有效，但对超参数的敏感性较高。

算法 108. AnnoDPO

输入：超参数 $\lambda_1,\lambda_2,\lambda_3,\beta$

输出：更新策略 π_θ

1) 注释预测损失：在多标签分类任务中，采用二元交叉熵损失函数进行计算，具体如式（272）所示：

$$L_{AP}=-\sum_{j\square N}\left(y_j^A\,log\,(p_j^A)+(1-y_j^A)\,log\,(1-p_j^A)\right),\tag{272}$$

其中，N表示注释词汇表的规模，即用于训练的基因本体论术语总数；y_j^A为二元变量（取值为 0 或 1），用于标识第j个特定 GO 术语是否为该蛋白质的真实注释；p_j^A表示预测概率，即模型为蛋白质分配第j个 GO 术语注释的可能性，其取值范围介于 0 至 1 之间。

2) 序列-注释对比损失：通过引入序列-注释对比损失机制，实现蛋白质序列嵌入与功能注释嵌入之间的有效双向对齐，具体如公式（273）所示：

$$L_{SAC}=-\frac{1}{2}\sum_{(i,j)}\left(log\frac{exp\,(h_i^S\cdot\frac{h_j^A}{\tau})}{\sum_k exp\,(h_i^S\cdot\frac{h_k^A}{\tau})}+log\frac{exp\,(h_i^S\cdot\frac{h_j^A}{\tau})}{\sum_k exp\,(h_k^S\cdot\frac{h_j^A}{\tau})}\right),\tag{273}$$

其中，(i,j)构成一个正样本对，其中i代表蛋白质序列的索引，而j对应于该序列的正确功能注释；h_i^S是由 ESM-C 编码器为第i个蛋白质序列生成的特征向量；h_j^A是由蛋白质 BERT 编码器为第j个功能注释所生成的特征向量；k作为求和索引，遍历第一部分中随机抽取的负样本标注数据集，以及第二部分中随机抽取的负样本序列数据集。

3) DPO 阶段：通过引入 DPO 损失函数，直接优化模型对真实标注的偏好，有效缓解奖励模型的不稳定性，具体如公式（274）所示：

$$L_{DPO}(\pi_\theta;\pi_{ref})=-E_{(x,y_w,y_l)}\left[log\,\sigma\left(\beta\,log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\beta\,log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)\right],\tag{274}$$

其中，β是温度超参数(β>0)，用于控制参考策略π_{ref}偏差程度的惩罚强度；σ代表 Sigmoid 函数，该函数将输入值映射至（0,1）区间，用于表示偏好值y_w相对于y_l的优越概率。

算法 109. DenseDPO

输入: Hyperparameters $\lambda_1,\lambda_2,\lambda_3,\beta$

输出: Updated policy π_θ

1) 结构化视频的引导式采样：给定真实视频 x，通过添加噪声可生成两个对齐视频，具体如（275）所示：

$$x_n^0=(1-\eta)x+\eta\epsilon^0,\quad x_n^1=(1-\eta)x+\eta\epsilon^1,\tag{275}$$

其中，x是真实视频；η表示噪声比，控制结构相似性；ϵ^0和ϵ^1是独立的高斯噪声，通常服从正态分布$N(0,I)$；x_n^0和x_n^1是经过部分降噪处理的视频片段，后续将用于去噪处理，以生成对齐的视频对。

2) 分段偏好标注：将每个视频划分为F个长度为s的片段，并获取相应的分段偏好标签。DenseDPO 的目标函数如公式（276）所示：

$$L(\theta)=-E_{(c,x^0,x^1),t,c\in N(0,1)}\,log\,\sigma\left(-\beta\sum_{f=1}^{F}l(x_f^0,x_f^1)\cdot\left(s(x^0,c,t,\theta)_f-s(x^1,c,t,\theta)_f\right)\right),\tag{276}$$

其中，C表示文本条件，(x^0,x^1)是视频对，$l(x_f^0,x_f^1)$表示第f个片段的偏好标签（+1 或-1），$s(x^1,c,t,\theta)_f$

表示第f段的隐式奖励值，具体如公式（277）所示：

$$s(x^*,c,t,\theta)=\|(\epsilon^*-x^*)-\pi_\theta(x_t^*,t,c)\|_2^2-\|(\epsilon^*-x^*)-\pi_{ref}(x_t^*,t,c)\|_2^2, \tag{277}$$

其中，x^*表示视频样本（x^0或x^1），$x_t^*=(1-t)x^*+t\epsilon^*$是噪声潜在表示，$\epsilon^*\sim N(0,I)$是噪声向量。

算法 110. Temporal Preference Optimization（TemPO）

输入：视频V，文本查询q，参考模型π_{ref}，偏好数据集$D=\{(V,q,r^+,r^-)\}$。

输出：更新后的策略π_θ

1) 偏好数据构建：构建四元组(V,q,r^+,r^-)，其中V是视频，q是查询，r^+是偏好响应（基于相关视频片段生成），r^-是非偏好响应（基于无关或残缺视频片段生成）。
2) 训练目标：采用 DPO 损失函数，具体如公式（278）所示，并结合监督微调（SFT）目标，具体如公式（279）所示，以优化模型参数，最终得到综合损失函数，具体形式如公式（280）所示：

$$L_{DPO}=-E_{(V,q,r^+,r^-)\sim D}\left[log\,\sigma\left(\beta\left(log\frac{\pi_\theta(r^+|V,q)}{\pi_{ref}(r^+|V,q)}-log\frac{\pi_\theta(r^-|V,q)}{\pi_{ref}(r^-|V,q)}\right)\right)\right], \tag{278}$$

$$L_{SFT}=-E_{(V,q,r^+,r^-)\sim D}\,log\,\pi_\theta(r^+|V,q), \tag{279}$$

$$L=L_{DPO}+\alpha L_{SFT}, \tag{280}$$

其中，π_θ代表待优化的视频-LMM 模型；π_{ref}指的是参考模型；β是用于控制偏好强度的超参数；α用于平衡 DPO 与 SFT 损失之间的权重；σ则表示 sigmoid 函数。

TemPO 凭借自训练机制和双粒度时序偏好设计，显著增强了模型对长视频中局部及全局时序关系的理解能力。然而，TemPO 依然依赖于外部语言模型进行数据生成与过滤，这一过程可能会引入潜在偏差。

算法 111. Triple Preference Optimization（TriplePO）

输入：提示x，黄金响应y_{gold}，优选响应y_w，拒绝响应y_l，策略模型π_θ，超参数：β，α。

输出：更新后的策略π_θ

1) 偏好优化损失：TriplePO 的偏好优化损失如公式（281）所示：

$$\mathrm{L}_{preference}=-E_{(x,y_w,y_l)\sim D}\left[log\,\sigma\left(\beta\,log\,\pi_\theta(y_w|x)-\beta\,log\,\pi_\theta(y_l|x)\right)\right], \tag{281}$$

其中，x代表输入提示，y_w和y_l分别表示偏好响应（优选和拒绝），π_θ为策略模型，β为温度参数，σ为 sigmoid 函数。

2) 行为克隆损失：TriplePO 的行为克隆损失如公式（282）所示：

$$\mathrm{L}_{reference}=-E_{(x,y_{gold})\sim D}\left[log\,\pi_\theta(y_{gold}|x)\right], \tag{282}$$

其中，y_{gold}代表黄金响应，即最优响应。

3) 目标函数构成：TriplePO 损失函数由两部分组成：偏好优化损失$L_{preference}$和行为克隆损失$L_{reference}$，通过超参数α进行加权组合，具体如公式（283）所示：

$$L_{TriplePO}=L_{preference}+\alpha L_{reference}, \tag{283}$$

TriplePO 的单步优化框架能够同步增强指令遵循和推理能力，即便在数据量有限或存在噪声的环境中，其表现依然稳健。然而，TriplePO 对三种偏好响应的质量要求颇高，导致数据构造的成本相对较大。

算法 112. Weak-to-Strong Preference Optimization（WSPO）

输入：强参考模型π_{ref}^{strong}，弱对齐模型π_{r}^{weak}，弱参考模型π_{ref}^{weak}，单响应数据集$D=\{(x,y)\}$，超参数γ。

输出：更新后的策略π_θ

最小化强模型与弱模型在对齐前后对数概率比之间的差异，使强模型在保持自身知识的基础上，吸收弱模型的对齐行为。目标函数的具体形式如公式（284）所示：

$$L_{WSPO}=E_{(x,y)\sim D}\left[\frac{1}{|y|}\left\|\gamma\, log\frac{\pi_{\theta}^{strong}(y|x)}{\pi_{ref}^{strong}(y|x)}-log\frac{\pi_{r}^{weak}(y|x)}{\pi_{ref}^{weak}(y|x)}\right\|_2^2\right], \tag{284}$$

其中，π_{θ}^{strong}代表待优化的强模型，π_{ref}^{strong}为强参考模型，π_{r}^{weak}为弱对齐模型，π_{ref}^{weak}为弱参考模型，γ则是用于控制对齐强度的超参数。

WSPO 的优势在于其训练效率极高、无需依赖成对偏好数据、对齐效果显著，并且具备出色的泛化能力，尤其在强模型上展现出对齐能力的“放大效应”。然而，该方法仍需依赖弱模型的对齐质量，且超参数γ的调优对结果具有显著影响。

算法 113. Length-Desensitized Direct Preference Optimization(LD-DPO)

输入：策略模型 π_θ, 参考模型 π_{ref}, 偏好数据集 $D=(x,y_w,y_l)$, 超参数 α,β.

输出：更新策略 π_θ

1) 公共长度定义：对于长度分别为 l_w 和 l_l 的偏好数据对 (y_w,y_l)，其公共长度的定义如式（285）所示：

$$l_p=min(l_w,l_l), \tag{285}$$

其中，l_p为公共长度，即优选和劣选响应中较短的长度。

2) 重新参数化似然：原始似然如式（286）所示，LD-DPO 将其修改为如式（287）所示：

$$\pi_\theta(y|x)=\prod_{i=1}^{l}p(y_i|x,y_{<i}), \tag{286}$$

$$\hat{\pi}_\theta(y|x)=\prod_{i=1}^{l}p^{\alpha}(y_i|x,y_{<i})\cdot\prod_{i=1}^{l_p}p^{1-\alpha}(y_i|x,y_{<i}), \tag{287}$$

其中，$p(y_i|x,y_{<i})$是模型在位置i生成y_i的条件概率；$\alpha\in[0,1]$为长度解耦强度参数，用于调控对

超出公共长度部分的惩罚力度。当α=1时，LD-DPO 退化为原始 DPO；当α=0时，仅公共长度部分参与似然计算。

LD-DPO 的优点在于能有效控制模型输出长度，提升响应质量与推理能力，且在多个基准测试中显著优于 DPO 及其变体。然而，该方法引入的超参数α需要根据模型能力调整，较弱模型需更小的α，增加了调优成本；此外，它仍依赖 DPO 的基本框架，未根本解决其依赖高质量偏好数据的问题，且在安全性与多轮对话评估方面仍有待验证。

算法 114. Mask Dirct Preference Optimization (Mask-DPO)

输入：提示x，策略模型π_θ，参考模型π_{ref}，句子级别幻觉标注器A，超参数β，采样数量K。

输出：更新策略 π_θ

1) 细粒度偏好数据构建：

- 对于每个提示x，采集多个响应，并利用句子级别幻觉标注器A对每个句子s_i进行标注，$a_i\in\{0,1\}$以区分其中a_i=0代表无幻觉，a_i=1代表有幻觉。
- 计算每个响应的真实性得分：$\frac{\sum(1-a_i)}{N}$，并据此构建偏好对(yw,yl)。

2) 掩码机制与优化目标：定义掩码后的 KL 散度项如（288）和（289）所示，从而得到优化目标如（290）所示：

$$\mathrm{M}_w=\sum_i I(a_i=0)\,log\frac{\pi_\theta(s_i|x,s_{<i})}{\pi_{ref}(s_i|x,s_{<i})}, \tag{288}$$

$$\mathrm{M}_l=\sum_j I(a_j=1)\,log\frac{\pi_\theta(s_j|x,s_{<j})}{\pi_{ref}(s_j|x,s_{<j})}, \tag{289}$$

$$\mathrm{L}_{Mask\text{-}DPO}=-E[log\,\sigma(\beta\mathrm{M}_w-\beta\mathrm{M}_l)], \tag{290}$$

其中，β是控制正则化强度的超参数；σ是 sigmoid 函数；$I(\cdot)$作为指示函数，当条件成立时其值为 1，否则为 0。

Mask-DPO 在提升模型事实性方面表现尤为出色，尤其在跨领域泛化能力上显著优于传统 DPO 方法。其优势包括细粒度监督带来的精准对齐、对幻觉内容的有效抑制以及出色的泛化性能。然而，该方法依赖于外部句子级别标注器，可能存在标注质量与模型泛化之间的耦合风险，同时在更细粒度的幻觉控制方面仍有进一步提升的空间。

算法 115. Sem-DPO

输入：原始提示x，偏好提示对(y_w,y_l)，参考策略模型π_{ref}，嵌入模型e_φ，超参数 α,β。

输出：更新策略π_θ

1) 语义权重定义：对于每个训练三元组(x,y_w,y_l)，定义语义一致性权重如（291）所示：

$$W(x,y_w)=exp\left(-\alpha\cdot d_{cos}\left(e_\varphi(x),e_\varphi(y_w)\right)\right), \tag{291}$$

其中，y_w代表被偏好的优化提示，$e_\varphi(\cdot)$是预训练的嵌入模型，d_{cos}表示余弦距离，α作为控制语义权强度的一个超参数。

2) Sem-DPO 损失函数：Sem-DPO 的损失函数如（292）所示：

$$L_{Sem\text{-}DPO}(\pi_\theta)=-E_{(x,y_w,y_l)\sim D}\left[W(x,y_w)\cdot \log\sigma(\Delta_\theta)\right], \tag{292}$$

其中，$\Delta_\theta=\beta\left(\log\frac{\pi_\theta(y_w|x)}{\pi_{ref}(y_w|x)}-\log\frac{\pi_\theta(y_l|x)}{\pi_{ref}(y_l|x)}\right)$，$\sigma$代表 sigmoid 函数，$\beta$则是用于调控偏好强度的超参数。

Sem-DPO 在保留 DPO 简洁高效特性的基础上，有效缓解了语义漂移问题，并通过理论分析为语义一致性提供了明确的边界保障，展现出显著的实用性和可解释性。然而，其性能高度依赖于外部嵌入模型的质量，且超参数α的调优对结果具有显著影响，可能需要在不同的任务或数据集上进行重新校准，这在一定程度上限制了其泛化能力。此外，该方法尚未完全解决风格过拟合等非语义层面的对齐问题。